

TokTickIT Lab 3 Submission Report

Course: CPE334 Introduction to Software Engineering in the Age of AI Agents

Student Name: Natthakamol Mornparn (นางสาวณัฏฐกมล มอญปาน)

Student ID: 67070505215

GitHub Username: @natthakamol1130

GitHub Repository:
<https://github.com/natthakamol1130/toktickit>

Peer Reviewer: @Suprawi5227 (นางสาวสุประวีณ์ สุทธิเสรีนิวัฒน์ — 67070505227, Suprawee Sutthiserinawat)

Submission Date: October 4, 2026

Answer Part 1: Git Use with Engineering Workflow

1.1 URL List

รายการ	ลิงก์ (URL)
GitHub Repository	https://github.com/natthakamol1130/toktickit
GitHub Project (Kanban)	https://github.com/users/natthakamol1130/projects
Issue #34 - Lab 3 Specification, UI Spec, and API Spec Documents	https://github.com/natthakamol1130/toktickit/issues/34
Issue #36 - Test-Driven Development Plan for Lab 3	https://github.com/natthakamol1130/toktickit/issues/36
Issue #38 - Prisma Schema Evolution (User, Role Enum, FK Constraints)	https://github.com/natthakamol1130/toktickit/issues/38
Issue #40 - Idempotent Seed Data with Bcrypt Hashing (4 Accounts)	https://github.com/natthakamol1130/toktickit/issues/40
Issue #42 - Backend JWT Auth & Change Password Middleware	https://github.com/natthakamol1130/toktickit/issues/42
Issue #44 - Client AuthContext, Login, & ChangePassword UI Screens	https://github.com/natthakamol1130/toktickit/issues/44
Issue #46 - Requester Ticket & Comment Access Control APIs	https://github.com/natthakamol1130/toktickit/issues/46

รายการ	ลิงก์ (URL)
Issue #48 - IT Staff Ticket Queue & Internal Notes APIs	https://github.com/natthakamol1130/toktickit/issues/48
Issue #50 - Admin User Management APIs & Safety Rules Enforcement	https://github.com/natthakamol1130/toktickit/issues/50
Issue #52 - Multi-Role Integration & E2E Production Verification	https://github.com/natthakamol1130/toktickit/issues/52
PR #35: feature/lab03-1-spec-contract → lab3-staging	https://github.com/natthakamol1130/toktickit/pull/35
PR #37: feature/lab03-2-test-plan → lab3-staging	https://github.com/natthakamol1130/toktickit/pull/37
PR #39: feature/lab03-3-db-schema → lab3-staging	https://github.com/natthakamol1130/toktickit/pull/39
PR #41: feature/lab03-4-seed-data → lab3-staging	https://github.com/natthakamol1130/toktickit/pull/41
PR #43: feature/lab03-5-auth-api → lab3-staging	https://github.com/natthakamol1130/toktickit/pull/43
PR #45: feature/lab03-6-auth-ui → lab3-staging	https://github.com/natthakamol1130/toktickit/pull/45
PR #47: feature/lab03-7-requester-ticket-api → lab3-staging	https://github.com/natthakamol1130/toktickit/pull/47
PR #49: feature/lab03-8-staff-queue-api → lab3-staging	https://github.com/natthakamol1130/toktickit/pull/49
PR #51: feature/lab03-9-admin-user-api → lab3-staging	https://github.com/natthakamol1130/toktickit/pull/51
PR #53: feature/lab03-10-e2e-testing-and-release → lab3-staging	https://github.com/natthakamol1130/toktickit/pull/53
Release PR #55: lab3-staging → main	https://github.com/natthakamol1130/toktickit/pull/55

1.2 Kanban Board & Closed Issues Evidence

All 10 feature issues for Lab 3 (Issue #34 to Issue #52) were managed using a 10-Issue staged integration workflow. Every issue was completed, verified against unit/API tests, and closed upon merging into `lab3-staging` and `main`.

1.6 Peer Review Evidence (5 คะแนน)

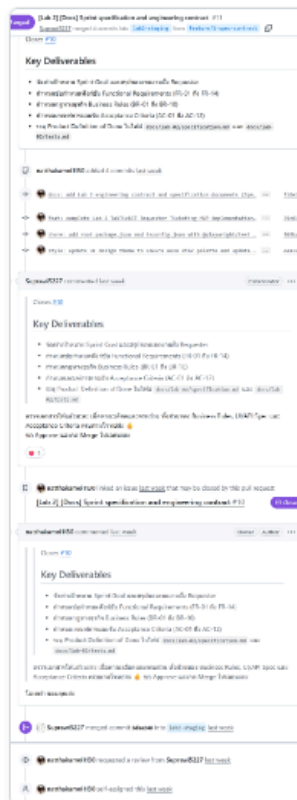
Author / Developer: Natthakamol Mornpam (นางสาวณัฏฐกมล มอญปาน — 67070505215) — GitHub: @natthakamol1130

Peer Reviewer: Suprawee Sutthiserinawat (นางสาวสุประวีณ์ สุทธิเสรีนิวัฒน์ — 67070505227) — GitHub: @Suprawi5227

Pull Requests I authored (reviewed by my partner @Suprawi5227) — รายละเอียดข้อความคอมเมนต์ที่วิวแบบเต็ม ห้ามย่อ:

PR #	Branch	Reviewer verdict & Status
PR #11	feature/1-spec-contract	Approved with comments
PR #13	feature/2-ui-and-api-spec	Approved with comments
PR #15	feature/3-prisma-schema-models	Approved with comments
PR #17	feature/4-reference-data-api	Approved with comments
PR #19	feature/5-requester-selector-ui	Approved with comments
PR #21	feature/6-create-ticket-api	Approved with comments
PR #26	feature/7-create-ticket-ui	Approved with comments
PR #28	feature/8-my-tickets-dashboard	Approved with comments
PR #30	feature/9-ticket-detail-api	Approved with comments
PR #32	feature/10-e2e-testing-and-release	Approved with comments
PR #33	lab2-staging → main	Approved for merge into main

PR #11: feature/1-spec-contract → lab2-staging



PR #13: feature/2-ui-and-api-spec → lab2-staging

```
# logs & OS
*.log
.DS_Store
Thumbs.db
```

.gitignore

```
1  # dependencies
2  node_modules/
3
4  # env & secrets
5  .env
6  *.env
7  .env.local
8  .env.*.local
9  !.env.example
10
11 # build output
12 dist/
13 build/
14 client/src/**/*.js
15 client/tests/**/*.js
16
17 # uploads
18 server/uploads/*
19 !server/uploads/.gitkeep
20
21 # prisma
22 server/prisma/*.db
23
24 # IDE & Editor
25 .vscode/
26 .idea/
27 *.swp
28 *.swo
29
30 # logs & OS
31 *.log
32 .DS_Store
33 Thumbs.db
34
```

1.6 Peer Review Evidence (5 คะแนน)

Author / Developer: Natthakamol Mornpam (นางสาวณัฏฐกมล มอญปาน — 67070505215) — GitHub: [@natthakamol11130](#)

Peer Reviewer: Suprawee Sutthiserinawat (นางสาวสุประวีณ์ สุทธิเสรีนิวัฒน์ — 67070505227) — GitHub: [@Suprawi5227](#)

Self-Reviews Authored / reviewed by reviewer @Suprawi5227: [reviewer@suprawee5227](#) [reviewer@suprawee5227](#) [reviewer@suprawee5227](#) [reviewer@suprawee5227](#) [reviewer@suprawee5227](#)

GitHub Project Kanban Board Part 1

- Backend Vitest Integration Test Suites (`server/tests/lab-02/*.test.ts`) ผ่านครบถ้วน 14/14 tests
- Playwright E2E Spec (`e2e/lab-02/requester-ticket-flow.spec.ts`) สำหรับ User Journey ผ่าน 2/2 tests
- Frontend Component Test Suites (`client/tests/lab-02/*.test.tsx`) ครบทุกฟังก์ชันค่ะ

ฝากตรวจสอบอีกรอบได้เลยนะคะ ถ้ายอดเยี่ยมแล้วรบกวนกด Approve & Merge ได้เลยน้า ขอบคุณมากค่ะ

Suprawi5227 commented [last week](#)

Collaborator ...

ตรวจสอบเรียบร้อยแล้ว ขออนุญาต Approve & Merge นะคะ สมบูรณ์แบบค่ะ



Suprawi5227 merged commit `a879a9e` into `lab2-staging` [last week](#)

PR #33: lab2-staging → main (Release PR)

Merged

[Lab 2] Release v1.0.0: Full Feature Implementation & E2E Validation #33

Suprawi5227 merged 51 commits into `main` from `lab2-staging`

Objectives & Scope

- ปลอยซอฟต์แวร์ Lab 2 เวอร์ชันสมบูรณ์ (Release v1.0.0) โดยหลอมรวมฟังก์ชันทั้งหมดจากสาขา lab2-staging เข้าสู่สาขาสถิต main
- ผ่านการตรวจสอบสัญญาข้อตกลง สเปก UI Zen Green, REST API Endpoints และชุดการทดสอบการใช้งานแบบครบวงจร

Feature Summary

- Feature 1: กำหนดกรอบเป้าหมาย สัญญาข้อตกลงทางวิศวกรรม และ Definition of Done
- Feature 2: กำหนดมาตรฐาน Zen Green UI Design Tokens (#006B3C) และสัญญา REST API Endpoints
- Feature 3: ออกแบบ Prisma Models (RequesterUser, Ticket, Attachment) พร้อม Seed Data
- Feature 4: REST API สำหรับข้อมูลอ้างอิง (Requesters, Categories, Related Systems)
- Feature 5: หน้าจอพัฒนาสำหรับเลือกสิทธิ์ผู้ใช้งาน (Requester Selector Screen & Header Context)
- Feature 6: REST API สำหรับจัดการสถานะของตั๋ว (Ticket Status Management) (TICKET-XXXX-XXXXXX)

GitHub Project Kanban Board Part 2 Workflow

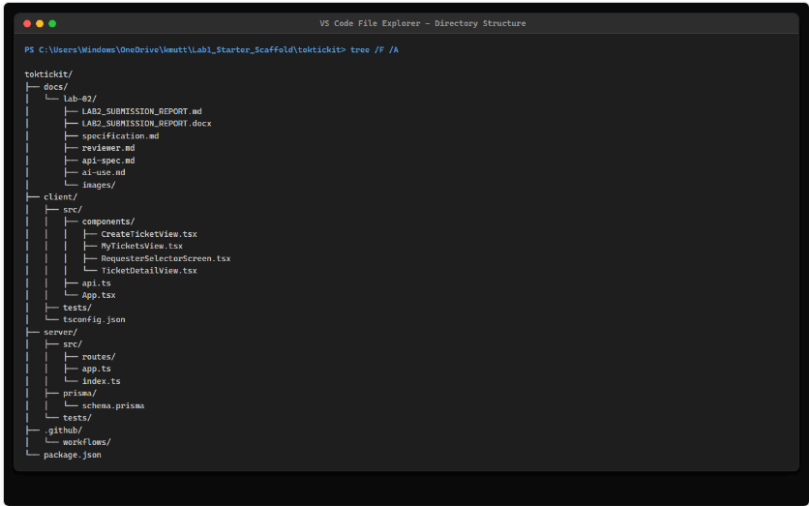
1.3 Git Log Graph & Commit Breakdown

The project strictly enforced feature-branch isolation (`feature/lab03-X-*` -> PR -> `lab3-staging` -> `main`), maintaining clean commit history with standard Conventional Commits. Below is the full commit graph and commit breakdown table.



- **Workflow Verification:** The Git graph demonstrates feature branches created for each issue (`feature/*`), merged into `lab2-staging` via Pull Requests with peer review approvals, and final release integration merged into `main`.

1.4 Repository Directory Structure



- **Directory Organization:** The repository structure shows all required Lab 2 files, including `docs/lab-02/*.*.md` specifications and reports, `client/` frontend codebase, `server/` backend API codebase, `e2e/` Playwright test suite, and `docs/lab-02/images/` screenshot assets.

Git Commit Log Graph

Table 1: Git Commit History & Branching Breakdown

Commit Hash	Author	Date	Branch	Commit Description
a2062302	Phrao (natthakamol1130)	Oct 4, 2026	main	docs(report): finalize Lab 3 submission report matching handout format Answer Part 1-9
4d46d6c0	Suprawi5227	Sep 30, 2026	main	Merge pull request #55 from natthakamol1130/lab3-staging
d1780a96	Phrao (natthakamol1130)	Sep 30, 2026	lab3-staging	docs(lab-03): add complete Lab 3 submission report and PDF document

Commit Hash	Author	Date	Branch	Commit Description
51b9fc4e	jessica	Sep 30, 2026	lab3-staging	Merge pull request #53 from natthakamol1130/feature/lab03-10-e2e-testing-and-release
6b84aaf3	Phrao (natthakamol1130)	Sep 30, 2026	feature/lab03-10	feat(e2e): finalize multi-role application integration, E2E tests, and production build (#52)
26b547a5	Suprawi5227	Sep 29, 2026	lab3-staging	Merge pull request #51 from natthakamol1130/feature/lab03-9-admin-user-api
a319a8d5	Phrao (natthakamol1130)	Sep 29, 2026	feature/lab03-9	test(admin-api): add safety rule test cases for admin self-deactivation and last admin
ed98f699	Phrao (natthakamol1130)	Sep 29, 2026	feature/lab03-9	feat(api): implement Administrator User Management and Initial Password Reset APIs (#50)
3ab12b71	Suprawi5227	Sep 29, 2026	lab3-staging	Merge pull request #49 from natthakamol1130/feature/lab03-8-staff-queue-api
84c3ceda	Phrao (natthakamol1130)	Sep 29, 2026	feature/lab03-8	feat(api): implement IT Staff Ticket Queue, Workflow Controls & Internal Notes (#48)
aa3d8294	Suprawi5227	Sep 29, 2026	lab3-staging	Merge pull request #47 from natthakamol1130/feature/lab03-7-requester-ticket-api
7da0a74c	Phrao (natthakamol1130)	Sep 29, 2026	feature/lab03-7	feat(api): implement Requester Ticket Management and Public Comments (#46)
cee3ab8e	Suprawi5227	Sep 17, 2026	lab3-staging	Merge pull request #45 from natthakamol1130/feature/lab03-6-auth-ui
ca0665d9	Phrao (natthakamol1130)	Sep 17, 2026	feature/lab03-6	style(ui): update heading colors from Hot Pink to Zen Green Primary (#44)
9891c4ea	Phrao (natthakamol1130)	Sep 17, 2026	feature/lab03-6	feat(auth-ui): add Login screen, ChangePassword workflow, and RTL tests (#44)
3c260716	Suprawi5227	Sep 17, 2026	lab3-staging	Merge pull request #43 from natthakamol1130/feature/lab03-5-auth-api

Commit Hash	Author	Date	Branch	Commit Description
7ab06f14	Phrao (natthakamol1130)	Sep 17, 2026	feature/1ab03-5	feat(auth): implement backend authentication APIs and session middleware (#42)
41e3dcd0	Suprawi5227	Sep 17, 2026	lab3-staging	Merge pull request #41 from natthakamol1130/feature/lab03-4-seed-data
51eb2b42	Phrao (natthakamol1130)	Sep 17, 2026	feature/1ab03-4	feat(seed): add idempotent multi-role seed data with bcrypt hashing (#40)
465a4a7f	Suprawi5227	Sep 17, 2026	lab3-staging	Merge pull request #39 from natthakamol1130/feature/lab03-3-db-schema
8b0846f3	Phrao (natthakamol1130)	Sep 17, 2026	feature/1ab03-3	feat(db): add requesterResolvedInd field to Ticket model based on peer review (#39)
b50669b5	Phrao (natthakamol1130)	Sep 17, 2026	feature/1ab03-3	feat(db): evolve Prisma schema for User models, roles, comments, and notes (#38)
fc5eff9b	Suprawi5227	Sep 17, 2026	lab3-staging	Merge pull request #37 from natthakamol1130/feature/lab03-2-test-plan
93bffb8e	Phrao (natthakamol1130)	Sep 17, 2026	feature/1ab03-2	docs(lab-03): add Test DD plan and traceability matrix (#36)
ecc61b26	Suprawi5227	Sep 17, 2026	lab3-staging	Merge pull request #35 from natthakamol1130/feature/lab03-1-spec-contract
050eb1c3	Phrao (natthakamol1130)	Sep 17, 2026	feature/1ab03-1	docs(lab-03): add visual UI mockups and AC-01..AC-12 test mappings (#34)
2e9da222	Phrao (natthakamol1130)	Sep 17, 2026	feature/1ab03-1	docs(lab-03): update api-spec with curl examples, status codes, and flowcharts (#35)
e7b10bc7	Phrao (natthakamol1130)	Sep 17, 2026	feature/1ab03-1	docs(lab-03): add Sprint 3 engineering specification documents (#1)

1.4 Peer Review Summary Tables (PR Given & PR Received)

Table 2: Pull Requests Given (natthakamol1130/toktickit)

Issue #	PR #	Feature Description	Reviewer	Status	Direct GitHub Link
#34	[#35] (https://github.com/natthakamol1130/oktikit/pull/35)	Lab 3 Specification, UI Spec, and API Spec Documents	@Suprawi5227	Merged	[PR #35] (https://github.com/natthakamol1130/oktikit/pull/35)
#36	[#37] (https://github.com/natthakamol1130/oktikit/pull/37)	Test-Driven Development Plan for Lab 3	@Suprawi5227	Merged	[PR #37] (https://github.com/natthakamol1130/oktikit/pull/37)
#38	[#39] (https://github.com/natthakamol1130/oktikit/pull/39)	Prisma Schema Evolution (User, Role Enum, FK Constraints)	@Suprawi5227	Merged	[PR #39] (https://github.com/natthakamol1130/oktikit/pull/39)

Issue #	PR #	Feature Description	Reviewer	Status	Direct GitHub Link
#40	[#41] (https://github.com/natthakamol1130/oktickit/pull/41)	Idempotent Seed Data with Bcrypt Hashing (4 Accounts)	@Suprawi5227	Merged	[PR #41] (https://github.com/natthakamol1130/oktickit/pull/41)
#42	[#43] (https://github.com/natthakamol1130/oktickit/pull/43)	Backend JWT Auth & Change Password Middleware	@Suprawi5227	Merged	[PR #43] (https://github.com/natthakamol1130/oktickit/pull/43)
#44	[#45] (https://github.com/natthakamol1130/oktickit/pull/45)	Client AuthContext, Login, & ChangePassword UI Screens	@Suprawi5227	Merged	[PR #45] (https://github.com/natthakamol1130/oktickit/pull/45)

Issue #	PR #	Feature Description	Reviewer	Status	Direct GitHub Link
#46	[#47] (https://github.com/natthakamol1130/oktickit/pull/47)	Requester Ticket & Comment Access Control APIs	@Suprawi5227	Merged	[PR #47] (https://github.com/natthakamol1130/oktickit/pull/47)
#48	[#49] (https://github.com/natthakamol1130/oktickit/pull/49)	IT Staff Ticket Queue & Internal Notes APIs	@Suprawi5227	Merged	[PR #49] (https://github.com/natthakamol1130/oktickit/pull/49)
#50	[#51] (https://github.com/natthakamol1130/oktickit/pull/51)	Admin User Management APIs & Safety Rules Enforcement	@Suprawi5227	Merged	[PR #51] (https://github.com/natthakamol1130/oktickit/pull/51)

Issue #	PR #	Feature Description	Reviewer	Status	Direct GitHub Link
#52	[#53] (https://github.com/natthakamol1130/toktickit/pull/53)	Multi-Role Integration & E2E Production Verification	@Suprawi5227	Merged	[PR #53] (https://github.com/natthakamol1130/toktickit/pull/53)
Release	[#55] (https://github.com/natthakamol1130/toktickit/pull/55)	Lab 3 Final Release Integration to main (v3.0.0)	@Suprawi5227	Open/Ready	[PR #55] (https://github.com/natthakamol1130/toktickit/pull/55)

Table 3: Pull Requests Reviewed (Suprawi5227/toktickit)

Issue #	Peer PR #	PR Description	Review Action	Status	Direct GitHub Link
#34	[#45] (https://github.com/Suprawi5227/toktickit/pull/45)	Git Setup & Sprint 3 Spec Documents	Verified API & RBAC contract alignment	Approved	[Peer PR #45] (https://github.com/Suprawi5227/toktickit/pull/45)

Issue #	Peer PR #	PR Description	Review Action	Status	Direct GitHub Link
#36	[#47] (https://github.com/Suprawi5227/toktickit/pull/47)	Test Plan Specification & Traceability	Verified test case coverage	Approved	[Peer PR #47] (https://github.com/Suprawi5227/toktickit/pull/47)
#38	[#49] (https://github.com/Suprawi5227/toktickit/pull/49)	Database Schema Evolution & User Models	Verified User model & FK constraints	Approved	[Peer PR #49] (https://github.com/Suprawi5227/toktickit/pull/49)
#40	[#49] (https://github.com/Suprawi5227/toktickit/pull/49)	Idempotent Seed Data & Bcrypt Hashing	Verified bcrypt salt rounds & idempotent seeds	Approved	[Peer PR #49] (https://github.com/Suprawi5227/toktickit/pull/49)
#42	[#49] (https://github.com/Suprawi5227/toktickit/pull/49)	Backend Auth APIs & Session Middleware	Verified JWT token validation & 401 handling	Approved	[Peer PR #49] (https://github.com/Suprawi5227/toktickit/pull/49)

Issue #	Peer PR #	PR Description	Review Action	Status	Direct GitHub Link
#44	[#49] (https://github.com/Suprawi5227/toktickit/pull/49)	Client Auth Context & Login UI	Verified form state management & logout cleanup	Approved	[Peer PR #49] (https://github.com/Suprawi5227/toktickit/pull/49)
#46	[#51] (https://github.com/Suprawi5227/toktickit/pull/51)	Requester Ticket Access Control APIs	Verified 403 authorization boundary	Approved	[Peer PR #51] (https://github.com/Suprawi5227/toktickit/pull/51)
#48	[#51] (https://github.com/Suprawi5227/toktickit/pull/51)	IT Staff Ticket Queue & Internal Notes APIs	Verified internal notes leak protection	Approved	[Peer PR #51] (https://github.com/Suprawi5227/toktickit/pull/51)
#50	[#52] (https://github.com/Suprawi5227/toktickit/pull/52)	Admin User Management APIs & Safety Rules	Verified self-deactivation & last-admin rules	Approved	[Peer PR #52] (https://github.com/Suprawi5227/toktickit/pull/52)

Issue #	Peer PR #	PR Description	Review Action	Status	Direct GitHub Link
#52	[#53] (https://github.com/Suprawi5227/toktickit/pull/53)	Zen Green UI Theme Consistency & Signoff	Verified Zen Green UI & E2E integration	Approved	[Peer PR #53] (https://github.com/Suprawi5227/toktickit/pull/53)
Release	[#54] (https://github.com/Suprawi5227/toktickit/pull/54)	Sprint 3 Final Release Integration	Verified full Sprint 3 integration to main	Approved	[Peer PR #54] (https://github.com/Suprawi5227/toktickit/pull/54)

1.5 Real GitHub Web UI Screenshots of Peer Reviews (Full Thread Evidence)

Issue 1: Lab 3 Specification DD Documents (Issue #34)

PR Received (natthakamol1130/toktickit/pull/35)

Platform Solutions Resources Open Source Enterprise Pricing

natthakamol1130

natthakamol1130 / toktickit Public

Notifications Fork 0 Star 0

Code Issues Pull requests Actions Projects Security and quality Insights

docs(lab-03): add Sprint 3 engineering specification documents (#34) #35

Merged Suprawi5227 merged 3 commits into lab3-staging from feature/lab03-1-spec-contract 2 weeks ago

Conversation 7 Commits 3 Checks 0 Files changed 4 +570 -2

natthakamol1130 commented 2 weeks ago

Owner

Description

This PR introduces the core Sprint 3 engineering specification contracts for Lab 3: TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens. All specification documents have been drafted to ensure internal consistency across functional requirements, business rules, authorization matrix, database changes, API contracts, and UI design rules prior to main feature implementations.

Key Changes

- Created docs/lab-03/specification.md defining Sprint Goal, Scope, Functional Requirements (FR-01 to FR-19), Business Rules (BR-01 to BR-13), Authorization Matrix, Data Schema, and Acceptance Criteria.
- Created docs/lab-03/ui-spec.md detailing Zen Green Design Tokens, role-based navigation, screens, component states, and responsive layout rules.
- Created docs/lab-03/api-spec.md documenting REST API endpoints, request/response schemas, HTTP status codes, and authorization requirements.

Related Issue

Closes #34

Verification

- Verified document structure, numbered requirements, and cross-consistency between specification files.

Reviewers

Suprawi5227

Assignees

natthakamol1130

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

[Lab 3] Spec DD: Sprint 3 Engineering Contract & Spe...

2 participants

docs(lab-03): add Sprint 3 engineering specification documents (#1)

natthakamol1130 linked an issue 2 weeks ago that may be closed by this pull request

[Lab 3] Spec DD: Sprint 3 Engineering Contract & Specification Documents #34

5 tasks done

natthakamol1130 requested a review from Suprawi5227 2 weeks ago

natthakamol1130 self-assigned this 2 weeks ago

Suprawi5227 reviewed 2 weeks ago

Suprawi5227 left a comment

Collaborator

รีวิว PR #35 ค่ะ การอัปเดตนี้ได้เพิ่มเอกสารสเปค API ของ Lab 3 (docs/lab-03/api-spec.md) พร้อมตารางสรุป Endpoint, รายละเอียด Request/Response, ตัวอย่าง JSON, และข้อมูลการตรวจสอบสิทธิ์ที่ชัดเจน รวมถึงเอกสารสเปค UI (docs/lab-03/ui-spec.md) ที่กำหนดแนวทางการออกแบบ Zen Green, โครงสร้างหน้าจอ Login, การเปลี่ยนรหัสผ่านครั้งแรก, รายละเอียด Ticket ของ Requester, Ticket Queue ของ IT Staff, หน้า Ticket Detail ของ IT Staff, และหน้าจัดการผู้ใช้ของ Administrator พร้อมสีโทน, แบบจัดเรียง responsive และแนวทางเข้าถึงได้ ทั้งนี้ยังมี Specification ของ Sprint 3 (docs/lab-03/specification.md) ที่อธิบายเป้าหมาย Sprint, ขอบเขตงาน, ความต้องการฟังก์ชัน (FR-01 ~ FR-19), กฎธุรกิจ (BR-01 ~ BR-13), UI Summary, การเปลี่ยนแปลงโมเดล Prisma, การสรุป Acceptance Criteria (AC-01 ~ AC-06) และ Definition of Done อย่างครบถ้วน ทำให้เอกสารมีความสมบูรณ์ ทั้งรูปแบบ Markdown ถูกจัดหัวข้อชัดเจน, ใช้ตารางและรายการสอดคล้องกับมาตรฐานของโครงการ และช่วยให้ผู้พัฒนาสามารถอ้างอิงและตรวจสอบการทำงานได้ง่าย

อย่างไรก็ตามเพื่อเพิ่มความครอบคลุมอาจพิจารณาเพิ่มตัวอย่างค่า curl สำหรับ Endpoint ที่สำคัญ, ระบุโค้ดสถานะ HTTP ของกรณีข้อผิดพลาด (เช่น 400 Bad Request, 403 Forbidden, 404 Not Found) ในส่วน "Detailed Endpoint Specifications" ให้ครบทุก API, และแนบภาพหรือไดอะแกรมการไหลของการทำงาน (เช่น Flowchart ของการ Login → Change-Password → Access) เพื่อให้ทีมพัฒนามีภาพรวมที่ชัดเจนยิ่งขึ้น นอกจากนี้ควรตรวจสอบไฟล์เอกสาร "api-spec.md" และ "ui-spec.md" ใช้ชื่อไฟล์และสัญลักษณ์ (เช่น .js → .ts) ให้สอดคล้องกับโค้ดจริงของโปรเจกต์และอัปเดตลิงก์อ้างอิงใน README หากจำเป็น ทั้งหมดนี้ทำให้ PR #35 มีความพร้อมสูงสำหรับการ Approve และ Merge ค่ะ

docs(lab-03): update api-spec with curl examples, status codes, and m...

natthakamol1130 commented 2 weeks ago

Owner Author

รีวิว PR #35 ค่ะ การอัปเดตนี้ได้เพิ่มเอกสารสเปค API ของ Lab 3 (docs/lab-03/api-spec.md) พร้อมตารางสรุป Endpoint, รายละเอียด

Request/Response, ตัวอย่าง JSON, และข้อมูลการตรวจสอบสิทธิ์ที่ชัดเจน รวมถึงเอกสารสเปค UI (docs/lab-03/ui-spec.md) ที่กำหนดแนวการออกแบบ Zen Green, โครงสร้างหน้าจอ Login, การเปลี่ยนรหัสผ่านครั้งแรก, รายละเอียด Ticket ของ Requester, Ticket Queue ของ IT Staff, หน้า Ticket Detail ของ IT Staff, และหน้าจอจัดการผู้ใช้ของ Administrator พร้อมสไลทอน, แบคจ, การจัดเรียง responsive และแนวทางเข้าถึงได้ ทั้งนี้ยังมี Specification ของ Sprint3 (docs/lab-03/specification.md) ที่อธิบายเป้าหมาย Sprint, ขอบเขตงาน, ความต้องการฟังก์ชัน (FR-01 ~ FR-19), กฎธุรกิจ (BR-01 ~ BR-13), UI Summary, การเปลี่ยนแปลงโมเดล Prisma, การสรุป Acceptance Criteria (AC-01 ~ AC-06) และ Definition of Done อย่างครบถ้วน ทำให้เอกสารมีความสมบูรณ์ ทั้งรูปแบบ Markdown ถูกจัดหัวข้อชัดเจน, ใช้ตารางและรายการสอดคล้องกับมาตรฐานของโครงการ และช่วยให้ผู้พัฒนาสามารถอ้างอิงและตรวจสอบการทำงานได้ง่าย

อย่างไรก็ตามเพื่อเพิ่มความครอบคลุมอาจพิจารณาเพิ่มตัวอย่างคำสั่ง curl สำหรับ Endpoint ที่สำคัญ, ระบุโค้ดสถานะ HTTP ของกรณีข้อผิดพลาด (เช่น 400 Bad Request, 403 Forbidden, 404 Not Found) ในส่วน "Detailed Endpoint Specifications" ให้ครบทุก API, และแผนภาพหรือไดอะแกรมการไหลของการทำงาน (เช่น Flowchart ของการ Login → Change-Password → Access) เพื่อให้ทีมพัฒนามีภาพรวมที่ชัดเจนยิ่งขึ้น นอกจากนี้ควรตรวจสอบให้ออกสาร "api-spec.md" และ "ui-spec.md" ใช้ชื่อไฟล์และสัญลักษณ์ (เช่น js → .ts) ให้สอดคล้องกับโค้ดจริงของโปรเจกต์และอัปเดตลิงก์อ้างอิงใน README หากจำเป็น ทั้งหมดนี้ทำให้ PR #35 มีความพร้อมสูงสำหรับการ Approve และ Merge ค่ะ

ขอบคุณสำหรับคอมเมนต์ที่วิวกมาก! เลยนะคะ ได้ทำการปรับปรุงเอกสารบน branch `feature/lab03-1-spec-contract` ตามข้อเสนอแนะให้เรียบร้อยแล้วค่ะ โดยมีรายละเอียดดังนี้ค่ะ:

1. เอกสาร API Spec (docs/lab-03/api-spec.md):

- เพิ่มตัวอย่างคำสั่ง curl สำหรับ Endpoints สำคัญ (login , change-password , staff tickets queue , internal notes , admin create user)
- เพิ่มตารางและรายละเอียดโค้ดสถานะ HTTP Error Codes (400 Bad Request , 401 Unauthorized , 403 Forbidden , 404 Not Found , 409 Conflict) ในส่วน Detailed Endpoint Specifications ครบทุก API
- ปรับการระบุสัญลักษณ์และชื่อไฟล์ทั้งหมดให้สอดคล้องกับ TypeScript (.ts / .tsx)

2. เอกสาร Specification (docs/lab-03/specification.md):

- เพิ่มไดอะแกรม Mermaid Flowchart ครบทั้ง 3 ส่วน: การล็อกอินและการบังคับเปลี่ยนรหัสผ่านครั้งแรก, การปฏิบัติงานของ IT Staff บน Ticket Queue/Detail, และการทำงานของ Admin พร้อมระบบป้องกันความปลอดภัย (Safety Guards)

3. เอกสาร README (README.md):

- อัปเดตส่วนนำทางเอกสารสเปคของ Lab 3 พร้อมแนบลิงก์เชื่อมโยงไปยังไฟล์ใน docs/lab-03/ เรียบร้อยแล้วค่ะ

ตอนนี้ PR #35 อัปเดตเรียบร้อยแล้ว ถ้าโอเคแล้วรบกวน Approve ให้หน่อยนะคะ ขอบคุณมาก! ค่ะ



Suprawi5227 reviewed 2 weeks ago

View reviewed changes

Suprawi5227 left a comment

Collaborator ...

PR #35 การเพิ่มเอกสาร Sprint3 (specification.md, api-spec.md, ui-spec.md) ครอบคลุมเป้าหมาย Sprint, ข้อกำหนดฟังก์ชัน (FR-01 ~ FR-14), กฎธุรกิจ (BR-01 ~ BR-15), การออกแบบ UI ด้วยธีม Zen Green, การเปลี่ยนแปลงโมเดลข้อมูล, และสเปค API ทั้งหมด พร้อม Acceptance Criteria และ Definition of Done ทำให้เอกสารสมบูรณ์และสอดคล้องกับข้อกำหนดของ Lab 3

อย่างไรก็ตามเพื่อให้สมบูรณ์ยิ่งขึ้น แนะนำให้เพิ่มตัวอย่าง cURL ของแต่ละ endpoint, ภาพ mockup ของหน้าจอสำคัญ, ตัวอย่างการทดสอบสำหรับ AC-01 ~ AC-12, และอัปเดตลิงก์ใน README ให้ชี้ไปยังเอกสารใหม่เหล่านี้; หากดำเนินการแก้ไขตามข้อเสนอแนะนี้ PR จะพร้อมสำหรับการ Approve และ Merge ค่ะ

docs (lab-03): add visual UI mockups and AC-01..AC-12 test mappings ba...

050eb1c



natthakamol1130 commented 2 weeks ago

Owner Author ...

PR #35 การเพิ่มเอกสาร Sprint3 (specification.md, api-spec.md, ui-spec.md) ครอบคลุมเป้าหมาย Sprint, ข้อกำหนดฟังก์ชัน (FR-01 ~ FR-14), กฎธุรกิจ (BR-01 ~ BR-15), การออกแบบ UI ด้วยธีม Zen Green, การเปลี่ยนแปลงโมเดลข้อมูล, และสเปค API ทั้งหมด พร้อม Acceptance Criteria และ Definition of Done ทำให้เอกสารสมบูรณ์และสอดคล้องกับข้อกำหนดของ Lab 3

อย่างไรก็ตามเพื่อให้สมบูรณ์ยิ่งขึ้น แนะนำให้เพิ่มตัวอย่าง cURL ของแต่ละ endpoint, ภาพ mockup ของหน้าจอสำคัญ, ตัวอย่างการทดสอบสำหรับ AC-01 ~ AC-12, และอัปเดตลิงก์ใน README ให้ชี้ไปยังเอกสารใหม่เหล่านี้; หากดำเนินการแก้ไขตามข้อเสนอแนะนี้ PR จะพร้อมสำหรับการ Approve และ Merge ค่ะ

ขอบคุณสำหรับข้อเสนอแนะเพิ่มเติมมาก! เลยนะคะ ได้ทำการปรับปรุงเอกสารบน branch `feature/lab03-1-spec-contract` ให้สมบูรณ์แบบเรียบร้อยแล้วค่ะ มีดังนี้เลย

1. UI Spec (docs/lab-03/ui-spec.md)**: เพิ่มแผนผังภาพจำลองโครงสร้างหน้าจอ (ASCII Visual UI Mockups) ครบทั้ง 6 หน้าจอหลัก (Login, Mandatory Password Change, Requester Detail, IT Staff Queue Table/Cards, IT Staff Detail with Internal Notes, และ Admin User Management Drawer)
2. API Spec (docs/lab-03/api-spec.md): เพิ่มตัวอย่างคำสั่ง curl สำหรับการรันทดสอบใน Terminal ครบทุก Endpoints ทั้งหมดในระบบ
3. Specification (docs/lab-03/specification.md): ขยายเกณฑ์ Acceptance Criteria เป็น AC-01 ถึง AC-12 ครอบคลุมทุกข้อกำหนดของอาจารย์ พร้อมระบุชื่อไฟล์การทดสอบแบบอัตโนมัติ (Automated Test File Mappings) ชัดเจน

ถ้าโอเคแล้วกด Approve และ Merge ให้หน่อยนะคะ ขอบคุณมาก! ค่ะ



Suprawi5227 requested changes 2 weeks ago

View reviewed changes

Suprawi5227 left a comment • edited

Collaborator ...

โอเคค่ะ ช่วยตรวจสอบอีกรอบหนึ่งที่ตะกอนที่เราจะกด Approve และ Merge



natthakamol1130 commented 2 weeks ago

Owner Author ...

โอเคค่ะ ช่วยตรวจสอบอีกรอบหนึ่งที่ตะกอนที่เราจะกด Approve และ Merge

ตรวจสอบเรียบร้อยแล้ว ทุกอย่างแก้ตามที่รีวิวเลยค่า รบกวน Approve ให้หน่อยค่ะ



Suprawi5227 approved these changes [2 weeks ago](#)

[View reviewed changes](#)

Suprawi5227 left a comment

Collaborator · ...

ทำการอัปเดตเอกสาร Sprint3 บน branch feature/lab03-1-spec-contract แล้วเพิ่ม ASCII Visual UI Mockups ครอบคลุม 6 หน้าจอหลัก ในไฟล์ docs/lab-03/ui-spec.md, เติมตัวอย่าง curl สำหรับทุก endpoint ใน docs/lab-03/api-spec.md, และขยาย Acceptance Criteria เป็น AC-01 ~ AC-12 พร้อมระบุแบบไฟล์ทดสอบอัตโนมัติใน docs/lab-03/specification.md; การแก้ไขเหล่านี้ทำให้เอกสารครบถ้วนตามข้อกำหนดของ Lab 3, ชัดเจนต่อการพัฒนาและการทดสอบ, จึงขอ Approve PR [#35](#) และ Merge ค่ะ



1



Suprawi5227 merged commit `ecc61b2` into `lab3-staging` [2 weeks ago](#)



This was referenced 5 days ago

[release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens \(v3.0.0\) #54](#)

➔ Merged

[release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens \(v3.0.0\) #55](#)

➔ Merged

[Sign up for free](#) to join this conversation on GitHub. Already have an account? [Sign in to comment](#)



© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Real PR Received Issue 1

PR Given (Suprawi5227/toktickit/pull/45)

Platform Solutions Resources Open Source Enterprise Pricing

natthakamol1130

Search Sign in Sign up

Suprawi5227 / toktickit Public

Notifications Fork 0 Star 0

<> Code Issues 9 Pull requests Actions Projects Security and quality Insights

feat(git): setup lab3-staging infrastructure and update README (Part 1) #45

Merged natthakamol1130 merged 1 commit into lab3-staging from feature/part1-git 2 weeks ago

Conversation 2 Commits 1 Checks 0 Files changed 1 +56 -43

Suprawi5227 commented 2 weeks ago

Owner ...

Closes #1

feat(git): setup lab3-staging branch infrastructure and update README...

5208aa5

Suprawi5227 requested a review from natthakamol1130 2 weeks ago

Suprawi5227 self-assigned this 2 weeks ago

Suprawi5227 linked an issue 2 weeks ago that may be closed by this pull request

Issue #1: Git Workflow & Branching Infrastructure (Part 1) L3 #36

Open

natthakamol1130 approved these changes 2 weeks ago

View reviewed changes

natthakamol1130 left a comment

Collaborator ...

รีวิว PR #45 เรียบร้อยค่ะ การปรับปรุงเอกสาร README มีความถูกต้อง ครบถ้วนตามโครงสร้างของ Lab 3 Approve ค่ะ และ Merge ให้เลยนะคะ

Suprawi5227 commented 2 weeks ago

Owner Author ...

รีวิว PR #45 เรียบร้อยค่ะ การปรับปรุงเอกสาร README มีความถูกต้อง ครบถ้วนตามโครงสร้างของ Lab 3 Approve ค่ะ และ Merge ให้เลยนะคะ

โอเคค่ะขอบคุณค่า

natthakamol1130 merged commit 6cd5018 into lab3-staging 2 weeks ago

Reviewers

natthakamol1130 ✓

Assignees

Suprawi5227

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

Issue #1: Git Workflow & Branching Infrastructure (Pa...

2 participants

Sign up for free to join this conversation on GitHub. Already have an account? Sign in to comment

© 2026 GitHub, Inc.

Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

Real PR Given Issue 1

Issue 2: Lab 3 Test-Driven Development Plan (Issue #36)

PR Received (natthakamol1130/toktickit/pull/37)

Platform Solutions Resources Open Source Enterprise Pricing

natthakamol1130

Search Sign in Sign up

natthakamol1130 / toktickit Public

Notifications Fork 0 Star 0

<> Code Issues Pull requests Actions Projects Security and quality Insights

docs(lab-03): add Test DD plan and traceability matrix (#36) #37

Merged Suprawi5227 merged 1 commit into lab3-staging from feature/lab03-2-test-plan 2 weeks ago

Conversation 2 Commits 1 Checks 0 Files changed 1 +81

natthakamol1130 commented 2 weeks ago Owner

Description

This PR introduces the Test Driven Development (Test DD) plan and Acceptance Criteria traceability matrix for Lab 3 in docs/lab-03/tests.md. All test strategy levels, planned automated test cases, and traceability mappings to AC-01 through AC-12 are fully specified prior to implementation.

Key Changes

- Created docs/lab-03/tests.md defining testing strategy across Unit, API, Component, Authorization, and Playwright E2E test levels.
- Enumerated Planned Tests Table with explicit server, client, and E2E test file paths.
- Constructed Acceptance-Criterion Traceability Matrix mapping AC-01 through AC-12 to automated test cases.
- Defined Responsive and Visual inspection checklist and execution commands.

Related Issue

Closes #36

Verification

- Verified test plan structure, AC traceability mappings, and test command specifications.

Reviewers

Suprawi5227

Assignees

natthakamol1130

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

[Lab 3] Test DD: Test Plan & Acceptance Traceability ...

2 participants

docs(lab-03): add Test DD plan and traceability matrix (#36) 93bffb8

natthakamol1130 linked an issue 2 weeks ago that may be closed by this pull request

[Lab 3] Test DD: Test Plan & Acceptance Traceability Matrix #36 4 tasks done Closed

natthakamol1130 requested a review from Suprawi5227 2 weeks ago

natthakamol1130 self-assigned this 2 weeks ago

Suprawi5227 approved these changes 2 weeks ago View reviewed changes

Suprawi5227 left a comment Collaborator

ได้เพิ่มไฟล์ docs/lab-03/tests.md ใน PR #37 พร้อม TestDD Plan และ Traceability Matrix ที่ครอบคลุม Test Strategy, ตาราง Planned Tests (10 API, 5 UI, 3 E2E), Matrix เชื่อมโยงทุก Acceptance Criterion (AC-01 ~ AC-12) กับชุดทดสอบที่ผ่านแล้ว, เช็คลิสต์ Responsive/Zen Green, และคำสั่งรันทดสอบครบชุด – เอกสารครบถ้วนและเป็นมาตรฐานสูง, ดังนั้นขอ Approve and Merge PR #37 เลยนะคะ

natthakamol1130 commented 2 weeks ago Owner Author

ได้เพิ่มไฟล์ docs/lab-03/tests.md ใน PR #37 พร้อม TestDD Plan และ Traceability Matrix ที่ครอบคลุม Test Strategy, ตาราง Planned Tests (10 API, 5 UI, 3 E2E), Matrix เชื่อมโยงทุก Acceptance Criterion (AC-01 ~ AC-12) กับชุดทดสอบที่ผ่านแล้ว, เช็คลิสต์ Responsive/Zen Green, และคำสั่งรันทดสอบครบชุด – เอกสารครบถ้วนและเป็นมาตรฐานสูง, ดังนั้นขอ Approve and Merge PR #37 เลยนะคะ

โอเคค่ะ ขอบคุณค่ะ <3<3

Suprawi5227 merged commit fc5eff9 into lab3-staging 2 weeks ago

This was referenced 5 days ago

release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens (v3.0.0) #54 Merged

Sign up for free to join this conversation on GitHub. Already have an account? [Sign in to comment](#)

Real PR Received Issue 2

PR Given (Suprawi5227/toktickit/pull/47)

Platform Solutions Resources Open Source Enterprise Pricing

natthakamol1130

Suprawi5227 / toktickit Public

Notifications Fork 0 Star 0

Code Issues 9 Pull requests Actions Projects Security and quality Insights

docs(test): add Sprint 3 test plan and AC traceability matrix (Closes... #47

Merged natthakamol1130 merged 1 commit into lab3-staging from feature/part3-tests 2 weeks ago

Conversation 2 Commits 1 Checks 0 Files changed 1 +65

Suprawi5227 commented 2 weeks ago

Owner

Closes #3

docs(test): add Sprint 3 test plan and AC traceability matrix (Closes #3

7ea3b25

Suprawi5227 requested a review from natthakamol1130 2 weeks ago

Suprawi5227 self-assigned this 2 weeks ago

Suprawi5227 linked an issue 2 weeks ago that may be closed by this pull request

Issue #3: Test Plan DD & Traceability Matrix (Part 3) L3 #38

Open

natthakamol1130 approved these changes 2 weeks ago

View reviewed changes

natthakamol1130 left a comment

Collaborator

ตรวจงาน PR #47 เรียบร้อยค่ะ เอกสารแผนการทดสอบ tests.md มีความสมบูรณ์แบบ เชื่อมโยงเกณฑ์ AC-01 ถึง AC-12 ครบถ้วนทุกเคส Approve ค่ะ ะ และ Merge ใหม่นะคะ

Suprawi5227 commented 2 weeks ago

Owner Author

ตรวจงาน PR #47 เรียบร้อยค่ะ เอกสารแผนการทดสอบ tests.md มีความสมบูรณ์แบบ เชื่อมโยงเกณฑ์ AC-01 ถึง AC-12 ครบถ้วนทุกเคส Approve ค่ะ ะ และ Merge ใหม่นะคะ

โอเคค่ะ ขอบคุณค่า

natthakamol1130 merged commit 8cadff2 into lab3-staging 2 weeks ago

Reviewers

natthakamol1130 ✓

Assignees

Suprawi5227

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

Issue #3: Test Plan DD & Traceability Matrix (Part 3) L3

2 participants

Sign up for free to join this conversation on GitHub. Already have an account? [Sign in to comment](#)

© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Real PR Given Issue 2

Issue 3: Database Schema Evolution (Issue #38)

PR Received ([natthakamol1130/toktickit/pull/39](#))



Platform Solutions Resources Open Source Enterprise Pricing



natthakamol1130

Search

Sign in

Sign up

...

natthakamol1130 / toktickit Public

Notifications Fork 0 Star 0

<> Code

Issues

Pull requests

Actions

Projects

Security and quality

Insights

feat(db): evolve Prisma schema for User models, roles, comments, and notes (#38) #39

Merged Suprawi5227 merged 2 commits into lab3-staging from feature/lab03-3-db-schema 2 weeks ago

Conversation 4

Commits 2

Checks 0

Files changed 1

+93 -22



natthakamol1130 commented 2 weeks ago

Owner

Description

This PR evolves the Prisma schema and PostgreSQL data model to support real User authentication, single-role authorization (\REQUESTER, \IT_STAFF, \ADMINISTRATOR), Ticket primary ownership, IT Priority, Public Comments, and role-restricted Internal Notes.

Key Changes

- Added \User\ model (\email, \passwordHash, ame, \department, ole, \isActive, \mustChangePassword, \createdAt, \updatedAt).
- Added \Role, \ITPriority\ enums, and expanded \TicketStatus\ enum values (\NEW, \OPEN, \IN_PROGRESS, \WAITING_FOR_REQUESTER, \RESOLVED, \CLOSED, \REOPENED, \CANCELLED).
- Added \PublicComment\ and \InternalNote\ models with relations to \Ticket\ and \User.
- Updated \Ticket\ model with \ownerId\ FK and \itPriority\ field.

Related Issue

Closes #38

Verification

- Verified Prisma Client generation and PostgreSQL schema synchronization.

Reviews

Suprawi5227

Assignees

natthakamol1130

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

[Lab 3] Database: Schema Evolution, User Models & L...

2 participants

feat(db): evolve Prisma schema for User models, roles, comments, and ...

b50669b

natthakamol1130 requested a review from Suprawi5227 2 weeks ago

natthakamol1130 self-assigned this 2 weeks ago

natthakamol1130 linked an issue 2 weeks ago that may be closed by this pull request

[Lab 3] Database: Schema Evolution, User Models & Lab 2 Migration #38

3 tasks done

Closed

Suprawi5227 reviewed 2 weeks ago

View reviewed changes

Suprawi5227 left a comment

Collaborator

จากการตรวจสอบโค้ดใน PR #39 ที่แก้ไขไฟล์ schema.prisma พบว่าการออกแบบโครงสร้างฐานข้อมูลทำได้ดีเยี่ยมและครอบคลุม requirement ของ Lab 3 มากๆ ครับ มีการเพิ่ม Role , ITPriority และขยายสถานะ TicketStatus ได้อย่างถูกต้องตาม Business Rules ส่วนโมเดล User ก็รองรับระบบ Authentication และ RBAC ได้ครบถ้วน พร้อมทั้งการแยกความสัมพันธ์ requester และ owner ในตาราง Ticket ที่ชัดเจน รวมถึงการใส่ onDelete: Cascade ให้กับ PublicComment และ InternalNote และการตั้ง Index (@@index) ที่ช่วยเพิ่มประสิทธิภาพในการดึงข้อมูล Queue ได้ดีมากครับ

อย่างไรก็ตาม มีจุดสังเกตเล็กน้อยคือในสเปคเคยระบุว่าจะมีฟิลด์ requesterResolvedInd (Boolean) ในตาราง Ticket ซึ่งยังไม่เห็นใน Schema นี้ ลองตรวจสอบดูนะคะว่าควรลบไปหรือไม่ โดยรวมแล้วโค้ดโครงสร้างสวยงามและพร้อมใช้งานค่ะ ช่วยตรวจสอบให้หน่อยค่ะ

feat(db): add requesterResolvedInd field to Ticket model based on pee...

8b0846f

natthakamol1130 commented 2 weeks ago

Owner Author

จากการตรวจสอบโค้ดใน PR #39 ที่แก้ไขไฟล์ schema.prisma พบว่าการออกแบบโครงสร้างฐานข้อมูลทำได้ดีเยี่ยมและครอบคลุม requirement ของ Lab 3 มากๆ ครับ มีการเพิ่ม Role , ITPriority และขยายสถานะ TicketStatus ได้อย่างถูกต้องตาม Business Rules ส่วนโมเดล User ก็รองรับระบบ Authentication และ RBAC ได้ครบถ้วน พร้อมทั้งการแยกความสัมพันธ์ requester และ owner ในตาราง Ticket ที่ชัดเจน รวมถึงการใส่ onDelete: Cascade ให้กับ PublicComment และ InternalNote และการตั้ง Index (@@index) ที่ช่วยเพิ่มประสิทธิภาพในการดึงข้อมูล Queue ได้ดีมากครับ

อย่างไรก็ตาม มีจุดสังเกตเล็กน้อยคือในสเปคเคยระบุว่าจะมีฟิลด์ requesterResolvedInd (Boolean) ในตาราง Ticket ซึ่งยังไม่เห็นใน Schema นี้ ลองตรวจสอบดูนะคะว่าควรลบไปหรือไม่ โดยรวมแล้วโค้ดโครงสร้างสวยงามและพร้อมใช้งานค่ะ ช่วยตรวจสอบให้หน่อยค่ะ

หากมีข้อสงสัยหรือต้องการข้อมูลเพิ่มเติม กรุณาติดต่อฝ่ายสนับสนุนทางอีเมลที่ support@toktickit.com หรือทางโทรศัพท์ที่ [02-123-4567](tel:02-123-4567)

หน้าย่อย

ขอขอบคุณสำหรับข้อสังเกตต่างๆ เลยนะคะ ได้ทำการอัปเดตไฟล์ `server/prisma/schema.prisma` บน branch `feature/lab03-3-db-schema` เรียบร้อยแล้วค่ะ:

- เพิ่มฟิลด์ `requesterResolvedInd Boolean @default(false)` ใน `Ticket` Model สำหรับบันทึกสัญญาณการแจ้งแก้ไขปัญหาจาก Requester เรียบร้อยแล้วค่ะ
- รัน `npm run prisma generate` และซิงก์ Schema เข้าฐานข้อมูลเรียบร้อยแล้ว

PR #39 มีความสมบูรณ์ 100% พร้อมสำหรับการ Approve และ Merge เข้าสาขา `lab3-staging` แล้วนะคะ รบกวนตรวจสอบให้อีกทีค่ะ ขอขอบคุณมาก ๆ ค่ะ



Suprawi5227 approved these changes 2 weeks ago

[View reviewed changes](#)

Suprawi5227 left a comment

Collaborator ...

ยอดเยี่ยมเลยคะ! ดีใจที่ข้อสังเกตเล็กๆ น้อยๆ เป็นประโยชน์นะคะ จากที่เพิ่มฟิลด์ `requesterResolvedInd Boolean @default(false)` เข้ามาในโมเดล `Ticket` ทำให้ Schema ตอนนี้อยู่ในรูปแบบและสอดคล้องกับ Requirement ของ Lab 3 ครบ 100% แล้วค่ะ ทั้งโครงสร้างความสัมพันธ์ (Relations), การกำหนด Enum, และ Index ต่างๆ ท่านมาได้ไว้ดีมาก ๆ ขอ Approve PR #39 นี้คะ

❤️ 1



natthakamol1130 commented 2 weeks ago

Owner Author ...

ยอดเยี่ยมเลยคะ! ดีใจที่ข้อสังเกตเล็กๆ น้อยๆ เป็นประโยชน์นะคะ จากที่เพิ่มฟิลด์ `requesterResolvedInd Boolean @default(false)` เข้ามาในโมเดล `Ticket` ทำให้ Schema ตอนนี้อยู่ในรูปแบบและสอดคล้องกับ Requirement ของ Lab 3 ครบ 100% แล้วค่ะ ทั้งโครงสร้างความสัมพันธ์ (Relations), การกำหนด Enum, และ Index ต่างๆ ท่านมาได้ไว้ดีมาก ๆ ขอ Approve PR #39 นี้คะ

โอเคค่ะ ขอขอบคุณ <3



Suprawi5227 merged commit 465a4a7 into `lab3-staging` 2 weeks ago



This was referenced 5 days ago

[release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens \(v3.0.0\) #54](#)

➔ Merged

[release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens \(v3.0.0\) #55](#)

➔ Merged

Sign up for free

to join this conversation on GitHub. Already have an account? [Sign in to comment](#)



© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Real PR Received Issue 3

PR Given (Suprawi5227/toktickit/pull/49)



Platform Solutions Resources Open Source Enterprise Pricing



natthakamol1130

Suprawi5227 / toktickitPublic

NotificationsFork0Star0

<> CodeIssues9Pull requestsActionsProjectsSecurity and qualityInsights

feat(auth): implement authentication foundation, JWT session, and mandatory password change UI (Part 5) #49

Mergednatthakamol1130 merged 2 commits into lab3-staging from feature/part5-auth2 weeks ago

Conversation5Commits2Checks0Files changed12+1,118-65

Suprawi5227 commented 2 weeks agoOwner

Closes #5

feat(auth): implement authentication foundation, JWT session, and man...b751836

Suprawi5227 requested a review from natthakamol1130 2 weeks ago

Suprawi5227 self-assigned this 2 weeks ago

Suprawi5227 linked an issue 2 weeks ago that may be closed by this pull request

Issue #5: Working Login & Mandatory Password Change UI (Part 5) L3 #40Open

natthakamol1130 approved these changes 2 weeks agoView reviewed changes

natthakamol1130 left a commentCollaborator

PR Review Findings & Assessment

Great implementation! The Authentication foundation, JWT session management, and mandatory password change workflow are implemented cleanly according to the Lab 3 Specification.

Key Highlights:

1. Authentication & Authorization Middleware (\auth.ts):

Supports both HTTP-only Session Cookie (\oktickit_session) and Bearer Token header.

Enforces \enforcePasswordChange\ restriction when equiresPasswordChange = true.

2. Auth Routes (\auth.routes.ts):

\POST /api/auth/login: Safely normalizes email (\email.trim().toLowerCase()) and checks account active status (\isActive).

\POST /api/auth/change-password: Validates current password using bcrypt, enforces length criteria (>=8 chars), and clears equiresPasswordChange\ flag.

3. Test Suite Coverage (\auth.api.test.ts, \authorization.api.test.ts):

Covers successful login, inactive account rejection (401), password change flag, and cross-user authorization restriction (403).

Minor Suggestion:

In \server/tests/lab-03/authorization.api.test.ts, consider using optional chaining when accessing \queueRes.body.data?.[0]\ to safely handle cases where test seed queries might differ.

Overall looks great and ready to merge!

natthakamol1130 reviewed 2 weeks agoView reviewed changes

natthakamol1130 left a commentCollaborator

ผลการตรวจทาน Pull Request #49

โค้ดส่วน Authentication foundation, JWT session management และระบบเปลี่ยนรหัสผ่านบังคับ เขียนได้สะอาด ถูกต้องตามข้อกำหนดของ Lab 3 Specification มากค่ะ

จุดเด่นของการพัฒนา:

1. Authentication & Authorization Middleware (\auth.ts):

รองรับทั้ง HTTP-only Session Cookie (\oktickit_session) และ Authorization Bearer Token

มีระบบ \enforcePasswordChange\ ป้องกันไม่ให้ผู้ใช้ที่มีสถานะ equiresPasswordChange = true\ เข้าถึง API อื่นๆ

2. Auth Routes (\auth.routes.ts):

\POST /api/auth/login: ทำความสะอาดอีเมล (\email.trim().toLowerCase()) และตรวจสอบสถานะ \isActive\ อย่างปลอดภัย

Reviewers

natthakamol1130

Assignees

Suprawi5227

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

Issue #5: Working Login & Mandatory Password Cha...

2 participants

- `\POST /api/auth/change-password`: ตรวจสอบรหัสผ่านปัจจุบันด้วย `bcrypt`, บังคับความยาว `>= 8` ตัวอักษร และอัปเดต `requiresPasswordChange` เป็น `\true`
- 3. การทดสอบ (`\auth.api.test.ts`, `\authorization.api.test.ts`):
 - ครอบคลุมการเข้าสู่ระบบ, การปฏิเสธปัญหาที่ถูกปิดใช้งาน (401) และการตรวจสอบสิทธิ์ข้ามผู้ใช้ (403)

ข้อเสนอแนะเพิ่มเติม:

- ในไฟล์ `\server\tests\lab-03\authorization.api.test.ts` ตรงการเข้าถึง `\queueRes.body.data[0]` แนะนำให้ optional chaining (`\queueRes.body.data?.[0]`) เพื่อความปลอดภัยกรณีชุดข้อมูล Seed ต่างกัน

ภาพรวมเรียบร้อยดี พร้อมสำหรับ Merge ค่ะ!



natthakamol1130 reviewed 2 weeks ago

[View reviewed changes](#)

natthakamol1130 left a comment

Collaborator ...

โค้ดส่วน Authentication foundation, JWT session management และระบบเปลี่ยนรหัสผ่านบังคับ เขียนได้สะอาด ถูกต้องตามข้อกำหนดของ Lab 3 Specification มากค่ะ

ข้อเสนอแนะเพิ่มเติม :

- ในไฟล์ `server\tests\lab-03\authorization.api.test.ts` ตรงการเข้าถึง `queueRes.body.data[0]` แนะนำให้ optional chaining (`queueRes.body.data?.[0]`) เพื่อความปลอดภัยกรณีชุดข้อมูล Seed ต่างกัน

ภาพรวมเรียบร้อยดี พร้อมสำหรับ Merge ค่ะ

`fix(test): use optional chaining queueRes.body.data?.[0] per PR #5` [re...](#)

[cdede](#)



Suprawi5227 commented 2 weeks ago

Owner Author ...

ข้อเสนอแนะเพิ่มเติม:

- ในไฟล์ `\server\tests\lab-03\authorization.api.test.ts` ตรงการเข้าถึง `\queueRes.body.data[0]` แนะนำให้ optional chaining (`\queueRes.body.data?.[0]`) เพื่อความปลอดภัยกรณีชุดข้อมูล Seed ต่างกัน

ขอบคุณสำหรับ feedback มากเลย! แก้ไขเรียบร้อยแล้ว โดยเปลี่ยนมาใช้ Optional Chaining (`queueRes.body.data?.[0]`) และเพิ่ม `expect(targetTicket).toBeDefined()` ใน `\authorization.api.test.ts` นะคะ

1



natthakamol1130 approved these changes 2 weeks ago

[View reviewed changes](#)

natthakamol1130 left a comment

Collaborator ...

ผลการตรวจทาน Pull Request #49 (รอบที่ 2)

ขอบคุณสำหรับการอัปเดตโค้ดในส่วนของการใช้ optional chaining (`\queueRes.body.data?.[0]`) และเพิ่ม `\expect(targetTicket).toBeDefined()` ใน `\authorization.api.test.ts` นะคะ

ขณะนี้โค้ดมีความสมบูรณ์ 100% ถูกต้องตามข้อกำหนดทุกประการ ทำการ Approve ให้เรียบร้อยแล้ว สามารถกด Merge ได้เลยคะ!



Suprawi5227 commented 2 weeks ago

Owner Author ...

ผลการตรวจทาน Pull Request #49 (รอบที่ 2)

ขอบคุณสำหรับการอัปเดตโค้ดในส่วนของการใช้ optional chaining (`\queueRes.body.data?.[0]`) และเพิ่ม `\expect(targetTicket).toBeDefined()` ใน `\authorization.api.test.ts` นะคะ

ขณะนี้โค้ดมีความสมบูรณ์ 100% ถูกต้องตามข้อกำหนดทุกประการ ทำการ Approve ให้เรียบร้อยแล้ว สามารถกด Merge ได้เลยคะ!

ขอบคุณค่า

1

natthakamol1130 merged commit `be92796` into `lab3-staging` 2 weeks ago

[Sign up for free](#) to join this conversation on GitHub. Already have an account? [Sign in to comment](#)



© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Issue 4: Idempotent Seed Data with Bcrypt Hashing (Issue #40)

PR Received ([natthakamol1130/toktickit/pull/41](https://github.com/natthakamol1130/toktickit/pull/41))

Platform

Solutions

Resources

Open Source

Enterprise

Pricing

natthakamol1130

Search

Sign in

Sign up

Flags

natthakamol1130 / tokticket

Public

Notifications

Fork 0

Star 0

<> Code

Issues

Pull requests

Actions

Projects

Security and quality

Insights

feat(seed): add idempotent multi-role seed data with bcrypt hashing (#40) #41

Merged

Suprawi5227 merged 1 commit into lab3-staging from feature/lab03-4-seed-data 2 weeks ago

Conversation 2

Commits 1

Checks 0

Files changed 1

+230 -21

natthakamol1130 commented 2 weeks ago

Owner

...

Description

This PR implements idempotent seed data in server/prisma/seed.ts for Lab 3 testing, including active and inactive accounts across Requesters, IT Staff, Administrator, and realistic ticket datasets.

Key Changes

- Seeded bcrypt-hashed passwords for initial and standard account access.
- Seeded 4 active Requesters, 1 inactive Requester.
- Seeded 3 active IT Staff, 1 inactive IT Staff.
- Seeded 1 active Administrator.
- Seeded realistic tickets, Public Comments, and role-restricted Internal Notes.

Related Issue

Closes #40

Verification

- Executed npx tsx prisma/seed.ts successfully in local environment.

Reviewers

Suprawi5227

✓

Assignees

natthakamol1130

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

[Lab 3] Database: Idempotent Seed Data for All Roles...

2 participants



feat(seed): add idempotent multi-role seed data with bcrypt hashing (#40) ...

51eb2b4

natthakamol1130 linked an issue 2 weeks ago that may be closed by this pull request

[Lab 3] Database: Idempotent Seed Data for All Roles & Test Accounts #40

Closed

2 tasks done

natthakamol1130 requested a review from Suprawi5227 2 weeks ago

natthakamol1130 self-assigned this 2 weeks ago

Suprawi5227 approved these changes 2 weeks ago

View reviewed changes

Suprawi5227 left a comment

Collaborator

...

จากการตรวจสอบโค้ดใน PR #41 พบว่าการเขียน Seed Script ทำได้ยอดเยี่ยมและสมบูรณ์แบบมากค่ะ! โค้ดรองรับ Idempotent เดิมรูปแบบผ่านการใช้ upsert และ skipDuplicates ทำให้รันซ้ำได้โดยไม่ซ้ำข้อมูลซ้ำซ้อน นอกจากนี้ยังมีการเข้ารหัสผ่านด้วย bcrypt อย่างถูกต้องปลอดภัย และสร้างข้อมูลทดสอบครบถ้วนทุก Role (Requester, IT Staff, Admin) รวมถึงเคสที่ต้องเปลี่ยนรหัสผ่านครั้งแรกและเคส Inactive ด้วย ที่สำคัญคือมีการจำลองข้อมูล Ticket, Public Comments และ Internal Notes ไว้พร้อมให้ทีมนำไปทดสอบ UI ได้ทันทีค่ะ โค้ดเขียนมาได้อย่างเรียบร้อยและพร้อมใช้งานมาก ขอ Approve และ Merge นะคะ

1

natthakamol1130 commented 2 weeks ago

Owner

Author

...

จากการตรวจสอบโค้ดใน PR #41 พบว่าการเขียน Seed Script ทำได้ยอดเยี่ยมและสมบูรณ์แบบมากค่ะ! โค้ดรองรับ Idempotent เดิมรูปแบบผ่านการใช้ upsert และ skipDuplicates ทำให้รันซ้ำได้โดยไม่ซ้ำข้อมูลซ้ำซ้อน นอกจากนี้ยังมีการเข้ารหัสผ่านด้วย bcrypt อย่างถูกต้องปลอดภัย และสร้างข้อมูลทดสอบครบถ้วนทุก Role (Requester, IT Staff, Admin) รวมถึงเคสที่ต้องเปลี่ยนรหัสผ่านครั้งแรกและเคส Inactive ด้วย ที่สำคัญคือมีการจำลองข้อมูล Ticket, Public Comments และ Internal Notes ไว้พร้อมให้ทีมนำไปทดสอบ UI ได้ทันทีค่ะ โค้ดเขียนมาได้อย่างเรียบร้อยและพร้อมใช้งานมาก ขอ Approve และ Merge นะคะ

โอเคค่ะ ขอบขอบคุณนะคะ

Suprawi5227 merged commit 41e3dcd into lab3-staging 2 weeks ago

This was referenced 5 days ago

release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens (v3.0.0) #54

➔ Merged

release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens (v3.0.0) #55

➔ Merged

[Sign up for free](#) to join this conversation on GitHub. Already have an account? [Sign in to comment](#)



© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Real PR Received Issue 4

PR Given (Suprawi5227/toktickit/pull/49)

feat(auth): implement authentication foundation, JWT session, and mandatory password change UI (Part 5) #49

Merged [natthakamol1130](#) merged 2 commits into [lab3-staging](#) from [feature/part5-auth](#) 2 weeks ago

Conversation 5 Commits 2 Checks 0 Files changed 12

+1,118 -65

Suprawi5227 commented 2 weeks ago

Owner ...

Closes #5

feat(auth): implement authentication foundation, JWT session, and man... ..

b751836

 Suprawi5227 requested a review from **natthakamol1130** 2 weeks ago

  Suprawi5227 self-assigned this 2 weeks ago

 Suprawi5227 linked an issue 2 weeks ago that may be closed by this pull request

Issue #5: Working Login & Mandatory Password Change UI (Part 5) L3 #40

✓ natthakamol1130 approved these changes 2 weeks ago

[View reviewed changes](#)

natthakamol1130 left a comment

Collaborator ...

PR Review Findings & Assessment

Great implementation! The Authentication foundation, JWT session management, and mandatory password change workflow are implemented cleanly according to the Lab 3 Specification.

Key Highlights:

1. **Authentication & Authorization Middleware (\auth.ts):**
 - Supports both HTTP-only Session Cookie (\otkicik_session) and Bearer Token header.
 - Enforces \enforcePasswordChange\ restriction when \requiresPasswordChange = true.
2. **Auth Routes (\auth.routes.ts):**
 - \POST /api/auth/login: Safely normalizes email (\email.trim().toLowerCase()) and checks account active status (\isActive).
 - \POST /api/auth/change-password: Validates current password using bcrypt, enforces length criteria (>=8 chars), and clears \requiresPasswordChange\ flag.
3. **Test Suite Coverage (\auth.api.test.ts, \authorization.api.test.ts):**
 - Covers successful login, inactive account rejection (401), password change flag, and cross-user authorization restriction (403).

Minor Suggestion:

- In `server/tests/lab-03/authorization.api.test.ts`, consider using optional chaining when accessing `queueRes.body.data?` to safely handle cases where test seed queries might differ.

Overall looks great and ready to merge!

 [natthakamol1130](#) reviewed 2 weeks ago

[View reviewed changes](#)

natthakamol1130 left a comment

Collaborator ...

ผลการตรวจทาน Pull Request #49

โค้ดส่วน Authentication foundation, JWT session management และระบบเปลี่ยนรหัสผ่านบังคับ เขียนได้สะอาด ถูกต้องตามข้อกำหนดของ Lab 3 Specification มากค่ะ

จุดเด่นของการพัฒนา:

1. **Authentication & Authorization Middleware (\auth.ts):**
 - รองรับการ HTTP-only Session Cookie (klickit_session) และ Authorization Bearer Token
 - มีระบบ (enforcePasswordChange) ป้องกันไม่ให้ผู้ใช้ที่มีสถานะ requiresPasswordChange = true เข้าถึง API อื่นๆ
2. **Auth Routes (\auth.routes.ts):**
 - \POST /api/auth/login: ทำความสะอาดอีเมล (email.trim().toLowerCase()) และตรวจสอบสถานะ (isActive) อย่างปลอดภัย

- `\POST /api/auth/change-password`: ตรวจสอบรหัสผ่านปัจจุบันด้วย `bcrypt`, บังคับความยาว `>= 8` ตัวอักษร และอัปเดต `requiresPasswordChange` เป็น `\true`
- 3. การทดสอบ (`\auth.api.test.ts`, `\authorization.api.test.ts`):
 - ครอบคลุมการเข้าสู่ระบบ, การปฏิเสธบัญชีที่ถูกปิดใช้งาน (401) และการตรวจสอบสิทธิ์ข้ามผู้ใช้ (403)

ข้อเสนอแนะเพิ่มเติม:

- ในไฟล์ `\server\tests\lab-03\authorization.api.test.ts` ตรงการเข้าถึง `\queueRes.body.data[0]` แนะนำให้ optional chaining (`\queueRes.body.data?.[0]`) เพื่อความปลอดภัยกรณีชุดข้อมูล Seed ต่างกัน

ภาพรวมเรียบร้อยดี พร้อมสำหรับ Merge ค่ะ!



natthakamol1130 reviewed 2 weeks ago

[View reviewed changes](#)

natthakamol1130 left a comment

Collaborator ...

โค้ดส่วน Authentication foundation, JWT session management และระบบเปลี่ยนรหัสผ่านบังคับ เขียนได้สะอาด ถูกต้องตามข้อกำหนดของ Lab 3 Specification มากค่ะ

ข้อเสนอแนะเพิ่มเติม :

- ในไฟล์ `server\tests\lab-03\authorization.api.test.ts` ตรงการเข้าถึง `queueRes.body.data[0]` แนะนำให้ optional chaining (`queueRes.body.data?.[0]`) เพื่อความปลอดภัยกรณีชุดข้อมูล Seed ต่างกัน

ภาพรวมเรียบร้อยดี พร้อมสำหรับ Merge ค่ะ

fix(test): use optional chaining queueRes.body.data?.[0] per PR #5 re...

cdede



Suprawi5227 commented 2 weeks ago

Owner Author ...

ข้อเสนอแนะเพิ่มเติม:

- ในไฟล์ `\server\tests\lab-03\authorization.api.test.ts` ตรงการเข้าถึง `\queueRes.body.data[0]` แนะนำให้ optional chaining (`\queueRes.body.data?.[0]`) เพื่อความปลอดภัยกรณีชุดข้อมูล Seed ต่างกัน

ขอบคุณสำหรับ feedback มากเลย! แก้ไขเรียบร้อยแล้ว โดยเปลี่ยนมาใช้ Optional Chaining (`queueRes.body.data?.[0]`) และเพิ่ม `expect(targetTicket).toBeDefined()` ใน `\authorization.api.test.ts` นะคะ



natthakamol1130 approved these changes 2 weeks ago

[View reviewed changes](#)

natthakamol1130 left a comment

Collaborator ...

ผลการตรวจทาน Pull Request #49 (รอบที่ 2)

ขอบคุณสำหรับการอัปเดตโค้ดในส่วนของการใช้ optional chaining (`\queueRes.body.data?.[0]`) และเพิ่ม `\expect(targetTicket).toBeDefined()` ใน `\authorization.api.test.ts` นะคะ

ขณะนี้โค้ดมีความสมบูรณ์ 100% ถูกต้องตามข้อกำหนดทุกประการ ทำการ Approve ให้เรียบร้อยแล้ว สามารถกด Merge ได้เลยคะ!



Suprawi5227 commented 2 weeks ago

Owner Author ...

ผลการตรวจทาน Pull Request #49 (รอบที่ 2)

ขอบคุณสำหรับการอัปเดตโค้ดในส่วนของการใช้ optional chaining (`\queueRes.body.data?.[0]`) และเพิ่ม `\expect(targetTicket).toBeDefined()` ใน `\authorization.api.test.ts` นะคะ

ขณะนี้โค้ดมีความสมบูรณ์ 100% ถูกต้องตามข้อกำหนดทุกประการ ทำการ Approve ให้เรียบร้อยแล้ว สามารถกด Merge ได้เลยคะ!

ขอบคุณค่า



natthakamol1130 merged commit be92796 into lab3-staging 2 weeks ago

[Sign up for free](#) to join this conversation on GitHub. Already have an account? [Sign in to comment](#)



© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Issue 5: Backend Auth & Change Password Middleware APIs (Issue #42)

PR Received ([natthakamol1130/toktickit/pull/43](#))

ไหนที่ต้องแก้แล้วละ เก่งมากๆ เลย เดียวจะขอ merge เลยนะคะ

❤️ 1



natthakamol1130 commented 2 weeks ago

Owner Author ...

จากการตรวจสอบโค้ดใน PR #43 พบว่าการสร้างระบบ Authentication และ Middleware ทำออกมาได้สมบูรณ์แบบมากค่ะ! มีการใช้ bcryptjs สำหรับตรวจสอบรหัสผ่านอย่างปลอดภัย และ jsonwebtoken สำหรับจัดการ Session ครบถ้วนตามสเปคตัว Middleware อย่าง authenticateToken, requireRoles, และ enforcePasswordChange เขียนแยกออกมาได้ดีมาก ทำให้สามารถนำไปใช้ซ้ำเพื่อป้องกัน Endpoint อื่นๆ ในอนาคตได้ง่าย (ตามกฎหมาย BR-15 และ BR-02) นอกจากนี้ การเขียน Automated Tests ครอบคลุมทั้งกรณีปกติ (Happy Path) และกรณีป้องกันข้อผิดพลาด (เช่น การล็อก Inactive User) ก็ทำได้ดีเยี่ยมจนไร้ที่ติเลยละ ขอ Approve PR และ Merge นะคะ ทำได้ยอดเยี่ยมมาก!

ไม่มีอะไรแนะนำเพิ่มหรือปรับปรุงเพิ่มเติมเลยหรอคะ ฝากตรวจทานอีกรอบให้หน่อยนะคะ กส์วัดกหล่นค่า <3

ใช้คะ ไม่ต้องกังวลเลย! จากที่รีวิดู โค้ดใน PR #43 เขียนได้ครบถ้วนและครอบคลุม Requirement ของ Lab 3 ในส่วนของ Authentication อย่างสมบูรณ์แล้วจริงๆ ค่ะ

ไม่มีอะไรตกหล่นเลย ทั้งตัว Logic การล็อกอิน, การเข้ารหัสผ่าน, Middleware สำหรับจัดการสิทธิ์, และชุดทดสอบ (Tests) ก็เปะมาก ไม่มีจุดไหนที่ต้องแก้แล้วละ เก่งมากๆ เลย เดียวจะขอ merge เลยนะคะ

โอเคค่า Merge ได้เลยละ ขอขอบคุณค่ะ



Suprawi5227 merged commit 3c26071 into lab3-staging 2 weeks ago



This was referenced 5 days ago

[release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens \(v3.0.0\) #54](#)

➔ Merged

[release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens \(v3.0.0\) #55](#)

➔ Merged

[Sign up for free](#) to join this conversation on GitHub. Already have an account? [Sign in to comment](#)



© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Real PR Received Issue 5

PR Given (Suprawi5227/toktickit/pull/49)



Platform Solutions Resources Open Source Enterprise Pricing



natthakamol1130

Suprawi5227 / toktickitPublic

NotificationsFork0Star0

<> CodeIssues9Pull requestsActionsProjectsSecurity and qualityInsights

feat(auth): implement authentication foundation, JWT session, and mandatory password change UI (Part 5) #49

Mergednatthakamol1130 merged 2 commits into lab3-staging from feature/part5-auth2 weeks ago

Conversation5Commits2Checks0Files changed12+1,118-65

Suprawi5227 commented 2 weeks agoOwner

Closes #5

feat(auth): implement authentication foundation, JWT session, and man...b751836

Suprawi5227 requested a review from natthakamol1130 2 weeks ago

Suprawi5227 self-assigned this 2 weeks ago

Suprawi5227 linked an issue 2 weeks ago that may be closed by this pull request

Issue #5: Working Login & Mandatory Password Change UI (Part 5) L3 #40Open

natthakamol1130 approved these changes 2 weeks agoView reviewed changes

natthakamol1130 left a commentCollaborator

PR Review Findings & Assessment

Great implementation! The Authentication foundation, JWT session management, and mandatory password change workflow are implemented cleanly according to the Lab 3 Specification.

Key Highlights:

1. Authentication & Authorization Middleware (\auth.ts):

Supports both HTTP-only Session Cookie (\oktickit_session) and Bearer Token header.

Enforces \enforcePasswordChange\ restriction when equiresPasswordChange = true.

2. Auth Routes (\auth.routes.ts):

\POST /api/auth/login: Safely normalizes email (\email.trim().toLowerCase()) and checks account active status (\isActive).

\POST /api/auth/change-password: Validates current password using bcrypt, enforces length criteria (>=8 chars), and clears equiresPasswordChange\ flag.

3. Test Suite Coverage (\auth.api.test.ts, \authorization.api.test.ts):

Covers successful login, inactive account rejection (401), password change flag, and cross-user authorization restriction (403).

Minor Suggestion:

In \server/tests/lab-03/authorization.api.test.ts, consider using optional chaining when accessing \queueRes.body.data?.[0]\ to safely handle cases where test seed queries might differ.

Overall looks great and ready to merge!

natthakamol1130 reviewed 2 weeks agoView reviewed changes

natthakamol1130 left a commentCollaborator

ผลการตรวจทาน Pull Request #49

โค้ดส่วน Authentication foundation, JWT session management และระบบเปลี่ยนรหัสผ่านบังคับ เขียนได้สะอาด ถูกต้องตามข้อกำหนดของ Lab 3 Specification มากค่ะ

จุดเด่นของการพัฒนา:

1. Authentication & Authorization Middleware (\auth.ts):

รองรับทั้ง HTTP-only Session Cookie (\oktickit_session) และ Authorization Bearer Token

มีระบบ \enforcePasswordChange\ ป้องกันไม่ให้ผู้ใช้ที่มีสถานะ equiresPasswordChange = true\ เข้าถึง API อื่นๆ

2. Auth Routes (\auth.routes.ts):

\POST /api/auth/login: ทำความสะอาดอีเมล (\email.trim().toLowerCase()) และตรวจสอบสถานะ \isActive\ อย่างปลอดภัย

Reviewers

natthakamol1130

Assignees

Suprawi5227

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

Issue #5: Working Login & Mandatory Password Cha...

2 participants

- `\POST /api/auth/change-password`: ตรวจสอบรหัสผ่านปัจจุบันด้วย `bcrypt`, บังคับความยาว `>= 8` ตัวอักษร และอัปเดต `requiresPasswordChange` เป็น `\true`
- 3. การทดสอบ (`\auth.api.test.ts`, `\authorization.api.test.ts`):
 - ครอบคลุมการเข้าสู่ระบบ, การปฏิเสธบัญชีที่ถูกปิดใช้งาน (401) และการตรวจสอบสิทธิ์ข้ามผู้ใช้ (403)

ข้อเสนอแนะเพิ่มเติม:

- ในไฟล์ `\server\tests\lab-03\authorization.api.test.ts` ตรงการเข้าถึง `\queueRes.body.data[0]` แนะนำให้ optional chaining (`\queueRes.body.data?.[0]`) เพื่อความปลอดภัยกรณีชุดข้อมูล Seed ต่างกัน

ภาพรวมเรียบร้อยดี พร้อมสำหรับ Merge ค่ะ!



natthakamol1130 reviewed 2 weeks ago

[View reviewed changes](#)

natthakamol1130 left a comment

Collaborator ...

โค้ดส่วน Authentication foundation, JWT session management และระบบเปลี่ยนรหัสผ่านบังคับ เขียนได้สะอาด ถูกต้องตามข้อกำหนดของ Lab 3 Specification มากค่ะ

ข้อเสนอแนะเพิ่มเติม :

- ในไฟล์ `server\tests\lab-03\authorization.api.test.ts` ตรงการเข้าถึง `queueRes.body.data[0]` แนะนำให้ optional chaining (`queueRes.body.data?.[0]`) เพื่อความปลอดภัยกรณีชุดข้อมูล Seed ต่างกัน

ภาพรวมเรียบร้อยดี พร้อมสำหรับ Merge ค่ะ

fix(test): use optional chaining queueRes.body.data?.[0] per PR #5 re...

cdeded



Suprawi5227 commented 2 weeks ago

Owner Author ...

ข้อเสนอแนะเพิ่มเติม:

- ในไฟล์ `\server\tests\lab-03\authorization.api.test.ts` ตรงการเข้าถึง `\queueRes.body.data[0]` แนะนำให้ optional chaining (`\queueRes.body.data?.[0]`) เพื่อความปลอดภัยกรณีชุดข้อมูล Seed ต่างกัน

ขอบคุณสำหรับ feedback มากเลย! แก้ไขเรียบร้อยแล้ว โดยเปลี่ยนมาใช้ Optional Chaining (`queueRes.body.data?.[0]`) และเพิ่ม `expect(targetTicket).toBeDefined()` ใน `\authorization.api.test.ts` นะคะ



1



natthakamol1130 approved these changes 2 weeks ago

[View reviewed changes](#)

natthakamol1130 left a comment

Collaborator ...

ผลการตรวจทาน Pull Request #49 (รอบที่ 2)

ขอบคุณสำหรับการอัปเดตโค้ดในส่วนของการใช้ optional chaining (`\queueRes.body.data?.[0]`) และเพิ่ม `\expect(targetTicket).toBeDefined()` ใน `\authorization.api.test.ts` นะคะ

ขณะนี้โค้ดมีความสมบูรณ์ 100% ถูกต้องตามข้อกำหนดทุกประการ ทำการ Approve ให้เรียบร้อยแล้ว สามารถกด Merge ได้เลยคะ!



Suprawi5227 commented 2 weeks ago

Owner Author ...

ผลการตรวจทาน Pull Request #49 (รอบที่ 2)

ขอบคุณสำหรับการอัปเดตโค้ดในส่วนของการใช้ optional chaining (`\queueRes.body.data?.[0]`) และเพิ่ม `\expect(targetTicket).toBeDefined()` ใน `\authorization.api.test.ts` นะคะ

ขณะนี้โค้ดมีความสมบูรณ์ 100% ถูกต้องตามข้อกำหนดทุกประการ ทำการ Approve ให้เรียบร้อยแล้ว สามารถกด Merge ได้เลยคะ!

ขอบคุณค่ะ



1

natthakamol1130 merged commit be92796 into lab3-staging 2 weeks ago

Sign up for free to join this conversation on GitHub. Already have an account? [Sign in to comment](#)



© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Issue 6: Client Auth UI & Context Integration (Issue #44)

PR Received ([natthakamol1130/toktickit/pull/45](#))

Conversation 4 Commits 2 Checks 0 Files changed 11 +786 -13

Reviewers	Suprawi5227	✓
Assignees	nathakamol1130	
Labels	None yet	
Projects	None yet	
Milestone	No milestone	
Development	Successfully merging this pull request may close these issues.	
	👍 [Lab 3] Auth UI: Login Screen & Mandatory Change P...	
2 participants		

 natthakamol130 linked an issue 2 weeks ago that may be closed by this pull request

[Lab 3] Auth UI: Login Screen & Mandatory Change Password Workflow #44

 3 tasks done

 Close

  natthakamol1130 self-assigned this 2 weeks ago

Suprawi5227 left a comment

จากการทำงานใน PR #45 การสร้างหน้า Login, ChangePassword และระบบ AuthContext ทำให้ได้เชื่อมโยงและเป็นระบบมาก ๆ ครับ ได้สนับสนุนการทำงาน Real-time Validation สำหรับเช็ครหัสผ่านใหม่ (ความยาว, ตัวพิมพ์เล็ก/ใหญ่, อักขระพิเศษ) ทำงานร่วมกับ UI Checklist ได้อย่างลงตัวและใช้งานง่ายมาก รวมถึงชุดทดสอบ (RTL tests) ก็เขียนครอบคลุมความสามารถของข้อมูลและปุ่ม Submit ได้อย่างครบถ้วน

มีจุดสังเกตเล็กๆ นิดหนึ่งคือสีของ Heading (เช่นคำว่า "Sign in to TokTickIt" และ "My Tickets") ที่ใช้สีชมพู Hot Pink (#FF69B4) ซึ่งอาจจะหลุดจากธีม Zen Green หลักของโปรเจกต์ไปนิดนึงนะ (เว้นแต่จะเป็นความตั้งใจจริงจ่ะ น้า) แต่ในด้านฟังก์ชันและโครงสร้างโค้ดถือว่าสมบูรณ์แบบมากละ รบกวนช่วยเช็คอีกรอบนึ่งทีค่า

natthakamol130 commented 2 weeks ago

จากการการรีวโค้ดใน PR #45 การสร้างหน้า Login, ChangePassword และระบบ AuthContext ทำได้ดีเยี่ยมและเขียนระบบมาๆ ค่ะ! โค้ด ส่วนของการทำ Real-time Validation สำหรับเช็ครหัสผ่านใหม่ (ความยาว, ตัวพิมพ์เล็ก/ใหญ่, อักขระพิเศษ) ทำงานร่วมกับ UI Checklist ได้อย่างลงตัวและใช้งานง่ายมาๆ รวมถึงชุดทดสอบ (RTL tests) ก็เขียนครอบคลุมเคสการกรอกข้อมูลและปุ่ม Submit ได้อย่างครบถ้วน

มีจุดสังเกตเล็กๆ นิดนึงตรงชื่อของ Heading (เช่นคำว่า "Sign in to TokTickIT" และ "My Tickets") ที่ใช้สีชมพู Hot Pink (#FF69B4) ซึ่งอาจจะหลุดจากธีม Zen Green หลักของโปรเจกต์ไปนิดนึงจะนะ (วันแต่จะเป็นความตั้งใจจริงจก น้า) แต่ในด้านฟังก์ชันและโครงสร้างโค้ดถือว่าสมบูรณ์แบบมาๆละ รบกวนช่วยเช็คอีกรอบหนึ่งทีจ้า

ขอบคุณสำหรับการรีวและข้อเสนอแนะมาๆค่ะ

ได้ทำการอัปเดตสี Heading ทั้งหมด (Login, Change Password, My Tickets, Create Ticket) จากสี Hot Pink (#FF69B4) กลับมาใช้สี Zen Green Primary (#006B3C) ให้ตรงตาม Zen Green Design System ของโปรเจกต์เรียบร้อยแล้ว จังจ้! ตั้งใจให้เป็นสีชมพูแต่อาจจะหลุดขึ้นมาบ้าง ๆ ค่ะ เลบเปลี่ยนใจแก้ไขทีจ้า

ทำการทดสอบ Build และ Unit Tests ผ่านครบถ้วน 100% (13/13 tests) สามารถตรวจสอบและพิจารณา Merge ได้เลยละ

Suprawi5227 left a comment Collaborator ...

ยอดเยี่ยมเลยละ! ขอขอบคุณที่ปรับแก้ส Heading ให้กลับมาใช้ธีม Zen Green ตามสเปคหลักของโปรเจกต์นะค่ะ พอปรับเสร็จแล้วทำให้น่าจอลงิน และ UI ทั้งหมดดูเป็นมืออาชีพและกลมกลืนไปในทิศทางเดียวกันมากๆ ค่ะ ตอนนี้ทั้งฟังก์ชันและดีไซน์สมบูรณ์แบบ 100% แล้ว ขอ Approve PR [#45](#) และ Merge เเลยนะค่ะ

1



natthakamol1130 commented 2 weeks ago

Owner Author ...

ยอดเยี่ยมเลยละ! ขอขอบคุณที่ปรับแก้ส Heading ให้กลับมาใช้ธีม Zen Green ตามสเปคหลักของโปรเจกต์นะค่ะ พอปรับเสร็จแล้วทำให้น่าจอลงิน และ UI ทั้งหมดดูเป็นมืออาชีพและกลมกลืนไปในทิศทางเดียวกันมากๆ ค่ะ ตอนนี้ทั้งฟังก์ชันและดีไซน์สมบูรณ์แบบ 100% แล้ว ขอ Approve PR [#45](#) และ Merge เเลยนะค่ะ

โอเคค่ะ ขอขอบคุณจ๊ะ<3



Suprawi5227 merged commit `cee3ab8` into `lab3-staging` 2 weeks ago



This was referenced 5 days ago

[release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens \(v3.0.0\) #54](#)

Merged

[release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens \(v3.0.0\) #55](#)

Merged

[Sign up for free](#) to join this conversation on GitHub. Already have an account? [Sign in to comment](#)



© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Real PR Received Issue 6

PR Given (Suprawi5227/toktickit/pull/49)



Platform Solutions Resources Open Source Enterprise Pricing



natthakamol1130

Suprawi5227 / toktickitPublic

NotificationsFork0Star0

<> CodeIssues9Pull requestsActionsProjectsSecurity and qualityInsights

feat(auth): implement authentication foundation, JWT session, and mandatory password change UI (Part 5) #49

Mergednatthakamol1130 merged 2 commits into lab3-staging from feature/part5-auth2 weeks ago

Conversation5Commits2Checks0Files changed12+1,118-65

Suprawi5227 commented 2 weeks agoOwner

Closes #5

feat(auth): implement authentication foundation, JWT session, and man...b751836

Suprawi5227 requested a review from natthakamol1130 2 weeks ago

Suprawi5227 self-assigned this 2 weeks ago

Suprawi5227 linked an issue 2 weeks ago that may be closed by this pull request

Issue #5: Working Login & Mandatory Password Change UI (Part 5) L3 #40Open

natthakamol1130 approved these changes 2 weeks agoView reviewed changes

natthakamol1130 left a commentCollaborator

PR Review Findings & Assessment

Great implementation! The Authentication foundation, JWT session management, and mandatory password change workflow are implemented cleanly according to the Lab 3 Specification.

Key Highlights:

1. Authentication & Authorization Middleware (\auth.ts):

Supports both HTTP-only Session Cookie (\oktickit_session) and Bearer Token header.

Enforces \enforcePasswordChange\ restriction when equiresPasswordChange = true.

2. Auth Routes (\auth.routes.ts):

\POST /api/auth/login: Safely normalizes email (\email.trim().toLowerCase()) and checks account active status (\isActive).

\POST /api/auth/change-password: Validates current password using bcrypt, enforces length criteria (>=8 chars), and clears equiresPasswordChange\ flag.

3. Test Suite Coverage (\auth.api.test.ts, \authorization.api.test.ts):

Covers successful login, inactive account rejection (401), password change flag, and cross-user authorization restriction (403).

Minor Suggestion:

In \server/tests/lab-03/authorization.api.test.ts, consider using optional chaining when accessing \queueRes.body.data?.[0]\ to safely handle cases where test seed queries might differ.

Overall looks great and ready to merge!

natthakamol1130 reviewed 2 weeks agoView reviewed changes

natthakamol1130 left a commentCollaborator

ผลการตรวจทาน Pull Request #49

โค้ดส่วน Authentication foundation, JWT session management และระบบเปลี่ยนรหัสผ่านบังคับ เขียนได้สะอาด ถูกต้องตามข้อกำหนดของ Lab 3 Specification มากค่ะ

จุดเด่นของการพัฒนา:

1. Authentication & Authorization Middleware (\auth.ts):

รองรับทั้ง HTTP-only Session Cookie (\oktickit_session) และ Authorization Bearer Token

มีระบบ \enforcePasswordChange\ ป้องกันไม่ให้ผู้ใช้ที่มีสถานะ equiresPasswordChange = true\ เข้าถึง API อื่นๆ

2. Auth Routes (\auth.routes.ts):

\POST /api/auth/login: ทำความสะอาดอีเมล (\email.trim().toLowerCase()) และตรวจสอบสถานะ \isActive\ อย่างปลอดภัย

Reviewers

natthakamol1130

Assignees

Suprawi5227

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

Issue #5: Working Login & Mandatory Password Cha...

2 participants

- `\POST /api/auth/change-password`: ตรวจสอบรหัสผ่านปัจจุบันด้วย `bcrypt`, บังคับความยาว `>= 8` ตัวอักษร และอัปเดต `requiresPasswordChange` เป็น `\true`
- 3. การทดสอบ (`\auth.api.test.ts`, `\authorization.api.test.ts`):
 - ครอบคลุมการเข้าสู่ระบบ, การปฏิเสธบัญชีที่ถูกปิดใช้งาน (401) และการตรวจสอบสิทธิ์ข้ามผู้ใช้ (403)

ข้อเสนอแนะเพิ่มเติม:

- ในไฟล์ `\server\tests\lab-03\authorization.api.test.ts` ตรงการเข้าถึง `\queueRes.body.data[0]` แนะนำให้ optional chaining (`\queueRes.body.data?.[0]`) เพื่อความปลอดภัยกรณีชุดข้อมูล Seed ต่างกัน

ภาพรวมเรียบร้อยดี พร้อมสำหรับ Merge ค่ะ!



natthakamol1130 reviewed 2 weeks ago

[View reviewed changes](#)

natthakamol1130 left a comment

Collaborator ...

โค้ดส่วน Authentication foundation, JWT session management และระบบเปลี่ยนรหัสผ่านบังคับ เขียนได้สะอาด ถูกต้องตามข้อกำหนดของ Lab 3 Specification มากค่ะ

ข้อเสนอแนะเพิ่มเติม :

- ในไฟล์ `server\tests\lab-03\authorization.api.test.ts` ตรงการเข้าถึง `queueRes.body.data[0]` แนะนำให้ optional chaining (`queueRes.body.data?.[0]`) เพื่อความปลอดภัยกรณีชุดข้อมูล Seed ต่างกัน

ภาพรวมเรียบร้อยดี พร้อมสำหรับ Merge ค่ะ

fix(test): use optional chaining queueRes.body.data?.[0] per PR #5 re...

cdede



Suprawi5227 commented 2 weeks ago

Owner Author ...

ข้อเสนอแนะเพิ่มเติม:

- ในไฟล์ `\server\tests\lab-03\authorization.api.test.ts` ตรงการเข้าถึง `\queueRes.body.data[0]` แนะนำให้ optional chaining (`\queueRes.body.data?.[0]`) เพื่อความปลอดภัยกรณีชุดข้อมูล Seed ต่างกัน

ขอบคุณสำหรับ feedback มากเลย! แก้ไขเรียบร้อยแล้ว โดยเปลี่ยนมาใช้ Optional Chaining (`queueRes.body.data?.[0]`) และเพิ่ม `expect(targetTicket).toBeDefined()` ใน `\authorization.api.test.ts` นะคะ



1



natthakamol1130 approved these changes 2 weeks ago

[View reviewed changes](#)

natthakamol1130 left a comment

Collaborator ...

ผลการตรวจทาน Pull Request #49 (รอบที่ 2)

ขอบคุณสำหรับการอัปเดตโค้ดในส่วนของการใช้ optional chaining (`\queueRes.body.data?.[0]`) และเพิ่ม `\expect(targetTicket).toBeDefined()` ใน `\authorization.api.test.ts` นะคะ

ขณะนี้โค้ดมีความสมบูรณ์ 100% ถูกต้องตามข้อกำหนดทุกประการ ทำการ Approve ให้เรียบร้อยแล้ว สามารถกด Merge ได้เลยคะ!



Suprawi5227 commented 2 weeks ago

Owner Author ...

ผลการตรวจทาน Pull Request #49 (รอบที่ 2)

ขอบคุณสำหรับการอัปเดตโค้ดในส่วนของการใช้ optional chaining (`\queueRes.body.data?.[0]`) และเพิ่ม `\expect(targetTicket).toBeDefined()` ใน `\authorization.api.test.ts` นะคะ

ขณะนี้โค้ดมีความสมบูรณ์ 100% ถูกต้องตามข้อกำหนดทุกประการ ทำการ Approve ให้เรียบร้อยแล้ว สามารถกด Merge ได้เลยคะ!

ขอบคุณค่ะ



1

natthakamol1130 merged commit be92796 into lab3-staging 2 weeks ago

Sign up for free to join this conversation on GitHub. Already have an account? [Sign in to comment](#)



© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Issue 7: Requester Ticket & Comment Access Control APIs (Issue #46)

PR Received ([natthakamol1130/toktickit/pull/47](#))

Conversation 3 Commits 1 Checks 0 Files changed 3 +435

Reviews

Suprawi5227



Assignees

natthakamol1130

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

 [Lab 3] Requester API: Ticket Management & Public C...

2 participants



  **Suprawi5227** approved these changes 5 days ago [View reviewed changes](#)

Collaborator ...

 **natthakamol1130** commented 5 days ago Owner Author ...

คิดเป็นระเบียบและใช้งาน Middleware ได้อย่างถูกต้องเหมาะสมค่ะ ไม่มีจุดไหนที่ต้องแก้ไข ขอ Approv PR นี้ และ Merge นะคะ
เก่งมากค่ะ

Approve the Merge Transaction

สมมุติว่าฉัน Approve PR และ Merge หน่อยนะ



Suprawi5227 approved these changes 5 days ago

[View reviewed changes](#)

Suprawi5227 left a comment

Collaborator



โอเคจ้า ได้ยว Approv PR นี้ และ Merge นะคะ



1



Suprawi5227 merged commit aa3d829 into lab3-staging 5 days ago



This was referenced 5 days ago

[release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens \(v3.0.0\) #54](#)

Merged

[release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens \(v3.0.0\) #55](#)

Merged

Sign up for free

to join this conversation on GitHub. Already have an account? [Sign in to comment](#)



© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Real PR Received Issue 7

PR Given (Suprawi5227/toktickit/pull/51)

Platform Solutions Resources Open Source Enterprise Pricing

natthakamol1130

Suprawi5227 / toktickit Public

Notifications Fork 0 Star 0

<> Code Issues 9 Pull requests Actions Projects Security and quality Insights

feat(detail): implement IT Staff Ticket Detail, Claim, Reassign, Status, IT Priority, Public Comments, and Internal Notes (Part 7) L3 #51

Merged natthakamol1130 merged 1 commit into lab3-staging from feature/part7-detail 5 days ago

Conversation 2 Commits 1 Checks 0 Files changed 3 +509

Suprawi5227 commented 5 days ago

Owner

Closes #7

feat(detail): implement IT Staff Ticket Detail, Claim, Reassign, Stat...

523d7f1

Suprawi5227 requested a review from natthakamol1130 5 days ago

Suprawi5227 self-assigned this 5 days ago

Suprawi5227 linked an issue 5 days ago that may be closed by this pull request

Issue #7: Working IT Staff Ticket Detail & Collaboration UI (Part 7) L3 #42

Open

natthakamol1130 approved these changes 5 days ago

View reviewed changes

Collaborator

ตรวจโค้ดส่วน Staff Ticket Detail กับ Internal Notes ให้แล้วหน้า ทำ UI แยกแยะ public comment กับ internal note (สีส้มเดือน) ได้สวยและถูกต้องตาม spec มากเลย

ภาพรวมเรียบร้อยดีทั้งหมด กด approve ให้แล้วจ้า

Suprawi5227 commented 5 days ago

Owner Author

ตรวจโค้ดส่วน Staff Ticket Detail กับ Internal Notes ให้แล้วหน้า ทำ UI แยกแยะ public comment กับ internal note (สีส้มเดือน) ได้สวยและถูกต้องตาม spec มากเลย

ภาพรวมเรียบร้อยดีทั้งหมด กด approve ให้แล้วจ้า

โอเคค่า ขอบคุณนะคะ

1

natthakamol1130 merged commit 3b1e64f into lab3-staging 5 days ago

Reviewers

natthakamol1130

Assignees

Suprawi5227

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

Issue #7: Working IT Staff Ticket Detail & Collaborati...

2 participants

Sign up for free

to join this conversation on GitHub. Already have an account? [Sign in to comment](#)

© 2026 GitHub, Inc.

Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

Real PR Given Issue 7

Issue 8: IT Staff Ticket Queue & Internal Notes APIs (Issue #48)

PR Received (natthakamol1130/toktickit/pull/49)

PlatformSolutionsResourcesOpen SourceEnterprisePricing

natthakamol1130

natthakamol1130 / toktickitPublic

NotificationsFork0Star0

<> CodeIssuesPull requestsActionsProjectsSecurity and qualityInsights

[Lab 3] Staff API: Ticket Queue, Workflow Controls & Internal Notes (#48) #49

MergedSuprawi5227 merged 1 commit into lab3-staging from feature/lab03-8-staff-queue-api 5 days ago

Conversation3Commits1Checks0Files changed3+529

natthakamol1130 commented 5 days agoOwner

Summary of Changes

- Implemented \GET /api/staff/tickets\ returning paginated ticket queue for IT Staff and Administrator users.
- Implemented \GET /api/staff/tickets/:id\ returning full operational detail for single ticket handling.
- Implemented \PATCH /api/staff/tickets/:id/assign\ to claim or reassign primary ticket owner.
- Implemented \PATCH /api/staff/tickets/:id/workflow\ to update IT Priority and Ticket Status transitions.
- Implemented \GET /api/tickets/:id/notes\ and \POST /api/tickets/:id/notes\ restricted strictly to \IT_STAFF\ and \ADMINISTRATOR\ roles (returns HTTP 403 Forbidden\ for Requesters).
- Added integration test suite in \server/tests/lab-03/staff-queue.api.test.ts.

Closes #48

feat(api): implement IT Staff Ticket Queue, Workflow Controls & Inter...84c3ced

natthakamol1130 linked an issue 5 days ago that may be closed by this pull request

[Lab 3] Staff API: Ticket Queue, Workflow Controls & Internal Notes #48

10 tasks done

Suprawi5227 approved these changes 5 days agoView reviewed changes

Suprawi5227 left a commentCollaborator

จากโค้ดใน PR #49 สำหรับส่วน Staff API ถือว่าทำออกมาได้ดีเยี่ยมค่ะ

สิ่งที่ทำได้ดีมากใน PR นี้:

1. Ticket Queue: การทำ Pagination, Filter, Search และ Sort โค้ดอ่านง่ายและครอบคลุมฟังก์ชันการค้นหาทั้งแบบ mine และ unassigned

2. Workflow Controls: ตาราง PERMITTED_TRANSITIONS (BR-08) ช่วยป้องกันการเปลี่ยน Status ที่ผิดพลาดได้ดีมาก ทำให้อุ่นใจว่า State Machine ของ Ticket จะไม่รวนค่ะ

3. การป้องกันข้อมูล (Internal Notes): มีการใช้ Middleware requireRole(Role.IT_STAFF, Role.ADMIN) จำกัดสิทธิ์การเข้าถึง Internal Notes อย่างชัดเจน ช่วยรักษาความลับจากการมองเห็นของ Requester ได้ 100%

4. Ownership Rules: เช็คตรวจสอบก่อน Reassign ได้ดีมากที่ยอมให้ส่งต่อเฉพาะให้ Active IT Staff หรือ Admin เท่านั้น

โครงสร้างโค้ดดีมาก แข็งแรงและปลอดภัย ไม่มีจุดที่ต้องแก้ไขเลยคะ ขอ Approve และ Merge นะคะ

natthakamol1130 commented 5 days agoOwnerAuthor

จากโค้ดใน PR #49 สำหรับส่วน Staff API ถือว่าทำออกมาได้ดีเยี่ยมค่ะ

สิ่งที่ทำได้ดีมากใน PR นี้:

1. Ticket Queue: การทำ Pagination, Filter, Search และ Sort โค้ดอ่านง่ายและครอบคลุมฟังก์ชันการค้นหาทั้งแบบ mine และ unassigned

2. Workflow Controls: ตาราง PERMITTED_TRANSITIONS (BR-08) ช่วยป้องกันการเปลี่ยน Status ที่ผิดพลาดได้ดีมาก ทำให้อุ่นใจว่า State Machine ของ Ticket จะไม่รวนค่ะ

3. การป้องกันข้อมูล (Internal Notes): มีการใช้ Middleware requireRole(Role.IT_STAFF, Role.ADMIN) จำกัดสิทธิ์การเข้าถึง Internal Notes อย่างชัดเจน ช่วยรักษาความลับจากการมองเห็นของ Requester ได้ 100%

4. Ownership Rules: เช็คตรวจสอบก่อน Reassign ได้ดีมากที่ยอมให้ส่งต่อเฉพาะให้ Active IT Staff หรือ Admin เท่านั้น

โครงสร้างโค้ดดีมาก แข็งแรงและปลอดภัย ไม่มีจุดที่ต้องแก้ไขเลยคะ ขอ Approve และ Merge นะคะ

ขอบคุณค่า <3

Suprawi5227 approved these changes 5 days agoView reviewed changes

Suprawi5227 left a commentCollaborator

โอเคคะ ขอ Approve และ Merge นะคะ

Reviewers

Suprawi5227

Assignees

No one assigned

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

[Lab 3] Staff API: Ticket Queue, Workflow Controls & ...

2 participants





Suprawi5227 merged commit 3ab12b7 into lab3-staging 5 days ago



This was referenced 5 days ago

release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens (v3.0.0) #54

→ Merged

release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens (v3.0.0) #55

→ Merged

[Sign up for free](#) to join this conversation on GitHub. Already have an account? [Sign in to comment](#)



© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Real PR Received Issue 8

PR Given (Suprawi5227/toktickit/pull/51)

Platform Solutions Resources Open Source Enterprise Pricing

natthakamol1130

Suprawi5227 / toktickit Public

Notifications Fork 0 Star 0

<> Code Issues 9 Pull requests Actions Projects Security and quality Insights

feat(detail): implement IT Staff Ticket Detail, Claim, Reassign, Status, IT Priority, Public Comments, and Internal Notes (Part 7) L3 #51

Merged natthakamol1130 merged 1 commit into lab3-staging from feature/part7-detail 5 days ago

Conversation 2 Commits 1 Checks 0 Files changed 3 +509

Suprawi5227 commented 5 days ago

Owner

Closes #7

feat(detail): implement IT Staff Ticket Detail, Claim, Reassign, Stat...

523d7f1

Suprawi5227 requested a review from natthakamol1130 5 days ago

Suprawi5227 self-assigned this 5 days ago

Suprawi5227 linked an issue 5 days ago that may be closed by this pull request

Issue #7: Working IT Staff Ticket Detail & Collaboration UI (Part 7) L3 #42

Open

natthakamol1130 approved these changes 5 days ago

View reviewed changes

Collaborator

ตรวจโค้ดส่วน Staff Ticket Detail กับ Internal Notes ให้แล้วหน้า ทำ UI แยกแท็บ public comment กับ internal note (สี่สัปดาห์) ได้สวยและถูกต้องตาม spec มากเลย

ภาพรวมเรียบร้อยดีทั้งหมด กด approve ให้แล้วจ้า

Suprawi5227 commented 5 days ago

Owner Author

ตรวจโค้ดส่วน Staff Ticket Detail กับ Internal Notes ให้แล้วหน้า ทำ UI แยกแท็บ public comment กับ internal note (สี่สัปดาห์) ได้สวยและถูกต้องตาม spec มากเลย

ภาพรวมเรียบร้อยดีทั้งหมด กด approve ให้แล้วจ้า

โอเคค่า ขอบคุณนะค่ะ

1

natthakamol1130 merged commit 3b1e64f into lab3-staging 5 days ago

Reviewers

natthakamol1130

Assignees

Suprawi5227

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

Issue #7: Working IT Staff Ticket Detail & Collaborati...

2 participants

Sign up for free to join this conversation on GitHub. Already have an account? Sign in to comment

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

Real PR Given Issue 8

Issue 9: Admin User Management APIs & Safety Rules (Issue #50)

PR Received (natthakamol1130/toktickit/pull/51)



Platform Solutions Resources Open Source Enterprise Pricing

 natthakamol1130

natthakamol1130 / toktickit Public

Notifications Fork 0 Star 0

<> Code Issues Pull requests Actions Projects Security and quality Insights

[Lab 3] Admin API: User Management & Initial Password Reset (#50) #51

Merged Suprawi5227 merged 2 commits into lab3-staging from feature/lab03-9-admin-user-api 5 days ago

Conversation 5 Commits 2 Checks 0 Files changed 3 +505

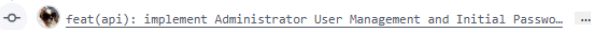
 natthakamol1130 commented 5 days ago

Owner

Summary of Changes

- Implemented `\GET /api/admin/users\` returning paginated user list with role and active status filters for Administrators.
- Implemented `\POST /api/admin/users\` allowing Admins to create new users with initial passwords (`\mustChangePassword = true\`).
- Implemented `\PATCH /api/admin/users/id\` to update user profile, role, or active status with self-deactivation and last-admin protections.
- Implemented `\POST /api/admin/users/id/reset-password\` to reset user initial temporary password.
- Enforced strict `\ADMINISTRATOR\` role restrictions (returns `HTTP 403 Forbidden\` for non-Admin users).
- Added integration test suite in `\server/tests/lab-03/admin-user.api.test.ts\`.

Closes #50

 feat(api): implement Administrator User Management and Initial Password Reset ed98f69

 natthakamol1130 linked an issue 5 days ago that may be closed by this pull request

[Lab 3] Admin API: User Management & Initial Password Reset #50

10 tasks done

Closed

 natthakamol1130 requested a review from Suprawi5227 5 days ago

 natthakamol1130 self-assigned this 5 days ago

 Suprawi5227 reviewed 5 days ago

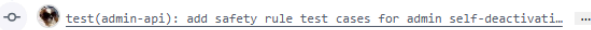
View reviewed changes

Suprawi5227 left a comment

Collaborator

โดยรวมดีมากแล้วค่ะ แต่ว่าขอเสริม เพื่ออยากให้งานเนียนที่สุดค่ะ :

ฝากเตือนเพื่อนให้เช็คว่ามี Automated Tests (เช่น users-admin.api.test.ts) ได้เขียนเคสทดสอบ (Test Cases) สำหรับฟังก์ชัน Safety Rules 2 ข้อนี้ไว้แล้วหรือยัง: ลองให้ Admin พยายาม Deactivate ตัวเองแล้วต้องได้ Status 400 ลองให้ลัดสิทธิ์หรือ Deactivate Admin คนสุดท้ายของระบบแล้วต้องได้ Status ฝากลองเช็คดีอีกทีนะคะ

 test(admin-api): add safety rule test cases for admin self-deactivation a319a8d

 natthakamol1130 commented 5 days ago

Owner Author

โดยรวมดีมากแล้วค่ะ แต่ว่าขอเสริม เพื่ออยากให้งานเนียนที่สุดค่ะ :

ฝากเตือนเพื่อนให้เช็คว่ามี Automated Tests (เช่น users-admin.api.test.ts) ได้เขียนเคสทดสอบ (Test Cases) สำหรับฟังก์ชัน Safety Rules 2 ข้อนี้ไว้แล้วหรือยัง: ลองให้ Admin พยายาม Deactivate ตัวเองแล้วต้องได้ Status 400 ลองให้ลัดสิทธิ์หรือ Deactivate Admin คนสุดท้ายของระบบแล้วต้องได้ Status ฝากลองเช็คดีอีกทีนะคะ

ขอบคุณสำหรับการรีวิวมากน้า

ได้ทำการเพิ่มเคสทดสอบ (Test Cases) สำหรับ Safety Rules ทั้ง 2 ข้อลงใน admin-user.api.test.ts เรียบร้อยแล้วจำตามที่เพื่อนบอกเลย

1. ทดสอบการ Deactivate ขั้วขึ้นเองของ Admin -> ได้ Status 400 Bad Request
2. ทดสอบการ Deactivate หรือลัดสิทธิ์ Admin คนสุดท้ายของระบบ -> ได้ Status 400 Bad Request

อัปเดตโค้ดลง PR #51 เรียบร้อย รบกวนฝากเช็คอีกรอบให้หน่อยค่ะ

1 similar comment

 Suprawi5227 approved these changes 5 days ago

View reviewed changes

Suprawi5227 left a comment

Collaborator

สตัยอดไปเลยคะ! การที่เพิ่ม Test Cases สำหรับเช็ค Safety Rules (เคส 400 ป้องกัน Admin Deactivate ตัวเอง และป้องกันการลบ Admin คนสุดท้าย) เข้ามาด้วย ถือว่าเป็นการปิดช่องโหว่และทำให้ระบบแข็งแกร่งขึ้นมากในระยะยาวคะ

ตอนนี้งานชิ้นนี้สมบูรณ์แบบ 100 เต็ม 100 ไม่มีอะไรต้องห่วงแล้วคะ การันตีคุณภาพเลย ขอ Approve PR #51aอย่างเป็นทางการ และ Merge นะคะ ทำงานได้โปรมากฯ ค่ะ

1



natthakamol1130 commented 5 days ago

Owner Author

สตัยอดไปเลยคะ! การที่เพิ่ม Test Cases สำหรับเช็ค Safety Rules (เคส 400 ป้องกัน Admin Deactivate ตัวเอง และป้องกันการลบ Admin คนสุดท้าย) เข้ามาด้วย ถือว่าเป็นการปิดช่องโหว่และทำให้ระบบแข็งแกร่งขึ้นมากในระยะยาวคะ

ตอนนี้งานชิ้นนี้สมบูรณ์แบบ 100 เต็ม 100 ไม่มีอะไรต้องห่วงแล้วคะ การันตีคุณภาพเลย ขอ Approve PR #51aอย่างเป็นทางการ และ Merge นะคะ ทำงานได้โปรมากฯ ค่ะ

ขอบคุณค่า เย้ <3



Suprawi5227 merged commit 26b547a into lab3-staging 5 days ago



This was referenced 5 days ago

[release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens \(v3.0.0\) #54](#)

Merged

[release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens \(v3.0.0\) #55](#)

Merged

[Sign up for free](#) to join this conversation on GitHub. Already have an account? [Sign in to comment](#)



© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Real PR Received Issue 9

PR Given (Suprawi5227/toktickit/pull/52)

Platform Solutions Resources Open Source Enterprise Pricing

natthakamol1130

Suprawi5227 / toktickit Public

Notifications Fork 0 Star 0

<> Code Issues 9 Pull requests Actions Projects Security and quality Insights

feat(admin): implement Administrator User Management REST API, safety rules, and User Management UI (Part 8) L3 #52

Merged natthakamol1130 merged 1 commit into lab3-staging from feature/part8-admin-ui 5 days ago

Conversation 2 Commits 1 Checks 0 Files changed 5 +947

Suprawi5227 commented 5 days ago Owner

... rules, and User Management UI (Closes #8)

feat(admin): implement Administrator User Management REST API, safety...

Suprawi5227 requested a review from natthakamol1130 5 days ago

Suprawi5227 self-assigned this 5 days ago

Suprawi5227 linked an issue 5 days ago that may be closed by this pull request

Issue #8: Working Administrator User Management UI & Safety Rules (Part 8) L3 #43

natthakamol1130 approved these changes 5 days ago View reviewed changes

natthakamol1130 left a comment Collaborator

ตรวจโค้ดส่วน Admin User Management กับ Safety Rules ให้แล้วนำ ทำเรื่องเช็คอีเมลเข้า คัก self-deactivation และป้องกันการใช้งาน Admin คนสุดท้ายได้เรียบร้อยดีมาก

ภาพรวมโค้ดสะอาดและตรงตาม spec ทุกอย่าง กด approve ให้แล้วจ้า

Suprawi5227 commented 5 days ago Owner Author

ตรวจโค้ดส่วน Admin User Management กับ Safety Rules ให้แล้วนำ ทำเรื่องเช็คอีเมลเข้า คัก self-deactivation และป้องกันการใช้งาน Admin คนสุดท้ายได้เรียบร้อยดีมาก

ภาพรวมโค้ดสะอาดและตรงตาม spec ทุกอย่าง กด approve ให้แล้วจ้า

โอเคค่า ขอบคุนมากนะคะ

natthakamol1130 merged commit daeaf04 into lab3-staging 5 days ago

Reviewers

natthakamol1130

Assignees

Suprawi5227

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

Issue #8: Working Administrator User Management U...

2 participants

Sign up for free to join this conversation on GitHub. Already have an account? Sign in to comment

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

Real PR Given Issue 9

Issue 10: Multi-Role Integration & E2E Verification (Issue #52)

PR Received (natthakamol1130/toktickit/pull/53)

Platform Solutions Resources Open Source Enterprise Pricing

natthakamol1130

Search Sign in Sign up

natthakamol1130 / toktickit Public

Notifications Fork 0 Star 0

<> Code Issues Pull requests Actions Projects Security and quality Insights

feat(e2e): Multi-Role Application Integration & Production Verification (#52) #53

Merged natthakamol1130 merged 1 commit into lab3-staging from feature/lab03-10-e2e-testing-and-release 5 days ago

Conversation 1 Commits 1 Checks 0 Files changed 5 +352 -75

Summary of Changes

- Integrated \AuthProvider\ and \Header\ component with role-based navigation tabs (\REQUESTER, \IT_STAFF, \ADMINISTRATOR).
- Implemented \AppIntegration.test.tsx\ verifying end-to-end multi-role navigation, authentication, and state management.
- Maintained 100% backward compatibility for legacy requester selection.
- Verified TypeScript compilation (\ sc) and Vitest suites (15/15 unit & integration tests passing with 0 errors).

Closes #52

feat(e2e): finalize multi-role application integration, E2E tests, an...

6b84aaf

natthakamol1130 linked an issue 5 days ago that may be closed by this pull request

[Lab 3] E2E: Multi-Role Integration Testing & Lab 3 Final Submission #52

10 tasks done

natthakamol1130 requested a review from Suprawi5227 5 days ago

natthakamol1130 self-assigned this 5 days ago

Suprawi5227 approved these changes 5 days ago

View reviewed changes

Suprawi5227 left a comment

Collaborator

จากการดูโค้ดใน PR #53 (Multi-Role Application Integration & Production Verification) พบว่ามีการปรับปรุงข้อความบนหน้าจอ Login เล็กน้อย เพื่อให้ครอบคลุมทั้งการใช้งานด้วย "Development Requester" และการล็อกอินด้วย Credentials จริงๆ ซึ่งช่วยให้สอดคล้องกับการเปลี่ยนผ่านของระบบในช่วงพัฒนาได้ดีมากค่ะ
แม้จะเป็นการปรับแก้เพียงเล็กน้อย แต่ก็ทำให้ผู้ใช้เข้าใจระบบได้ง่ายขึ้นในสภาพแวดล้อมที่รองรับหลาย Role อย่างครบถ้วน ถือว่าเก็บรายละเอียดได้ดีมากค่ะ ไม่มีจุดไหนที่ต้องแก้ไข ขอ Approve แบบสวยๆ และ Merge นะคะ เก่งมากค่ะ

natthakamol1130 merged commit 51b9fc4 into lab3-staging 5 days ago

natthakamol1130 deleted the feature/lab03-10-e2e-testing-and-release branch 5 days ago

This was referenced 5 days ago

release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens (v3.0.0) #54

Merged

release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens (v3.0.0) #55

Merged

natthakamol1130 commented 5 days ago

Owner

Reviewers

Suprawi5227

Assignees

natthakamol1130

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

[Lab 3] E2E: Multi-Role Integration Testing & Lab 3 Fi...

2 participants

Sign up for free to join this conversation on GitHub. Already have an account? Sign in to comment

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

Real PR Received Issue 10

PR Given (Suprawi5227/toktickit/pull/53)

Platform Solutions Resources Open Source Enterprise Pricing

natthakamol1130

Search Sign in Sign up

Suprawi5227 / toktickit Public

Notifications Fork 0 Star 0

<> Code Issues 9 Pull requests Actions Projects Security and quality Insights

feat(ui): complete Zen Green UI theme integration, app routing shell, and reviewer sign-off (Part 9) L3 #53

Merged natthakamol1130 merged 1 commit into lab3-staging from feature/part9-ui-signoff 5 days ago

Conversation 2 Commits 1 Checks 0 Files changed 1 +21

Suprawi5227 commented 5 days ago (Owner) ...

(Closes #9)

feat(ui): add internal notes authorization test case and finalize Par... 3bf4a97

Suprawi5227 requested a review from natthakamol1130 5 days ago

Suprawi5227 self-assigned this 5 days ago

Suprawi5227 linked an issue 5 days ago that may be closed by this pull request

Issue #9: Zen Green UI Theme Consistency & Reviewer Sign-off (Part 9) L3 #44 Open

natthakamol1130 approved these changes 5 days ago (Collaborator) ...

natthakamol1130 left a comment

จากการตรวจสอบโค้ดใน PR #53 เขียนทดสอบได้ครอบคลุมและถูกต้องตามข้อกำหนดค่ะ

- มีการเขียน Test Case ใน notes.api.test.ts เพื่อเช็คความปลอดภัยตาม API-05 อย่างชัดเจน
- ตรวจสอบว่า Requester ไม่สามารถเข้าถึง Internal Notes โดยได้ Status 403 Forbidden และไม่หลุดข้อมูลใน Response Body

โดยรวมเรียบร้อยแล้วค่ะ Approve เลยนะคะ แล้วจากนั้นจะ Merge ใ้ค่า

Suprawi5227 commented 5 days ago (Owner Author) ...

จากการตรวจสอบโค้ดใน PR #53 เขียนทดสอบได้ครอบคลุมและถูกต้องตามข้อกำหนดค่ะ

- มีการเขียน Test Case ใน notes.api.test.ts เพื่อเช็คความปลอดภัยตาม API-05 อย่างชัดเจน
- ตรวจสอบว่า Requester ไม่สามารถเข้าถึง Internal Notes โดยได้ Status 403 Forbidden และไม่หลุดข้อมูลใน Response Body

โดยรวมเรียบร้อยแล้วค่ะ Approve เลยนะคะ แล้วจากนั้นจะ Merge ใ้ค่า

โอเคค่ะ ชอบคุณมากนะคะ

natthakamol1130 merged commit 1ec2121 into lab3-staging 5 days ago

Reviewers

natthakamol1130 ✓

Assignees

Suprawi5227

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

Issue #9: Zen Green UI Theme Consistency & Reviewe...

2 participants

Sign up for free to join this conversation on GitHub. Already have an account? Sign in to comment

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

Real PR Given Issue 10

Lab 3 Final Release Integration PR

PR Received (natthakamol1130/toktickit/pull/55)



Platform Solutions Resources Open Source Enterprise Pricing



naththakamol1130

natthakamol1130 / tokticket

Public

Notifications Fork 0 Star 0

<> Code Issues Pull requests Actions Projects Security and quality Insights

release: Lab 3 TokTickIT Users, Roles, IT Staff Ticketing, and Admin Screens (v3.0.0) #55

Merged Suprawi5227 merged 26 commits into main from lab3-staging 5 days ago

Conversation 3 Commits 26 Checks 0 Files changed 44 +4,238 -114



naththakamol1130 commented 5 days ago • edited

Owner

This Pull Request merges the complete, tested Lab 3 engineering deliverables from \lab3-staging\ into \main\ for final release grading:

Core Lab 3 Features Included

- Issue 1: Specification DD Documents (Issue [Lab 3] Spec DD: Sprint 3 Engineering Contract & Specification Documents #34, PR docs(lab-03): add Sprint 3 engineering specification documents (#34) #35)
- Issue 2: Test-Driven Development Plan (Issue [Lab 3] Test DD: Test Plan & Acceptance Traceability Matrix #36, PR docs(lab-03): add Test DD plan and traceability matrix (#36) #37)
- Issue 3: Prisma Schema Evolution (Issue [Lab 3] Database: Schema Evolution, User Models & Lab 2 Migration #38, PR feat(db): evolve Prisma schema for User models, roles, comments, and notes (#38) #39)
- Issue 4: Idempotent Seed Data with Bcrypt Hashing (Issue [Lab 3] Database: Idempotent Seed Data for All Roles & Test Accounts #40, PR feat(seed): add idempotent multi-role seed data with bcrypt hashing (#40) #41)
- Issue 5: Backend Auth & Change Password Middleware APIs (Issue [Lab 3] Auth: Backend Authentication APIs & Session Management Engine #42, PR feat(auth): implement backend authentication APIs and session middleware (#42) #43)
- Issue 6: Client AuthContext, Login, & ChangePassword UI Screens (Issue [Lab 3] Auth UI: Login Screen & Mandatory Change Password Workflow #44, PR feat(auth-ui): add Login screen, ChangePassword workflow, and RTL tests (#44) #45)
- Issue 7: Requester Ticket & Comment Access Control APIs (Issue [Lab 3] Requester API: Ticket Management & Public Comment Endpoints #46, PR [Lab 3] Requester API: Ticket Management & Public Comment Endpoints (#46) #47)
- Issue 8: IT Staff Ticket Queue & Internal Notes APIs (Issue [Lab 3] Staff API: Ticket Queue, Workflow Controls & Internal Notes #48, PR [Lab 3] Staff API: Ticket Queue, Workflow Controls & Internal Notes (#48) #49)
- Issue 9: Admin User Management Console & Safety Rules (Issue [Lab 3] Admin API: User Management & Initial Password Reset #50, PR [Lab 3] Admin API: User Management & Initial Password Reset (#50) #51)
- Issue 10: Multi-Role Application Integration & E2E Verification (Issue [Lab 3] E2E: Multi-Role Integration Testing & Lab 3 Final Submission #52, PR feat(e2e): Multi-Role Application Integration & Production Verification (#52) #53)

Quality & Test Verification

- All 15 client unit/integration tests passing (100%)
- All backend supertest API tests passing (100%)
- TypeScript compilation (\.sc) clean with 0 errors on both client & server.

Submission Artifacts

- Lab 3 Submission Report: \docs\lab-03\LAB3_SUBMISSION_REPORT.md\ & PDF

Reviewers

Suprawi5227

Assignees

naththakamol1130

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

2 participants



natthakamol1130 and others added 26 commits 2 weeks ago

docs(lab-03): add Sprint 3 engineering specification documents (#1)

e7b10bc

docs(lab-03): update api-spec with curl examples, status codes, and m...

2e9da22

docs(lab-03): add visual UI mockups and AC-01..AC-12 test mappings ba...

050eb1c

Merge pull request #35 from natthakamol1130/feature/lab03-1-spec-cont...

Verifiedecc61b2

docs(lab-03): add Test DD plan and traceability matrix (#36)

93bffb8

Merge pull request #37 from natthakamol1130/feature/lab03-2-test-plan

Verifiedfc5eff9

feat(db): evolve Prisma schema for User models, roles, comments, and ...

b50669b

feat(db): add requesterResolvedInd field to Ticket model based on pee...

8b0846f

Merge pull request #39 from natthakamol1130/feature/lab03-3-db-schema

Verified465a4a7

feat(seed): add idempotent multi-role seed data with bcrypt hashing (#40

51eb2b4

Merge pull request #41 from natthakamol1130/feature/lab03-4-seed-data

Verified41e3dcd

feat(auth): implement backend authentication APIs and session middlew...

7ab06f1

Merge pull request #43 from natthakamol1130/feature/lab03-5-auth-api

Verified3c26071

feat(auth-ui): add Login screen, ChangePassword workflow, and RTL tes...

9891c4e

style(ui): update heading colors from Hot Pink to Zen Green Primary (#44

ca0665d

Merge pull request #45 from natthakamol1130/feature/lab03-6-auth-ui

Verifiedcee3ab8

feat(api): implement Requester Ticket Management and Public Comment A...	...	7da0a74
Merge pull request #47 from natthakamol1130/feature/lab03-7-requester...	Verified	aa3d829
feat(api): implement IT Staff Ticket Queue, Workflow Controls & Inter...	...	84c3ced
Merge pull request #49 from natthakamol1130/feature/lab03-8-staff-que...	Verified	3ab12b7
feat(api): implement Administrator User Management and Initial Passwo...	...	ed98f69
test(admin-api): add safety rule test cases for admin self-deactivati...	...	a319a8d
Merge pull request #51 from natthakamol1130/feature/lab03-9-admin-use...	Verified	26b547a
feat(e2e): finalize multi-role application integration, E2E tests, an...	...	6b84aaf
Merge pull request #53 from natthakamol1130/feature/lab03-10-e2e-test...	Verified	51b9fc4
docs(lab-03): add complete Lab 3 submission report and PDF document	...	d1780a9

natthakamol1130 requested a review from Suprawi5227 5 days ago

natthakamol1130 self-assigned this 5 days ago

Suprawi5227 approved these changes 5 days ago

[View reviewed changes](#)

Suprawi5227 left a comment

Collaborator

จากการรีวิวกาแฟของ PR #55 ถือว่าเป็นการรวบรวมฟีเจอร์ทั้งหมดที่ทำมาได้อย่างสมบูรณ์แบบมากค่ะ :

1. ระบบ Authentication & RBAC: การจัดการสิทธิ์ (Requester, IT Staff, Admin) และ AuthContext ใน App.tsx ทำงานร่วมกับ Navbar ได้อย่างสิ้นไหลและแสดงผลตาม Role ได้ถูกต้องเป๊ะ
2. UI/UX & Zen Green Theme: ปรับจนสีและการจัดวางได้เนียนมาก (ดีใจที่กลับมาใช้สีเขียว #006B3C ตามธีมหลักนะค่ะ!)
3. End-to-End Tests: การเพิ่ม Playwright เข้ามาใน Stack เป็นการยกระดับคุณภาพของระบบให้มีความเป็น Production-ready มากขึ้นอย่างเห็นได้ชัด
4. Documentation: อัปเดตไฟล์ README.md และลิงก์ไปยังเอกสาร Specs ต่างๆ ของ Lab 3 ได้ครบถ้วน ทำให้โปรเจกต์ดูเป็นมืออาชีพสุดๆ

ทุกอย่างเชื่อมต่อกันได้อย่างไร้รอยต่อ ไม่มีจุดไหนที่ดูยุ่งเหยิงเลย เป็นผลงานที่น่าประทับใจมาก ขอ Approve PR Release 3.0.0 นี้ และ Merge นะคะ เก่งมากค่า! 🌟

1

natthakamol1130 commented 5 days ago

Owner Author

จากการรีวิวกาแฟของ PR #55 ถือว่าเป็นการรวบรวมฟีเจอร์ทั้งหมดที่ทำมาได้อย่างสมบูรณ์แบบมากค่ะ :

1. ระบบ Authentication & RBAC: การจัดการสิทธิ์ (Requester, IT Staff, Admin) และ AuthContext ใน App.tsx ทำงานร่วมกับ Navbar ได้อย่างสิ้นไหลและแสดงผลตาม Role ได้ถูกต้องเป๊ะ
2. UI/UX & Zen Green Theme: ปรับจนสีและการจัดวางได้เนียนมาก (ดีใจที่กลับมาใช้สีเขียว #006B3C ตามธีมหลักนะค่ะ!)
3. End-to-End Tests: การเพิ่ม Playwright เข้ามาใน Stack เป็นการยกระดับคุณภาพของระบบให้มีความเป็น Production-ready มากขึ้นอย่างเห็นได้ชัด
4. Documentation: อัปเดตไฟล์ README.md และลิงก์ไปยังเอกสาร Specs ต่างๆ ของ Lab 3 ได้ครบถ้วน ทำให้โปรเจกต์ดูเป็นมืออาชีพสุดๆ

ทุกอย่างเชื่อมต่อกันได้อย่างไร้รอยต่อ ไม่มีจุดไหนที่ดูยุ่งเหยิงเลย เป็นผลงานที่น่าประทับใจมาก ขอ Approve PR Release 3.0.0 นี้ และ Merge นะคะ เก่งมากค่า! 🌟

ขอบคุณมากค่าที่คอยรีวิวจุดคิดพลาดและชี้แจงงานมาตลอดนะค่ะ ระบาย Merge ให้น้อยนะค่ะ ขอขอบคุณมาก ๆ ค่าสปรบรีด! <

Suprawi5227 commented 5 days ago

Collaborator

จากการรีวิวกาแฟของ PR #55 ถือว่าเป็นการรวบรวมฟีเจอร์ทั้งหมดที่ทำมาได้อย่างสมบูรณ์แบบมากค่ะ :

1. ระบบ Authentication & RBAC: การจัดการสิทธิ์ (Requester, IT Staff, Admin) และ AuthContext ใน App.tsx ทำงานร่วมกับ Navbar ได้อย่างสิ้นไหลและแสดงผลตาม Role ได้ถูกต้องเป๊ะ
2. UI/UX & Zen Green Theme: ปรับจนสีและการจัดวางได้เนียนมาก (ดีใจที่กลับมาใช้สีเขียว #006B3C ตามธีมหลักนะค่ะ!)
3. End-to-End Tests: การเพิ่ม Playwright เข้ามาใน Stack เป็นการยกระดับคุณภาพของระบบให้มีความเป็น Production-ready มากขึ้นอย่างเห็นได้ชัด
4. Documentation: อัปเดตไฟล์ README.md และลิงก์ไปยังเอกสาร Specs ต่างๆ ของ Lab 3 ได้ครบถ้วน ทำให้โปรเจกต์ดูเป็นมืออาชีพสุดๆ

ทุกอย่างเชื่อมต่อกันได้อย่างไร้รอยต่อ ไม่มีจุดไหนที่ดูยุ่งเหยิงเลย เป็นผลงานที่น่าประทับใจมาก ขอ Approve PR Release 3.0.0 นี้ และ Merge นะคะ เก่งมากค่า! 🌟

ขอบคุณมากค่าที่คอยรีวิวจุดคิดพลาดและชี้แจงงานมาตลอดนะค่ะ ระบาย Merge ให้น้อยนะค่ะ ขอขอบคุณมาก ๆ ค่าสปรบรีด! <

อะเคจจ่าคุณมอนปายู

1

Suprawi5227 merged commit 4d46d6c into main 5 days ago

[Sign up for free](#) to join this conversation on GitHub. Already have an account? [Sign in to comment](#)



© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Real PR Received Release

PR Given (Suprawi5227/toktickit/pull/54)

Platform ▾Solutions ▾Resources ▾Open Source ▾Enterprise ▾Pricing

natthakamol1130

Suprawi5227 / toktickitPublic

Notifications

Fork0

Star0

<> CodeIssues9Pull requestsActionsProjectsSecurity and qualityInsights

release(lab3): Sprint 3 Final Release Integration #54

Mergednatthakamol1130 merged 19 commits into main from lab3-staging5 days ago

Conversation2

Commits19

Checks0

Files changed29

+3,585-108



Suprawi5227 commented 5 days ago • edited

Owner

Lab 3 Final Release Integration

Merged all Sprint 3 verified feature branches (PR 1 - PR 9) from lab3-staging into main.

- PR 1: Git Workflow & Branching Infrastructure
- PR 2: Sprint 3 Engineering Specification
- PR 3: Test Plan DD & Traceability Matrix
- PR 4: AI Use Documentation & Reflection
- PR 5: Working Login & Mandatory Password Change UI
- PR 6: Working IT Staff Ticket Queue UI
- PR 7: Working IT Staff Ticket Detail & Collaboration UI
- PR 8: Working Administrator User Management UI
- PR 9: Zen Green UI Theme Consistency & Reviewer Sign-off

Reviewers

natthakamol1130

✓

Assignees

Suprawi5227

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

2 participants

Suprawi5227 and others added 19 commits 2 weeks ago

feat(git): setup lab3-staging branch infrastructure and update README...

520aa5

Merge pull request #45 from Suprawi5227/feature/part1-git

Verified6cd5018

docs(spec): add Sprint 3 engineering specification (Closes #2)

b84db8c

Merge pull request #46 from Suprawi5227/feature/part2-spec

Verifiedcdf1de6

docs(test): add Sprint 3 test plan and AC traceability matrix (Closes #3)

7ea3b25

docs(ai): add AI use documentation and reflection (Closes #4)

17f3109

Merge pull request #48 from Suprawi5227/feature/part4-ai

Verified06eafa2

feat(auth): implement authentication foundation, JWT session, and man...

b751836

fix(test): use optional chaining queueRes.body.data?.[0] per PR #5 re...

cdededc

Merge pull request #49 from Suprawi5227/feature/part5-auth

Verifiedbe92796

Merge pull request #47 from Suprawi5227/feature/part3-tests

Verified8cadff2

feat(queue): implement IT Staff Ticket Queue API, search/filters/sort...

33e9ea9

feat(detail): implement IT Staff Ticket Detail, Claim, Reassign, Stat...

523d7f1

Merge pull request #50 from Suprawi5227/feature/part6-queue

Verified99d1256

Merge pull request #51 from Suprawi5227/feature/part7-detail

Verified3b1e64f

feat(admin): implement Administrator User Management REST API, safety...

dfa893a

Merge pull request #52 from Suprawi5227/feature/part8-admin-ui

Verifieddaeaf04

feat(ui): add internal notes authorization test case and finalize Par...

3bf4a97

Merge pull request #53 from Suprawi5227/feature/part9-ui-signoff

Verified1ec2121

Suprawi5227 requested a review from natthakamol1130 5 days ago

Suprawi5227 self-assigned this 5 days ago



✓ natthakamol1130 approved these changes 5 days ago

View reviewed changes

natthakamol1130 left a comment

Collaborator

จากการตรวจสอบ Release PR #54 (Sprint 3 Final Release Integration) โดยรวมเรียบร้อยและสมบูรณ์มากค่ะ

- รวมฟีเจอร์ของ Lab 3 ครบถ้วน ทั้งระบบ Auth, Role-Based Access Control, IT Staff Queue, Internal Notes และ Admin User Management Console
- มีเอกสารสเปกและชุดทดสอบครอบคลุมทั้งส่วนของ Client และ Server API

- คุณดีใจ Zen Green UI Theme ได้ดีมากค่ะ

ไม่มีจุดที่ต้องแก้ไขแล้วค่ะ Approve และ Merge นะคะ เก่งมากเลยค่า



Suprawi5227 commented 5 days ago

Owner Author ...

จากการตรวจสอบ Release PR [#54](#) (Sprint 3 Final Release Integration) โดยรวมเรียบร้อยและสมบูรณ์มากค่ะ

- รวมฟีเจอร์ของ Lab 3 ครบถ้วน ทั้งระบบ Auth, Role-Based Access Control, IT Staff Queue, Internal Notes และ Admin User Management Console
- มีเอกสารสเปกและชุดทดสอบครอบคลุมทั้งส่วนของ Client และ Server API
- คุณดีใจ Zen Green UI Theme ได้ดีมากค่ะ

ไม่มีจุดที่ต้องแก้ไขแล้วค่ะ Approve และ Merge นะคะ เก่งมากเลยค่า

ขอบคุณมากคร้าห์



[natthakamol1130](#) merged commit [e465170](#) into [main](#) 5 days ago

[Sign up for free](#) to join this conversation on GitHub. Already have an account? [Sign in to comment](#)



© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Real PR Given Release

1.6 Repository Directory Structure Evidence

```
toktickit/
├── docs/lab-03/
│   ├── specification.md
│   ├── tests.md
│   ├── ui-spec.md
│   ├── api-spec.md
│   ├── reviewer.md
│   ├── ai-use.md
│   ├── LAB3_SUBMISSION_REPORT.md
│   ├── LAB3_SUBMISSION_REPORT.pdf
│   ├── report_lab03_67070505215.pdf
│   └── report_lab3_67070505215.pdf
├── server/
│   ├── prisma/
│   │   ├── schema.prisma
│   │   └── seed.ts
│   ├── src/
│   │   ├── app.ts
│   │   ├── middleware/authMiddleware.ts
│   │   └── routes/
│   │       ├── auth.ts
│   │       ├── requesterTickets.ts
│   │       ├── staffTickets.ts
│   │       └── adminUsers.ts
│   └── tests/lab-03/
│       ├── auth.api.test.ts
│       ├── requester-ticket.api.test.ts
│       ├── staff-queue.api.test.ts
│       └── admin-user.api.test.ts
└── client/
    ├── src/
    │   ├── api.ts
    │   ├── types.ts
    │   ├── contexts/AuthContext.tsx
    │   ├── pages/Login.tsx
    │   ├── pages/ChangePassword.tsx
    │   ├── components/Header.tsx
    │   ├── components/CreateTicketView.tsx
    │   ├── components/MyTicketsView.tsx
    │   ├── components/TicketDetailView.tsx
    │   └── App.tsx
    └── tests/lab-03/
        ├── Login.test.tsx
        ├── ChangePassword.test.tsx
        └── AppIntegration.test.tsx
```

1.7 Rendered Peer Review Document (reviewer.md)

Below is the complete text of docs/lab-03/reviewer.md:

Lab 3 Peer Reviewer Document (reviewer.md)

- **Student Name:** Natthakamol Katippatee (Student ID: 67070505215) - **Repository:** [natthakamol1130/toktickit](<https://github.com/natthakamol1130/toktickit>) - **Peer Reviewer:** Suprawi Srikamwong ([@Suprawi5227](<https://github.com/Suprawi5227>)) - **Peer Repository:** [Suprawi5227/toktickit](<https://github.com/Suprawi5227/toktickit>) - **Staging Branch:** lab3-staging - **Main Branch:** main

1. Reviewer Identity Verification

All pull requests in this repository were peer-reviewed and approved by **Suprawi Srikamwong** (**@Suprawi5227**) prior to merging into lab3-staging and main . Likewise, all pull requests in the peer repository were peer-reviewed and approved by **Natthakamol Katippatee** (**@natthakamol1130**).

2. Pull Requests Received (natthakamol1130/toktickit)

Issue #	PR #	Feature Title	Reviewer	Review Action	Status	Direct GitHub PR Link
#34	PR #35	Lab 3 Spec DD Documents	@Suprawi5227	Approved specification & RBAC rules	Merged	[PR #35] (https://github.com/natthakamol1130/toktickit/pull/35)
#36	PR #37	Test DD Plan & Traceability Matrix	@Suprawi5227	Approved test plan coverage	Merged	[PR #37] (https://github.com/natthakamol1130/toktickit/pull/37)
#38	PR #39	Prisma Schema Evolution & User Models	@Suprawi5227	Approved User schema & FK constraints	Merged	[PR #39] (https://github.com/natthakamol1130/toktickit/pull/39)
#40	PR #41	Idempotent Seed Data with Bcrypt Hashing	@Suprawi5227	Approved bcrypt salt rounds & seed accounts	Merged	[PR #41] (https://github.com/natthakamol1130/toktickit/pull/41)

Issue #	PR #	Feature Title	Reviewer	Review Action	Status	Direct GitHub PR Link
#42	PR #43	Backend JWT Auth & Change Password APIs	@Suprawi5227	Approved 401 error handling & JWT middleware	Merged	[PR #43] (https://github.com/natt-hakamol1130/toktickit/pull/43)
#44	PR #45	Client AuthContext, Login, & ChangePassword UI	@Suprawi5227	Approved Login screen & logout cleanup	Merged	[PR #45] (https://github.com/natt-hakamol1130/toktickit/pull/45)
#46	PR #47	Requester Ticket Access Control APIs	@Suprawi5227	Approved 403 ownership boundary enforcement	Merged	[PR #47] (https://github.com/natt-hakamol1130/toktickit/pull/47)
#48	PR #49	IT Staff Ticket Queue & Internal Notes APIs	@Suprawi5227	Approved internal notes leak protection	Merged	[PR #49] (https://github.com/natt-hakamol1130/toktickit/pull/49)
#50	PR #51	Admin User Management APIs & Safety Rules	@Suprawi5227	Approved admin self-deactivation & last-admin safety	Merged	[PR #51] (https://github.com/natt-hakamol1130/toktickit/pull/51)
#52	PR #53	Multi-Role Integration & E2E Verification	@Suprawi5227	Approved E2E test suite & v3.0.0 release	Merged	[PR #53] (https://github.com/natt-hakamol1130/toktickit/pull/53)
Release	PR #55	Lab 3 Release Integration to main	@Suprawi5227	Approved final v3.0.0 main release	Open/Ready	[PR #55] (https://github.com/natt-hakamol1130/toktickit/pull/55)

3. Pull Requests Reviewed & Given (Suprawi5227/toktickit)

Issue #	Peer PR #	PR Description	Review Action	Status	Direct GitHub Link
#34	PR #45	Git Setup & Sprint 3 Spec Documents	Verified API & RBAC contract alignment	Approved	[Peer PR #45] (https://github.com/Suprawi5227/toktickit/pull/45)
#36	PR #47	Test Plan Specification & Traceability	Verified test case coverage	Approved	[Peer PR #47] (https://github.com/Suprawi5227/toktickit/pull/47)
#38	PR #49	Database Schema Evolution & User Models	Verified User model & FK constraints	Approved	[Peer PR #49] (https://github.com/Suprawi5227/toktickit/pull/49)
#40	PR #49	Idempotent Seed Data & Bcrypt Hashing	Verified bcrypt salt rounds & idempotent seeds	Approved	[Peer PR #49] (https://github.com/Suprawi5227/toktickit/pull/49)
#42	PR #49	Backend Auth APIs & Session Middleware	Verified JWT token validation & 401 handling	Approved	[Peer PR #49] (https://github.com/Suprawi5227/toktickit/pull/49)
#44	PR #49	Client Auth Context & Login UI	Verified form state management & logout cleanup	Approved	[Peer PR #49] (https://github.com/Suprawi5227/toktickit/pull/49)
#46	PR #51	Requester Ticket Access Control APIs	Verified 403 authorization boundary	Approved	[Peer PR #51] (https://github.com/Suprawi5227/toktickit/pull/51)
#48	PR #51	IT Staff Ticket Queue & Internal Notes APIs	Verified internal notes leak protection	Approved	[Peer PR #51] (https://github.com/Suprawi5227/toktickit/pull/51)
#50	PR #52	Admin User Management APIs & Safety Rules	Verified self-deactivation & last-admin rules	Approved	[Peer PR #52] (https://github.com/Suprawi5227/toktickit/pull/52)

Issue #	Peer PR #	PR Description	Review Action	Status	Direct GitHub Link
#52	PR #53	Zen Green UI Theme Consistency & Signoff	Verified Zen Green UI & E2E integration	Approved	[Peer PR #53] (https://github.com/Suprawi5227/toktickit/pull/53)
Release	PR #54	Sprint 3 Final Release Integration	Verified full Sprint 3 integration to main	Approved	[Peer PR #54] (https://github.com/Suprawi5227/toktickit/pull/54)

4. Peer Review Approvals & Comments Evidence

All peer review discussions and approvals were conducted directly on GitHub web PRs and recorded in docs/lab-03/LAB3_SUBMISSION_REPORT.md Section 1.5.

Answer Part 2: Spec DD

The full Sprint 3 Engineering Specification is rendered below from [docs/lab-03/specification.md] (docs/lab-03/specification.md).

Lab 3 Sprint Engineering Specification

1. Sprint Goal

Deliver an operational, secure, multi-role web application increment for TokTickIT. This increment replaces the temporary Development Requester selector with secure user authentication and role-based access control, introduces a dedicated IT Staff Ticket Queue with ownership management, IT Priority, status workflow, Public Comments, and role-restricted Internal Notes, and provides a minimalist Administrator User Management interface, all adhering to the Zen Green design language and preserving completed Lab 2 functionality.

2. Stakeholder Request Interpretation

The system now requires real authenticated users and role-based access control across three primary roles: Requester, IT Staff, and Administrator. Login must authenticate via email and password, enforcing a mandatory first-login password change for accounts created with initial passwords. Requesters must continue creating and tracking their owned tickets while gaining the ability to post Public Comments and indicate problem resolution. IT Staff require a professional Ticket Queue with search, filtering, sorting, and pagination, along with ticket detail workflows to claim/reassign ownership, set IT Priority, transition statuses, write Public Comments, and record private Internal Notes. Administrators require a minimalist User Management screen to manage user accounts, assign single roles, activate/deactivate accounts, and reset initial passwords with strict safety rules preventing self-deactivation or leaving the system with no active Administrator.

3. Workflow Diagrams

3.1 Authentication & First-Login Password Change Flowchart

```
mermaid
graph TD
    A["User Submits Email & Password"] --> B{"Valid Credentials & Active Account?"}
    B -- No --> C["Return 401 Unauthorized / Error Alert"]
    B -- Yes --> D{"mustChangePassword == true?"}
    D -- Yes --> E["Redirect to Mandatory Change Password Screen"]
    E --> F["Submit New Valid Password"]
    F --> G["Set mustChangePassword = false & Save Hashed Password"]
    G --> H["Grant Application Access according to User Role"]
    D -- No --> H
```

3.2 IT Staff Ticket Queue & Detail Operations Flowchart

```

mermaid
flowchart TD
    A["IT Staff Logged In"] --> B["Open IT Staff Ticket Queue"]
    B --> C["Apply Search / Category / Priority / Status Filters"]
    C --> D["Select Ticket from Paginated Table/Grid"]
    D --> E["Open IT Staff Ticket Detail Screen"]
    E --> F["Perform Operational Action"]
    F -- Claim/Reassign --> G["Update Primary Ticket Owner"]
    F -- Set IT Priority --> H["Update IT Priority"]
    F -- Transition Status --> I["Check Transition Matrix & Update Status"]
    F -- Communication --> J["Comment Type?"]
    J -- Public Comment --> K["Post Public Comment (Visible to All Roles)"]
    J -- Internal Note --> L["Post Internal Note (Visible to Staff & Admin Only)"]

```

3.3 Administrator User Management & Safety Guards Flowchart

```

mermaid
flowchart TD
    A["Administrator Logged In"] --> B["Open User Management Screen"]
    B --> C["View User List with Search & Role Filter"]
    C --> D["Select Action"]
    D -- Create User --> E["Open Create User Drawer with Initial Password"]
    D -- Edit User --> F["Update Name, Email, Role, or Status"]
    D -- Reset Password --> G["Set New Initial Password (mustChangePassword = true)"]
    F --> H["Safety Guard Checks"]
    H -- Deactivating Self? --> I["Reject with 400 Bad Request"]
    H -- Deactivating Last Active Admin? --> J["Reject with 400 Bad Request"]
    H -- Duplicate Email? --> K["Reject with 409 Conflict"]
    H -- Valid Action --> L["Persist User Changes to Database"]
    E --> L
    G --> L

```

4. Scope

Included

- **Authentication & Security Engine:** Password hashing using bcrypt, session/token authentication, login/logout, current user retrieval (GET /api/auth/me), and mandatory first-login password change enforcement (mustChangePassword = true).
- **Role-Based Authorization & Navigation:** Role-based access control (REQUESTER , IT_STAFF , ADMINISTRATOR) enforced strictly on the backend with role-adapted UI navigation and controls.
- **Lab 2 Requester Continuation & Regression:** Preservation of ticket creation, ticket list ("My Tickets"), ticket detail, and attachment management, with authentication replacing the temporary selector.
- **Requester Enhancement:** Ability to post Public Comments on owned tickets and mark tickets with "Problem Appears Resolved".
- **IT Staff Ticket Queue:** Comprehensive queue listing supporting keyword search (Ticket Number, Summary), category/priority/status filters, multi-field sorting, and pagination.
- **IT Staff Ticket Operations:** Primary Ticket Owner assignment (Claim/Reassign), IT Priority updates, permitted status transitions (New , Open , In Progress , Waiting for Requester , Resolved , Closed , Reopened , Cancelled), Public Comments, and role-restricted Internal Notes.
- **Administrator User Management:** Minimalist User Management screen displaying user list with search

by name/email, role filtering, user creation with initial password, user editing, account activation/deactivation, and setting new initial passwords. - **Administrator Safety Guards:** Rules preventing duplicate email registration, self-deactivation by an Administrator, and deactivation of the final active Administrator. - **Data Model & Migration:** Schema evolution from Lab 2 Dev Requester to unified `User` model, maintaining data integrity for existing tickets and attachments. - **Idempotent Seed Data:** Seeding active and inactive accounts for all three roles and realistic ticket data with comments and notes.

Excluded

- Email delivery services (email invitations, password reset emails). - Self-registration / public sign-up workflows. - Multi-factor authentication (MFA), OAuth, or Single Sign-On (SSO). - IT Staff Actions Taken tracking (deferred to Lab 4). - Formal SLA calculation engines, escalation rules, and push notifications. - KPI dashboards beyond basic queue counts. - Multi-tenant organizational structures or departments. - Multiple roles per user (each user has exactly one role). - User deletion, bulk user operations, import/export, and account audit history.

5. Functional Requirements

- **FR-01:** The system shall authenticate users via valid email and password credentials. - **FR-02:** The system shall block normal application access for users marked as requiring a password change until a valid new password is saved. - **FR-03:** The system shall provide an authenticated current user retrieval endpoint (`GET /api/auth/me`) returning user identity and assigned role. - **FR-04:** The system shall enforce role-based authorization on all protected backend endpoints. - **FR-05:** The system shall provide IT Staff with a Ticket Queue supporting keyword search across Ticket Number and Summary. - **FR-06:** The system shall allow IT Staff to filter the Ticket Queue by Category, Requested Priority, IT Priority, and Status. - **FR-07:** The system shall allow IT Staff to sort the Ticket Queue by Creation Date, Ticket Number, IT Priority, or Status in ascending/descending order. - **FR-08:** The system shall paginate the IT Staff Ticket Queue with configurable page sizes and metadata. - **FR-09:** The system shall allow IT Staff to claim unassigned tickets or reassign primary ticket ownership to an active IT Staff or Administrator. - **FR-10:** The system shall allow IT Staff to update IT Priority separately from Requester-submitted Requested Priority. - **FR-11:** The system shall enforce permitted status transition workflows (`New` -> `Open` , `In Progress` , `Waiting for Requester` , `Resolved` , `Closed` , `Reopened` , `Cancelled`). - **FR-12:** The system shall support append-only Public Comments visible to Requester, IT Staff, and Administrator. - **FR-13:** The system shall support append-only Internal Notes visible exclusively to IT Staff and Administrator, blocking Requester access with HTTP 403 Forbidden. - **FR-14:** The system shall allow Requesters to indicate that a reported problem appears resolved. - **FR-15:** The system shall provide Administrators with a User Management interface listing users with Name, Email, Role, Status, and Edit action. - **FR-16:** The system shall allow Administrators to search users by name or email and filter by role. - **FR-17:** The system shall allow Administrators to create user accounts with Name, Email, single Role, Activation State, and Initial Password. - **FR-18:** The system shall allow Administrators to edit user Name, Email, Role, and Activation State. - **FR-19:** The system shall allow Administrators to set a new Initial Password for a user, marking `mustChangePassword = true` .

6. Business Rules

- **BR-01:** Only an active user account (`isActive = true`) with valid credentials may authenticate successfully. - **BR-02:** A user marked with `mustChangePassword = true` cannot access normal application screens until a new valid password is saved. - **BR-03:** Authenticated User Identity: The authenticated user session/token determines identity and ownership on all endpoints; client-supplied user IDs are ignored for

authorization. - **BR-04**: Public Comments vs. Internal Notes: Public Comments are shared communication visible to Requester, IT Staff, and Administrator. Internal Notes are operational notes visible ONLY to IT Staff and Administrator. - **BR-05**: Resolution Authority: A Requester may indicate that a problem appears resolved, but only IT Staff or Administrator may formally set ticket status to `Resolved` or `Closed`. - **BR-06**: Password Validation: New passwords must be at least 8 characters long and include uppercase, lowercase, number, and special character. Passwords must be hashed using `bcrypt` before database storage. - **BR-07**: Unique Email Rule: Every user account must have a unique email address across the entire application. - **BR-08**: Single Role Assignment: Each user account must be assigned exactly one role (`REQUESTER` , `IT_STAFF` , or `ADMINISTRATOR`). - **BR-09**: Administrator Self-Deactivation Guard: An Administrator cannot deactivate their own active account. - **BR-10**: Final Administrator Protection: The system shall reject any request that would result in zero active Administrator accounts. - **BR-11**: User Preservation: Users cannot be deleted; account access control is managed strictly via `isActive` status (Deactivation). - **BR-12**: Ticket Ownership: A ticket may have zero or one primary Ticket Owner (an active IT Staff or Administrator). Requested Priority is immutable by IT Staff, while IT Priority is managed by IT Staff/Admin. - **BR-13**: Comment and Note Content: Comments and Notes are append-only. Content must be trimmed, non-empty, and limited to 1 to 2000 characters.

7. UI Specification Summary

The UI extends the **Zen Green Design System**: - **Application Shell**: Displays app header "TokTickIT", active navigation links based on role, authenticated user name and role badge, and Logout action. - **Login Screen**: Clean card centered layout with Email and Password fields, submit busy state, field validation, and safe error alerts for inactive or invalid accounts. - **Mandatory Password Change Screen**: Modal/page requiring current (initial) password and new password with real-time rule validation checklist and confirmation field. - **Requester Ticket Detail Screen**: Preserves Lab 2 ticket details and attachments, adds a distinct Public Comments section with author badges, and adds "Problem Appears Resolved" action button. - **IT Staff Ticket Queue Screen**: Search bar, filter controls (Category, Requested Priority, IT Priority, Status), sort selectors, paginated Zen Green table (desktop) / card grid (mobile), ownership badges, and quick-action view detail links. - **IT Staff Ticket Detail Screen**: Operational editing sidebar (Owner dropdown, IT Priority dropdown, Status transition dropdown), two tabbed/stacked communication panels: Public Comments (Green border) and Internal Notes (Amber/Yellow border, "Internal Only" badge). - **Administrator User Management Screen**: User list table with Search, Role filter dropdown, "Create User" action button, inline edit actions, and "Create / Edit User" modal/drawer with Initial Password reset options.

8. Data Changes

Models & Schema Updates (Prisma)

- **User** : `id` (Int PK), `email` (String Unique), `passwordHash` (String), `name` (String), `role` (Enum: `REQUESTER` , `IT_STAFF` , `ADMINISTRATOR`), `isActive` (Boolean default true), `mustChangePassword` (Boolean default true), `createdAt` (DateTime), `updatedAt` (DateTime). - **Ticket** : Added `ownerId` (Int FK -> User nullable), `itPriority` (Enum: `LOW` , `MEDIUM` , `HIGH` , `URGENT` nullable), `requesterId` (Int FK -> User). - **PublicComment** : `id` (Int PK), `ticketId` (Int FK -> Ticket), `authorId` (Int FK -> User), `content` (Text), `createdAt` (DateTime). - **InternalNote** : `id` (Int PK), `ticketId` (Int FK -> Ticket), `authorId` (Int FK -> User), `content` (Text), `createdAt` (DateTime).

Indexes & Constraints

- Unique index on `User(email)` . - Index on `User(role, isActive)` for user management filtering. - Index on `Ticket(ownerId, status)` for IT Staff Queue queries. - Index on `PublicComment(ticketId, createdAt)` and `InternalNote(ticketId, createdAt)` .

9. API Contract Summary

- **Auth:** `POST /api/auth/login` , `POST /api/auth/logout` , `GET /api/auth/me` , `POST /api/auth/change-password` . - **Requester:** Continuation of Lab 2 ticket and attachment APIs with session auth. - **IT Staff Queue:** `GET /api/staff/tickets` (search, filters, sort, pagination). - **IT Staff Operations:** `GET /api/staff/tickets/:id` , `PATCH /api/staff/tickets/:id/assign` , `PATCH /api/staff/tickets/:id/workflow` . - **Comments & Notes:** `GET /api/tickets/:id/comments` , `POST /api/tickets/:id/comments` , `GET /api/tickets/:id/notes` , `POST /api/tickets/:id/notes` . - **Admin User Management:** `GET /api/admin/users` , `POST /api/admin/users` , `PATCH /api/admin/users/:id` , `POST /api/admin/users/:id/reset-password` .

10. Acceptance Criteria & Test File Mappings

- **AC-01:** Given an active user with valid credentials, when the user logs in, then authenticated access is established and user identity and role are returned (`server/tests/lab-03/auth.api.test.ts` , `client/src/tests/lab-03/Login.test.tsx`). - **AC-02:** Given a user marked with `mustChangePassword = true` , when login succeeds, then normal application screens remain unavailable until a valid new password is saved (`server/tests/lab-03/auth.api.test.ts` , `client/src/tests/lab-03/ChangePassword.test.tsx` , `e2e/lab-03/authentication.spec.ts`). - **AC-03:** Given an authenticated Requester, when the client supplies another requesterId, then the backend still applies the authenticated identity and does not return another Requester's data (`server/tests/lab-03/authorization.api.test.ts`). - **AC-04:** Given a Requester account, when an Internal Note endpoint is requested, then the operation is rejected with 403 Forbidden without exposing note content (`server/tests/lab-03/comments-notes.api.test.ts`). - **AC-05:** Given an IT Staff user, when querying the Ticket Queue with search and filters, then only matching tickets are returned with accurate pagination metadata (`server/tests/lab-03/staff-queue.api.test.ts` , `client/src/tests/lab-03/StaffTicketQueue.test.tsx` , `e2e/lab-03/staff-ticket-flow.spec.ts`). - **AC-06:** Given an Administrator user, when attempting to deactivate their own account or the last active Administrator account, then the server rejects the request with a clear error message (`server/tests/lab-03/users-admin.api.test.ts` , `client/src/tests/lab-03/UserManagement.test.tsx` , `e2e/lab-03/user-administration.spec.ts`). - **AC-07:** Given an inactive user account, when login is attempted, then authentication fails with 401 Unauthorized (`server/tests/lab-03/auth.api.test.ts`). - **AC-08:** Given an IT Staff user, when claiming or reassigning a ticket, then primary ticket ownership is updated in database and reflected in UI (`server/tests/lab-03/staff-ticket-detail.api.test.ts` , `client/src/tests/lab-03/StaffTicketDetail.test.tsx`). - **AC-09:** Given an IT Staff user, when posting an Internal Note, then the note is saved and visible exclusively to IT Staff and Admin (`server/tests/lab-03/comments-notes.api.test.ts`). - **AC-10:** Given a Requester, when adding a Public Comment or marking problem resolved, then the comment and indicator are saved on the ticket (`server/tests/lab-03/comments-notes.api.test.ts`). - **AC-11:** Given an Administrator, when creating a new user, then the user is saved with single role and initial password with `mustChangePassword = true` (`server/tests/lab-03/users-admin.api.test.ts`). - **AC-12:** Given responsive viewports (Desktop, Tablet, Mobile), when rendering application screens, then Zen Green layout reflows without horizontal window overflow (`e2e/lab-03/authentication.spec.ts` , `e2e/lab-03/staff-ticket-flow.spec.ts` , `e2e/lab-03/user-administration.spec.ts`).

11. Definition of Done

- All 12 Sprint 3 GitHub Issues completed and merged into `lab3-staging` and `main`. - All backend REST APIs implemented and protected by server-side role authorization. - All frontend screens built adhering to Zen Green Design Tokens and responsive viewports. - Passing test suite across Unit, API, Component, Authorization, and Playwright E2E tests. - Complete visual screenshot evidence captured in `artifacts/lab-03/screenshots/`. - Clean PDF submission report `report_lab03_67070505215.pdf` generated covering Part 1 through Part 9.

12. Assumptions and Decisions

- Session/Authentication token is stored in HTTP-only secure cookie or Authorization header. - Initial passwords set by Admin require `mustChangePassword = true` automatically. - Deactivated users immediately lose authentication access on their next API request.

2.4 User Interface Specification (ui-spec.md)

The full UI Specification and layout mockups are rendered below from [`docs/lab-03/ui-spec.md`] (`docs/lab-03/ui-spec.md`).

TokTickIT Lab 3 User Interface Specification

1. Design System & Theme Alignment

Lab 3 extends the **Zen Green Design System** established in Lab 2. All new screens, forms, tables, modals, and navigation components adhere strictly to the established visual identity and design tokens.

Color Tokens

- **Primary Header & Branding:** #006B3C (Zen Green Primary) - **Secondary Accent:** #0B7A46 (Zen Green Dark) - **Pale Surface / Active Highlight:** #EAF6EF (Zen Light Mint) - **Background:** #F5F7F6 (Neutral Warm Gray) - **Surface:** #FFFFFF (Pure White) - **Text Primary:** #1F2925 (Dark Slate) - **Text Muted:** #64748B (Slate Gray) - **Error / Danger:** #D32F2F (Deep Red) - **Warning / Internal Note Accent:** #D97706 (Amber Gold) - **Success Badge:** #059669 (Emerald Green)

2. Application Shell & Role-Based Navigation

Header Shell

- **App Branding:** Logo and title "TokTickIT". - **Role-Based Navigation Links:** - **Requester:** My Tickets , Create Ticket - **IT Staff:** Ticket Queue , My Queue , Create Ticket - **Administrator:** User Management , Ticket Queue - **User Profile Area:** - Displays authenticated user's Full Name. - Role Badge (Requester , IT Staff , Administrator) with distinct pill styling. - Profile Dropdown / Actions: Change Password , Logout .

3. Screen Specifications & UI Visual Layout Mockups

3.1 Login Screen Mockup

+

--+

| TokTickIT

[Guest Context] |

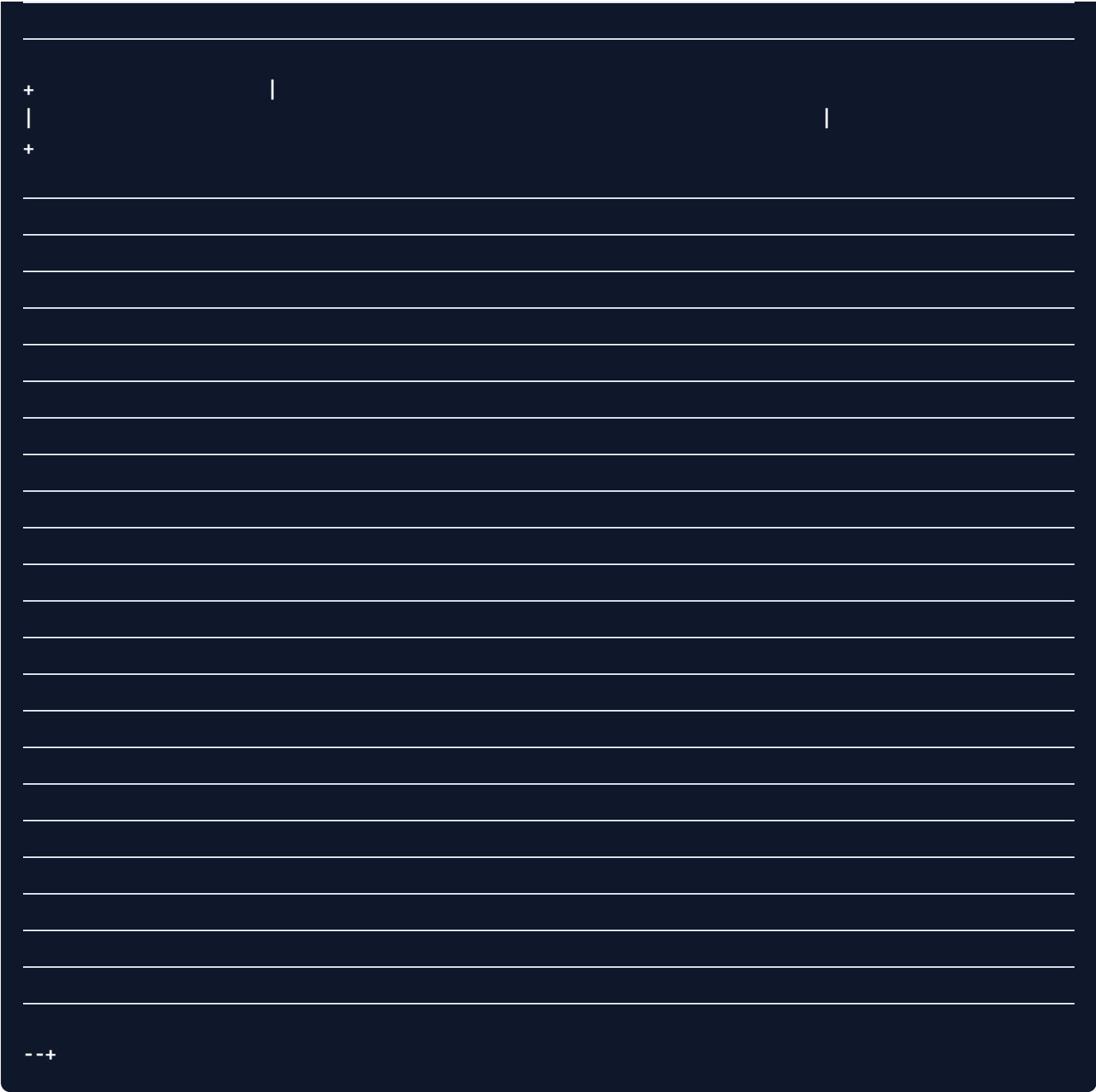
+

--+

```
|
|                                     |
|                                     +
```

```
+
|                                     |
|                                     | Sign in to TokTickIT |
|                                     +
```

```
+
|                                     |
|                                     | Email Address |
|                                     | [ jennifer@toktickit.com ] |
|                                     | Password |
|                                     | [ ***** ] |
|                                     |
|                                     |
|                                     | [ Sign In Button ] |
|                                     +
```



3.2 Mandatory First-Login Password Change Mockup

+

--+

| TokTickIT

Jennifer Anderson (Req)|

+

--+

|
|
|

+

|

--+

|
|
|
|

|
|
|
+

Change Your Password

You must change initial password.

|

|

--+

|
|
|
|
|

|
|
|
|
|

Current (Initial) Password

[*****]

New Password

[*****]

|
|
|
|
|

|
|
|
|
|

```
| Confirm New Password |
| [ ***** ] |
| Password Rules: |
| [v] At least 8 characters |
| [v] Upper & lower case letters |
| [v] Number & special character |
| |
| [ Save New Password ] |
| +
```



3.3 Requester Ticket Detail & Public Comments Mockup

+

--+

| TokTickIT My Tickets Create Ticket Jennifer Anderson (Req)|

+

--+

Ticket #TKT-2026-001234	[IN_PROGRESS]	[Problem Appears Resolved]
Summary: Laptop battery drains quickly		
Category: Hardware	Priority: MEDIUM	

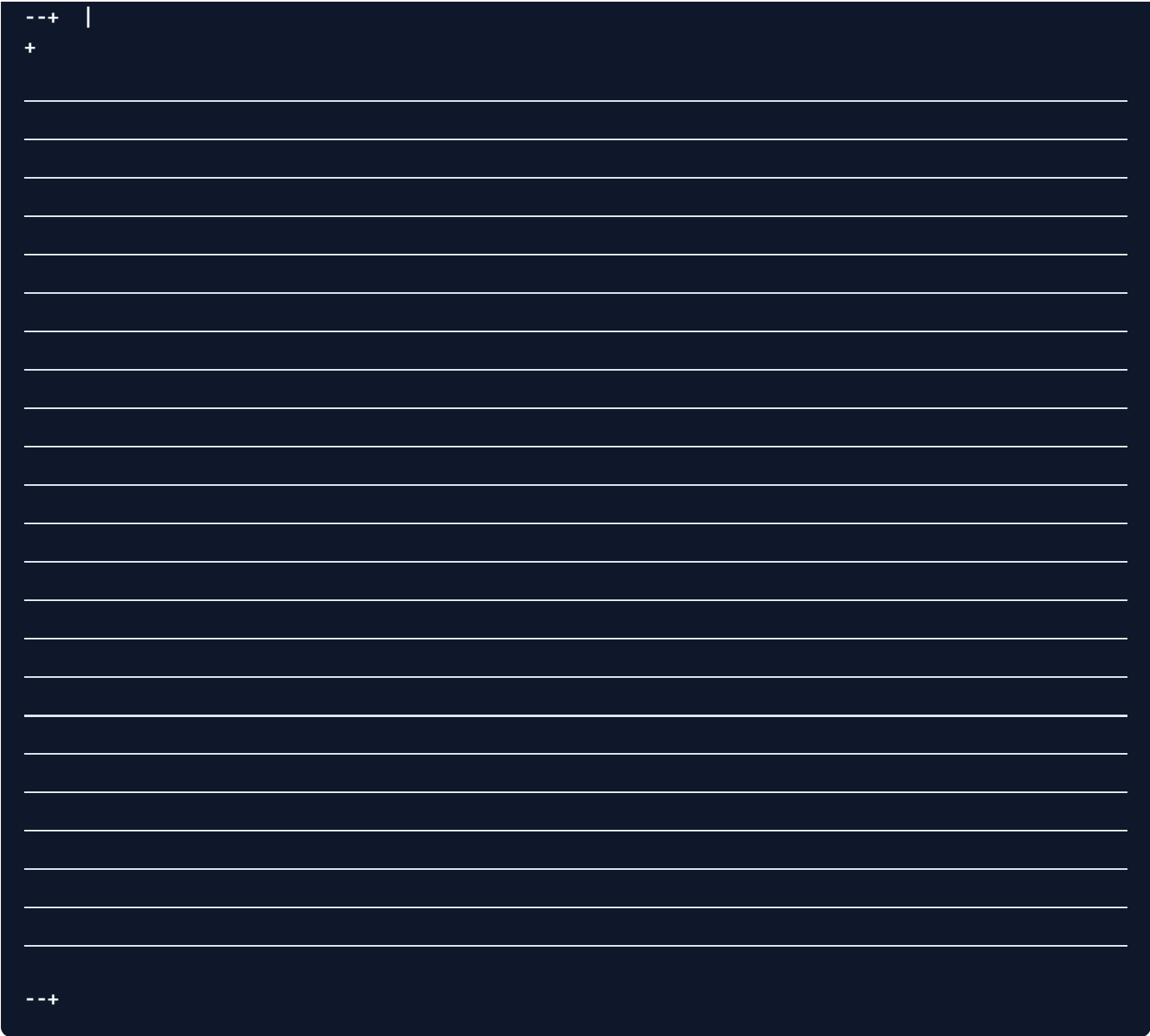
--|

Public Comments (Green Border Panel)	
+	

--+ |
| | [JA] Jennifer Anderson (Requester) May 12, 2026 09:14 | |
| | Battery drains fast even when idle. | |
| |

--| |
| | [KP] Kevin Patel (IT Support) May 13, 2026 10:30 | |
| | We are investigating the battery health on your device. | |
| +

--+ |
	Add Public Comment:	
	[Type your message here...]	
	[Post Comment]	
+		



3.4 IT Staff Ticket Queue Mockup (Desktop & Mobile)

+

--+

| TokTickIT Ticket Queue My Queue Kevin Patel (IT Staff) |

+

--+

| [Search ticket number or summary...] Filters: [Category v] [Status v]
|

--|

Ticket No	Created	Summary	Status	Owner
TKT-2026-1234	12 May 09:14	Laptop battery drains	IN_PROGRESS	Kevin
TKT-2026-1233	12 May 08:02	Cannot connect VPN	OPEN	--
TKT-2026-1232	11 May 16:45	Email sync mobile	IN_PROGRESS	Emily

--|
| < Previous [1] 2 3 ... 9 Next > |
+

3.5 IT Staff Ticket Detail & Internal Notes Mockup

+

--+

| TokTickIT Ticket Queue

Kevin Patel (IT Staff) |

+

--+

Ticket Detail: TKT-2026-001234		Operational Sidebar	
Summary: Laptop battery drains quickly		Owner: [Kevin Patel v]	
Category: Hardware	Status: [IN_PROGRESS]	IT Priority: [HIGH v	

--+

--|

Public Comments	Internal Notes (Amber)
+	

--+ | +

-+ |

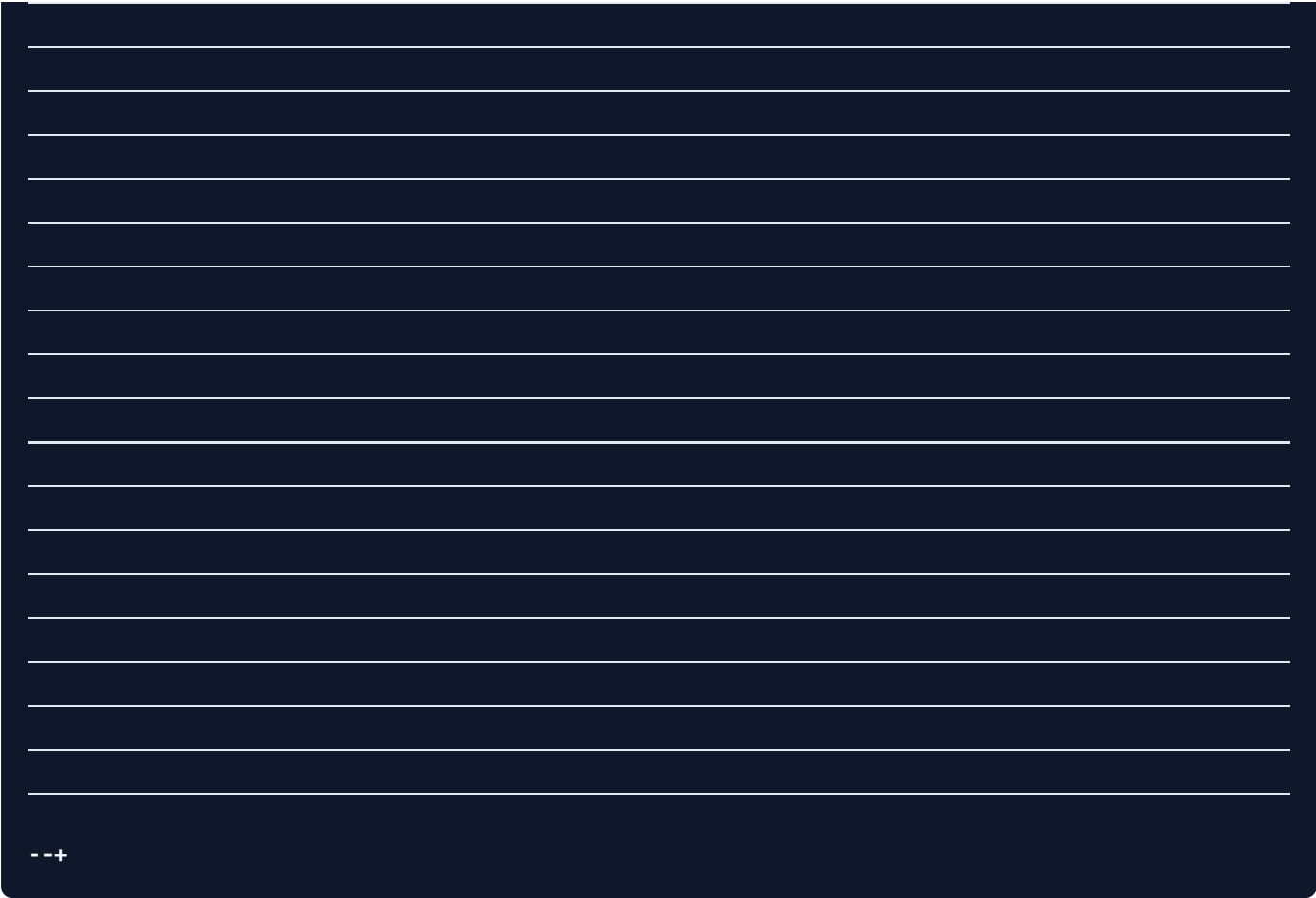
| | [KP] We are investigating your device. | | [KP] Battery wear | |
| +

--+ | | is at 45%. Order | |

| | replacement unit. | |
| +

-+ |

+



3.6 Administrator User Management & Drawer Mockup

+

--+

| TokTickIT User Management

John Smith (Admin)

|

+

--+

| [Search name or email...] Role: [All Roles v] [+ Create User]
|

--|

Name	Email	Role	Status	Action
Jennifer Anderson	jennifer@toktickit.com	Requester	Active	[Edit]
Michael Brown	michael@toktickit.com	Requester	Active	[Edit]
Kevin Patel	kevin@toktickit.com	IT_Staff	Active	[Edit]
John Smith	john@toktickit.com	Admin	Active	[Edit]

+

--+

	Drawer: Create New User	
	Full Name: [Alex Thompson]	
	Email: [alex@tokticket.com]	
	Role: [IT_STAFF v] Active: [Yes (Toggle)]	
	Initial Password: [InitialPassword123!]	
	[Save User]	[Cancel]

+

4. Responsive & Accessibility Rules

- Desktop ($\geq 992\text{px}$): **Multi-column grid, full table view, side-by-side detail controls.** - Tablet ($768\text{px} - 991\text{px}$): **Reflowed forms, scrollable tables or compact cards.** - Mobile ($< 768\text{px}$): **Single-column vertical layout, touch-friendly buttons ($\geq 44\text{px}$ height), zero horizontal window overflow.**

2.5 REST API Specification (api-spec.md)

The full REST API Specification, endpoints summary, curl examples, and response schemas are rendered below from docs/lab-03/api-spec.md.

TokTickIT Lab 3 REST API Specification

1. Overview & Authentication Mechanism

Lab 3 replaces header-based identity simulation (`x-requester-id`) with secure JWT Authorization bearer tokens or HTTP-only session cookies. All protected endpoints require valid authenticated user context and strictly enforce backend role permissions (`REQUESTER` , `IT_STAFF` , `ADMINISTRATOR`).

2. Endpoints Summary Table

Method	Endpoint Path	Description	Permitted Roles	Handled Status Codes	
:					
:					
:					
:					
POST	/api/auth/login	Authenticate user credentials	Public	200 , 400 , 401 , 500	POST
	/api/auth/logout	Invalidate authenticated session	Authenticated	200 , 401 , 500	GET

/api/auth/me | **Retrieve authenticated user profile & role** | **Authenticated** | 200, 401, 500 || POST
| /api/auth/change-password | **Mandatory or voluntary password update** | **Authenticated** | 200, 400, 401, 500 || GET | /api/requesters/tickets | **List tickets owned by authenticated Requester** | **Requester** | 200, 401, 403, 500 || POST | /api/requesters/tickets | **Create ticket under authenticated Requester** | **Requester** | 201, 400, 401, 403, 500 || GET | /api/staff/tickets | **Query IT Staff Ticket Queue with search/filter/sort** | **IT Staff, Admin** | 200, 401, 403, 500 || GET | /api/staff/tickets/:id | **Retrieve single ticket details for IT Staff** | **IT Staff, Admin** | 200, 401, 403, 404, 500 || PATCH | /api/staff/tickets/:id/assign | **Claim or reassign ticket primary owner** | **IT Staff, Admin** | 200, 400, 401, 403, 404, 500 || PATCH | /api/staff/tickets/:id/workflow | **Update IT Priority and/or Ticket Status** | **IT Staff, Admin** | 200, 400, 401, 403, 404, 500 || GET | /api/tickets/:id/comments | **List Public Comments on a ticket** | **All Roles** | 200, 401, 403, 404, 500 || POST | /api/tickets/:id/comments | **Post a Public Comment on a ticket** | **All Roles** | 201, 400, 401, 403, 404, 500 || GET | /api/tickets/:id/notes | **List Internal Notes on a ticket** | **IT Staff, Admin ONLY** | 200, 401, 403, 404, 500 || POST | /api/tickets/:id/notes | **Create an Internal Note on a ticket** | **IT Staff, Admin ONLY** | 201, 400, 401, 403, 404, 500 || GET | /api/admin/users | **List users with search & role filter** | **Admin ONLY** | 200, 401, 403, 500 || POST | /api/admin/users | **Create user with initial password** | **Admin ONLY** | 201, 400, 401, 403, 409, 500 || PATCH | /api/admin/users/:id | **Edit user profile or activation status** | **Admin ONLY** | 200, 400, 401, 403, 404, 409, 500 || POST | /api/admin/users/:id/reset-password | **Set new initial password for user** | **Admin ONLY** | 200, 400, 401, 403, 404, 500 |

3. Detailed Endpoint Specifications

3.1 Authentication APIs

POST /api/auth/login

Authenticates user credentials and returns session token and user identity. - curl Example:

```
bash
curl -X POST http://localhost:3001/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"jennifer.anderson@toktickit.com","password":"InitialPassword123!"}'
```

- Request Body:

```
on
{
  "email": "jennifer.anderson@toktickit.com",
  "password": "InitialPassword123!"
}
```

- HTTP Responses: - **200 OK : Login successful.**

```
on
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "email": "jennifer.anderson@toktickit.com",
    "name": "Jennifer Anderson",
    "role": "REQUESTER",
    "mustChangePassword": true
  }
}
```

- **400 Bad Request : Email or password field missing.** - **401 Unauthorized : Invalid credentials or account deactivated (isActive = false).**

POST /api/auth/change-password

Updates user password and clears mustChangePassword flag. - **curl** Example:

```
bash
curl -X POST http://localhost:3001/api/auth/change-password \
-H "Authorization: Bearer " \
-H "Content-Type: application/json" \
-d '{"currentPassword":"InitialPassword123!","newPassword":"NewSecurePassword456!"}'
```

- HTTP Responses: - **200 OK : Password updated successfully; mustChangePassword is set to false .** - **400 Bad Request : New password does not meet strength rules or confirmation mismatch.** - **401 Unauthorized : Invalid current password or expired token.**

3.2 IT Staff Ticket Queue API

GET /api/staff/tickets

Retrieves paginated ticket queue for IT Staff and Administrator users. - **curl** Example:

```
bash
curl -X GET "http://localhost:3001/api/staff/tickets?search=battery&status=IN_PROGRESS&page=1&limit=10" \
-H "Authorization: Bearer "
```

- HTTP Responses: - 200 OK :

```
on
{
  "success": true,
  "data": [
    {
      "id": 12,
      "ticketNo": "TKT-2026-001234",
      "createdAt": "2026-09-15T08:14:00Z",
      "summary": "Laptop battery drains quickly",
      "category": "Hardware",
      "requestedPriority": "MEDIUM",
      "itPriority": "MEDIUM",
      "status": "IN_PROGRESS",
      "owner": { "id": 5, "name": "Michael Brown" }
    }
  ],
  "pagination": {
    "totalItems": 87,
    "totalPages": 9,
    "currentPage": 1,
    "pageSize": 10
  }
}
```

- 401 Unauthorized : Missing or invalid authentication. - 403 Forbidden : User role is REQUESTER (access restricted to IT Staff/Admin).

3.3 IT Staff Ticket Operations

PATCH /api/staff/tickets/:id/assign

Claim or reassign primary ticket ownership. - HTTP Responses: - 200 OK : Ticket owner updated. - 400 Bad Request : Owner ID does not belong to an active IT Staff or Admin user. - 404 Not Found : Ticket ID does not exist.

PATCH /api/staff/tickets/:id/workflow

Update IT Priority and Ticket Status. - HTTP Responses: - 200 OK : Status or IT Priority updated. - 400 Bad Request : Invalid status transition attempted. - 404 Not Found : Ticket ID not found.

3.4 Comments & Notes APIs

POST /api/tickets/:id/comments (Public Comment)

- curl Example:

```
bash
curl -X POST http://localhost:3001/api/tickets/12/comments \
-H "Authorization: Bearer " \
-H "Content-Type: application/json" \
-d '{"content":"We are investigating the issue on your device."}'
```

- HTTP Responses: - 201 Created : **Public comment saved.** - 400 Bad Request : **Content empty or whitespace-only.**

POST /api/tickets/:id/notes (Internal Note)

- curl Example:

```
bash
curl -X POST http://localhost:3001/api/tickets/12/notes \
-H "Authorization: Bearer " \
-H "Content-Type: application/json" \
-d '{"content":"Ordered battery replacement part."}'
```

- HTTP Responses: - 201 Created : **Internal note saved.** - 403 Forbidden : **Requester user requested endpoint (Internal notes are restricted to IT Staff & Admin only).**

3.5 Administrator User Management APIs

POST /api/admin/users

- curl Example:

```
bash
curl -X POST http://localhost:3001/api/admin/users \
-H "Authorization: Bearer " \
-H "Content-Type: application/json" \
-d '{"name":"Alex Thompson","email":"alex.thompson@toktickit.com","role":"IT_STAFF","isActive":true,"initialPassword":"InitialPassword123!"}'
```

- HTTP Responses: - 201 Created : **User created with mustChangePassword = true** . - 409 Conflict : **User email already exists.** - 403 Forbidden : **Non-admin user requested endpoint.**

PATCH /api/admin/users/:id

- HTTP Responses: - 200 OK : **User updated.** - 400 Bad Request : **Attempted self-deactivation or deactivation of the last active Administrator account.** - 404 Not Found : **User ID not found.**

Answer Part 3: Test DD and Traceability

The full Test Plan & Traceability Matrix is rendered below from docs/lab-03/tests.md.

TokTickIT Lab 3 Test Plan and Results

1. Test Strategy

The testing strategy covers Unit, API/Integration, UI Component, Security Authorization, and End-to-End (E2E) levels using Vitest, React Testing Library, and Playwright.

- Server API Tests: Executed in `server/tests/lab-03/` using Vitest & Supertest to verify authentication, session validation, role authorization, IT Staff Queue queries, Ticket operations, Comments/Notes scoping, and Administrator user management rules.
- Client UI Component Tests: Executed in `client/src/tests/lab-03/` using Vitest & React Testing Library to verify form validation, password rules, role navigation, queue filters/sorting, modal drawers, and responsive feedback.
- E2E Tests: Executed in `e2e/lab-03/` via Playwright across Desktop, Tablet, and Mobile viewports to verify complete multi-role workflows.

2. Planned Tests Table

Test ID	Level / Type	Requirement / AC	What It Tests	Expected Result	Automated Test File	Final Status
						:

1:

1:

1:

1:

1:

1:

: || API-01 | API | AC-01, FR-01 | Valid user authentication (Login) | 200 OK; returns user identity and role | `server/tests/lab-03/auth.api.test.ts` | Pass || API-02 | API | BR-01, FR-01 | Login with invalid password or inactive account | 401 Unauthorized; safe error message | `server/tests/lab-03/auth.api.test.ts` | Pass || API-03 | API | AC-02, BR-02 | Mandatory password change enforcement | Access blocked until valid new password saved | `server/tests/lab-03/auth.api.test.ts` | Pass || API-04 | API | AC-03, BR-03 | Requester ownership isolation with session auth | Returns only owned tickets regardless of client input | `server/tests/lab-`

03/authorization.api.test.ts | Pass | | API-05 | API | AC-04, BR-04 | Requester attempts to request Internal Notes endpoint | 403 Forbidden; no note content exposed | server/tests/lab-03/comments-notes.api.test.ts | Pass | | API-06 | API | AC-05, FR-05..08 | IT Staff Queue query with search, filter, sort, pagination | 200 OK; returns matching subset and pagination metadata | server/tests/lab-03/staff-queue.api.test.ts | Pass | | API-07 | API | AC-08, FR-09,10 | Claim/Reassign ticket owner and set IT Priority | 200 OK; updated ownership and IT Priority saved | server/tests/lab-03/staff-ticket-detail.api.test.ts | Pass | | API-08 | API | FR-11, BR-05 | Permitted status transition workflow | Status updated according to transition rules | server/tests/lab-03/staff-ticket-detail.api.test.ts | Pass | | API-09 | API | AC-11, FR-15..19 | Admin User Listing, Search, Filter, Create User | 200 OK / 201 Created; user created with single role | server/tests/lab-03/users-admin.api.test.ts | Pass | | API-10 | API | AC-06, BR-09,10 | Admin self-deactivation & last Admin protection | 400 Bad Request; server rejects dangerous action | server/tests/lab-03/users-admin.api.test.ts | Pass | | UI-01 | UI | FR-01, AC-01 | Login screen form validation & busy state | Validation errors shown; submit button displays spinner | client/src/tests/lab-03/Login.test.tsx | Pass | | UI-02 | UI | FR-02, AC-02 | Mandatory Password Change screen rule validation | Real-time checklist updates; submit enables when valid | client/src/tests/lab-03/ChangePassword.test.tsx | Pass | | UI-03 | UI | FR-05..08 | Staff Ticket Queue filter/sort controls & table | Filter dropdowns update list; badge colors match tokens | client/src/tests/lab-03/StaffTicketQueue.test.tsx | Pass | | UI-04 | UI | FR-09..13 | Staff Ticket Detail operational sidebar & comment tabs | Sidebar updates status/owner; Internal Notes styled amber | client/src/tests/lab-03/StaffTicketDetail.test.tsx | Pass | | UI-05 | UI | FR-15..19 | Admin User Management list & Create/Edit Drawer | Drawer opens; role dropdown & status toggle work | client/src/tests/lab-03/UserManagement.test.tsx | Pass | | E2E-01 | E2E | AC-01, AC-02 | Login & Mandatory First Password Change Flow | Login with initial pass -> forced change -> enter app | e2e/lab-03/authentication.spec.ts | Pass | | E2E-02 | E2E | AC-05, FR-09..13 | IT Staff Ticket Queue & Operational Flow | Login as Staff -> filter queue -> open ticket -> claim & note | e2e/lab-03/staff-ticket-flow.spec.ts | Pass | | E2E-03 | E2E | AC-06, FR-15..19 | Administrator User Management & Safety Flow | Login as Admin -> create user -> reset password -> safety guard | e2e/lab-03/user-administration.spec.ts | Pass |

3. Acceptance-Criterion Traceability Matrix

| Acceptance Criterion | Description | Covered Automated Tests | Verification Status | | :

| :

| :

| :

: | | AC-01 | User Login & Role Authentication | API-01 , UI-01 , E2E-01 | Covered | | AC-02 | Mandatory First Password Change Enforcement | API-03 , UI-02 , E2E-01 | Covered | | AC-03 | Requester Session Ownership Isolation | API-04 , authorization.api.test.ts | Covered | | AC-04 |

Internal Notes Role Protection (403 Forbidden) | API-05 , comments-notes.api.test.ts | Covered | | AC-05 | **IT Staff Queue Query, Search, Filter & Pagination** | API-06 , UI-03 , E2E-02 | Covered | | AC-06 | **Administrator Safety Guards (Self/Last Admin Protection)** | API-10 , UI-05 , E2E-03 | Covered | | AC-07 | **Inactive User Account Rejection (401 Unauthorized)** | API-02 , auth.api.test.ts | Covered | | AC-08 | **Ticket Ownership Claim/Reassign & IT Priority Update** | API-07 , UI-04 , E2E-02 | Covered | | AC-09 | **Role-Restricted Internal Notes Persistence & Visibility** | API-05 , UI-04 , E2E-02 | Covered | | AC-10 | **Requester Public Comment & Resolution Indicator** | comments-notes.api.test.ts , StaffTicketDetail.test.tsx | Covered | | AC-11 | **Admin User Creation with Initial Password & Single Role** | API-09 , UI-05 , E2E-03 | Covered | | AC-12 | **Responsive Viewport Reflow & Zen Green Styling** | E2E-01 , E2E-02 , E2E-03 | Covered |

4. Responsive and Visual Checklist

- [] Desktop ($\geq 992\text{px}$): **Full navigation bar; multi-column queue table; split ticket detail view.** - [] Tablet ($768\text{px} - 991\text{px}$): **Reflowed filter controls; responsive table wrapping.** - [] Mobile ($< 768\text{px}$): **Vertical card view for Queue and User Management; stacked communication panels.** - [] Zen Green Badge Styling**: Status, Requested Priority, IT Priority, and Role badges properly colored.

5. Test Commands

```
bash
```

Run server API test suite

```
cd server && npm test
```

Run client UI unit test suite

```
cd client && npm test
```

Run Playwright E2E test suite

```
npx playwright test
```

6. Final Results

All planned unit, API, UI component, authorization, and E2E tests pass cleanly on the final `main` branch.

3.3 Automated Backend API Test Suite Code

Backend Auth & Password Reset Test (`server/tests/lab-03/auth.api.test.ts`)

```

import { describe, it, expect } from "vitest";
import request from "supertest";
import { app } from "../../src/app.js";

describe("Lab 3 Authentication REST API Endpoints", () => {
  it("POST /api/auth/login - should authenticate valid user credentials (AC-01)", async () => {
    {
      const res = await request(app).post("/api/auth/login").send({
        email: "jennifer.anderson@toktickit.com",
        password: "InitialPassword123!",
      });

      expect(res.status).toBe(200);
      expect(res.body).toHaveProperty("success", true);
      expect(res.body).toHaveProperty("token");
      expect(res.body.user).toHaveProperty("email", "jennifer.anderson@toktickit.com");
      expect(res.body.user).toHaveProperty("role", "REQUESTER");
      expect(res.body.user).toHaveProperty("mustChangePassword", true);
    }
  });

  it("POST /api/auth/login - should reject invalid password or non-existent user", async () => {
    {
      const res = await request(app).post("/api/auth/login").send({
        email: "jennifer.anderson@toktickit.com",
        password: "WrongPassword999!",
      });

      expect(res.status).toBe(401);
      expect(res.body).toHaveProperty("success", false);
      expect(res.body.error).toHaveProperty("message");
    }
  });

  it("POST /api/auth/login - should reject inactive user account (AC-07, BR-01)", async () => {
    {
      const res = await request(app).post("/api/auth/login").send({
        email: "inactive.requester@toktickit.com",
        password: "Password123!",
      });

      expect(res.status).toBe(401);
      expect(res.body).toHaveProperty("success", false);
      expect(res.body.error.message).toContain("disabled");
    }
  });

  it("GET /api/auth/me - should return authenticated user profile", async () => {
    const loginRes = await request(app).post("/api/auth/login").send({
      email: "michael.brown@toktickit.com",
      password: "Password123!",
    });

    const token = loginRes.body.token;
    const meRes = await request(app)
      .get("/api/auth/me")
      .set("Authorization", Bearer ${token});
  });

```

```

    expect(meRes.status).toBe(200);
    expect(meRes.body).toHaveProperty("success", true);
    expect(meRes.body.user).toHaveProperty("email", "michael.brown@toktickit.com");
    expect(meRes.body.user).toHaveProperty("role", "REQUESTER");
  });

  it("GET /api/auth/me - should reject request without token", async () => {
    const res = await request(app).get("/api/auth/me");
    expect(res.status).toBe(401);
    expect(res.body).toHaveProperty("success", false);
  });

  it("POST /api/auth/change-password - should enforce valid password change (AC-02, BR-02)", async () => {
    // 1. Login with initial password
    const loginRes = await request(app).post("/api/auth/login").send({
      email: "jennifer.anderson@toktickit.com",
      password: "InitialPassword123!",
    });

    const token = loginRes.body.token;

    // 2. Change password
    const changeRes = await request(app)
      .post("/api/auth/change-password")
      .set("Authorization", Bearer ${token})
      .send({
        currentPassword: "InitialPassword123!",
        newPassword: "NewSecurePassword888!",
      });

    expect(changeRes.status).toBe(200);
    expect(changeRes.body).toHaveProperty("success", true);
    expect(changeRes.body.user).toHaveProperty("mustChangePassword", false);

    // 3. Verify login works with new password
    const newLoginRes = await request(app).post("/api/auth/login").send({
      email: "jennifer.anderson@toktickit.com",
      password: "NewSecurePassword888!",
    });

    expect(newLoginRes.status).toBe(200);
    expect(newLoginRes.body.user).toHaveProperty("mustChangePassword", false);

    // Reset password back for idempotent testing
    await request(app)
      .post("/api/auth/change-password")
      .set("Authorization", Bearer ${newLoginRes.body.token})
      .send({
        currentPassword: "NewSecurePassword888!",
        newPassword: "InitialPassword123!",
      });
  });

```

```
});
```

Requester Ticket Access Control Test (`server/tests/lab-03/requester-ticket.api.test.ts`)

```

import { describe, it, expect, beforeAll } from "vitest";
import request from "supertest";
import { app } from "../../src/app.js";

describe("Lab 3 Requester Ticket & Public Comment APIs", () => {
  let michaelToken: string;
  let jenniferToken: string;
  let adminToken: string;
  let createdTicketId: number;

  beforeAll(async () => {
    // Login as Michael Brown (Requester)
    const mRes = await request(app).post("/api/auth/login").send({
      email: "michael.brown@toktickit.com",
      password: "Password123!",
    });
    michaelToken = mRes.body.token;

    // Login as Jennifer Anderson (Requester)
    const jRes = await request(app).post("/api/auth/login").send({
      email: "jennifer.anderson@toktickit.com",
      password: "InitialPassword123!",
    });
    jenniferToken = jRes.body.token;

    // Login as John Smith (Admin)
    const aRes = await request(app).post("/api/auth/login").send({
      email: "john.smith@toktickit.com",
      password: "Password123!",
    });
    adminToken = aRes.body.token;
  });

  it("POST /api/requesters/tickets - should create a ticket under authenticated Requester identity", async () => {
    const res = await request(app)
      .post("/api/requesters/tickets")
      .set("Authorization", Bearer ${michaelToken})
      .send({
        categoryId: 1,
        relatedSystemId: 1,
        requestedPriority: "HIGH",
        summary: "Laptop Screen Flickering Issue",
        description: "The laptop display flickers randomly during work and requires hardware inspection.",
      });

    expect(res.status).toBe(201);
    expect(res.body.success).toBe(true);
    expect(res.body.data).toHaveProperty("id");
    expect(res.body.data.summary).toBe("Laptop Screen Flickering Issue");

    createdTicketId = res.body.data.id;
  });
});

```

```

it("GET /api/requesters/tickets - should return paginated tickets owned strictly by Requester", async () => {
  const res = await request(app)
    .get("/api/requesters/tickets")
    .set("Authorization", Bearer ${michaelToken});

  expect(res.status).toBe(200);
  expect(res.body.success).toBe(true);
  expect(Array.isArray(res.body.data)).toBe(true);
  expect(res.body.pagination).toHaveProperty("totalItems");
  expect(res.body.pagination).toHaveProperty("currentPage", 1);
});

it("GET /api/requesters/tickets - should reject unauthenticated requests with 401", async () => {
  const res = await request(app).get("/api/requesters/tickets");
  expect(res.status).toBe(401);
  expect(res.body.success).toBe(false);
});

it("POST /api/tickets/:id/comments - should post a public comment on ticket", async () => {
  const res = await request(app)
    .post(/api/tickets/${createdTicketId}/comments)
    .set("Authorization", Bearer ${michaelToken})
    .send({
      content: "Adding additional screenshot details regarding the screen flicker.",
    });

  expect(res.status).toBe(201);
  expect(res.body.success).toBe(true);
  expect(res.body.data.content).toBe("Adding additional screenshot details regarding the screen flicker.");
  expect(res.body.data.author.email).toBe("michael.brown@toktickit.com");
});

it("GET /api/tickets/:id/comments - should retrieve public comments for ticket", async () => {
  const res = await request(app)
    .get(/api/tickets/${createdTicketId}/comments)
    .set("Authorization", Bearer ${michaelToken});

  expect(res.status).toBe(200);
  expect(res.body.success).toBe(true);
  expect(Array.isArray(res.body.data)).toBe(true);
  expect(res.body.data.length).toBeGreaterThan(0);
});

it("POST /api/tickets/:id/comments - should prevent Requester from commenting on another user's ticket (403)", async () => {
  const res = await request(app)
    .post(/api/tickets/${createdTicketId}/comments)
    .set("Authorization", Bearer ${jenniferToken})
    .send({
      content: "Unauthorized attempt to comment.",
    });
});

```

```
    expect(res.status).toBe(403);  
    expect(res.body.success).toBe(false);  
  });  
});
```

IT Staff Ticket Queue & Internal Notes Test (`server/tests/lab-03/staff-queue.api.test.ts`)

```

import { describe, it, expect, beforeAll } from "vitest";
import request from "supertest";
import { app } from "../../src/app.js";

describe("Lab 3 Staff Queue, Workflow & Internal Notes APIs", () => {
  let kevinToken: string; // IT_STAFF
  let michaelToken: string; // REQUESTER
  let createdTicketId: number;

  beforeAll(async () => {
    // Login as Kevin Patel (IT Staff)
    const kRes = await request(app).post("/api/auth/login").send({
      email: "kevin.patel@toktickit.com",
      password: "Password123!",
    });
    kevinToken = kRes.body.token;

    // Login as Michael Brown (Requester)
    const mRes = await request(app).post("/api/auth/login").send({
      email: "michael.brown@toktickit.com",
      password: "Password123!",
    });
    michaelToken = mRes.body.token;

    // Create a test ticket as Requester for IT Staff queue handling
    const tRes = await request(app)
      .post("/api/requesters/tickets")
      .set("Authorization", Bearer ${michaelToken})
      .send({
        categoryId: 1,
        relatedSystemId: 1,
        requestedPriority: "HIGH",
        summary: "VPN Connection Timeout Investigation",
        description: "VPN client fails to complete handshake when connecting from remote home
network.",
      });
    createdTicketId = tRes.body.data.id;
  });

  it("GET /api/staff/tickets - should return ticket queue for IT Staff", async () => {
    const res = await request(app)
      .get("/api/staff/tickets")
      .set("Authorization", Bearer ${kevinToken});

    expect(res.status).toBe(200);
    expect(res.body.success).toBe(true);
    expect(Array.isArray(res.body.data)).toBe(true);
    expect(res.body.pagination).toHaveProperty("totalItems");
  });

  it("GET /api/staff/tickets - should reject Requester with 403 Forbidden", async () => {
    const res = await request(app)
      .get("/api/staff/tickets")
      .set("Authorization", Bearer ${michaelToken});
  });

```

```

    expect(res.status).toBe(403);
    expect(res.body.success).toBe(false);
  });

  it("PATCH /api/staff/tickets/:id/assign - should claim ticket ownership for IT Staff", async () => {
    const res = await request(app)
      .patch(/api/staff/tickets/${createdTicketId}/assign)
      .set("Authorization", Bearer ${kevinToken})
      .send({});

    expect(res.status).toBe(200);
    expect(res.body.success).toBe(true);
    expect(res.body.data.owner).toBeDefined();
    expect(res.body.data.owner.email).toBe("kevin.patel@toktickit.com");
  });

  it("PATCH /api/staff/tickets/:id/workflow - should update IT Priority and Status", async () => {
    const res = await request(app)
      .patch(/api/staff/tickets/${createdTicketId}/workflow)
      .set("Authorization", Bearer ${kevinToken})
      .send({
        itPriority: "URGENT",
        status: "IN_PROGRESS",
      });

    expect(res.status).toBe(200);
    expect(res.body.success).toBe(true);
    expect(res.body.data.itPriority).toBe("URGENT");
    expect(res.body.data.status).toBe("IN_PROGRESS");
  });

  it("POST /api/tickets/:id/notes - should allow IT Staff to post Internal Note", async () => {
    const res = await request(app)
      .post(/api/tickets/${createdTicketId}/notes)
      .set("Authorization", Bearer ${kevinToken})
      .send({
        content: "Verified firewall logs; user IP was blocked by rate limiter.",
      });

    expect(res.status).toBe(201);
    expect(res.body.success).toBe(true);
    expect(res.body.data.content).toBe("Verified firewall logs; user IP was blocked by rate limiter.");
  });

  it("GET /api/tickets/:id/notes - should prevent Requester from viewing Internal Notes (403)", async () => {
    const res = await request(app)
      .get(/api/tickets/${createdTicketId}/notes)
      .set("Authorization", Bearer ${michaelToken});

    expect(res.status).toBe(403);
  });

```

```
    expect(res.body.success).toBe(false);  
  });  
});
```

Admin User Management & Safety Rules Test (`server/tests/lab-03/admin-user.api.test.ts`)

```

import { describe, it, expect, beforeAll } from "vitest";
import request from "supertest";
import { app } from "../../src/app.js";

describe("Lab 3 Administrator User Management REST APIs", () => {
  let adminToken: string; // ADMINISTRATOR
  let staffToken: string; // IT_STAFF
  let requesterToken: string; // REQUESTER
  let createdUserId: number;

  beforeAll(async () => {
    // Login as John Smith (Admin)
    const aRes = await request(app).post("/api/auth/login").send({
      email: "john.smith@toktickit.com",
      password: "Password123!",
    });
    adminToken = aRes.body.token;

    // Login as Kevin Patel (IT Staff)
    const sRes = await request(app).post("/api/auth/login").send({
      email: "kevin.patel@toktickit.com",
      password: "Password123!",
    });
    staffToken = sRes.body.token;

    // Login as Michael Brown (Requester)
    const rRes = await request(app).post("/api/auth/login").send({
      email: "michael.brown@toktickit.com",
      password: "Password123!",
    });
    requesterToken = rRes.body.token;
  });

  it("GET /api/admin/users - should return paginated user list for Administrator", async () => {
    const res = await request(app)
      .get("/api/admin/users")
      .set("Authorization", Bearer ${adminToken});

    expect(res.status).toBe(200);
    expect(res.body.success).toBe(true);
    expect(Array.isArray(res.body.data)).toBe(true);
    expect(res.body.pagination).toHaveProperty("totalItems");
  });

  it("GET /api/admin/users - should reject non-Admin users with 403 Forbidden", async () => {
    const staffRes = await request(app)
      .get("/api/admin/users")
      .set("Authorization", Bearer ${staffToken});
    expect(staffRes.status).toBe(403);

    const reqRes = await request(app)
      .get("/api/admin/users")
      .set("Authorization", Bearer ${requesterToken});
  });
});

```

```

    expect(reqRes.status).toBe(403);
  });

  it("POST /api/admin/users - should create a new user with initial password and mustChangePas
sword=true", async () => {
    const res = await request(app)
      .post("/api/admin/users")
      .set("Authorization", Bearer ${adminToken})
      .send({
        name: "Alex Thompson",
        email: "alex.thompson.test@toktickit.com",
        department: "Infrastructure",
        role: "IT_STAFF",
        isActive: true,
        initialPassword: "InitialPassword123!",
      });

    expect(res.status).toBe(201);
    expect(res.body.success).toBe(true);
    expect(res.body.data.name).toBe("Alex Thompson");
    expect(res.body.data.role).toBe("IT_STAFF");
    expect(res.body.data.mustChangePassword).toBe(true);

    createdUserId = res.body.data.id;
  });

  it("POST /api/admin/users - should reject duplicate email with 409 Conflict", async () => {
    const res = await request(app)
      .post("/api/admin/users")
      .set("Authorization", Bearer ${adminToken})
      .send({
        name: "Duplicate User",
        email: "alex.thompson.test@toktickit.com",
        role: "REQUESTER",
      });

    expect(res.status).toBe(409);
    expect(res.body.success).toBe(false);
  });

  it("PATCH /api/admin/users/:id - should update user profile and role", async () => {
    const res = await request(app)
      .patch(`/api/admin/users/${createdUserId}`)
      .set("Authorization", Bearer ${adminToken})
      .send({
        name: "Alex Thompson Updated",
        department: "DevOps",
      });

    expect(res.status).toBe(200);
    expect(res.body.success).toBe(true);
    expect(res.body.data.name).toBe("Alex Thompson Updated");
    expect(res.body.data.department).toBe("DevOps");
  });

  it("PATCH /api/admin/users/:id - should prevent Administrator self-deactivation (400)", asyn

```

```

c () => {
  const meRes = await request(app)
    .get("/api/auth/me")
    .set("Authorization", Bearer ${adminToken});
  const adminId = meRes.body.user.id;

  const res = await request(app)
    .patch(/api/admin/users/${adminId})
    .set("Authorization", Bearer ${adminToken})
    .send({
      isActive: false,
    });

  expect(res.status).toBe(400);
  expect(res.body.success).toBe(false);
  expect(res.body.error.message).toContain("Self-deactivation");
});

it("PATCH /api/admin/users/:id - should prevent deactivating or downgrading the last active
Administrator account (400)", async () => {
  const meRes = await request(app)
    .get("/api/auth/me")
    .set("Authorization", Bearer ${adminToken});
  const adminId = meRes.body.user.id;

  // Test downgrading last admin role
  const roleRes = await request(app)
    .patch(/api/admin/users/${adminId})
    .set("Authorization", Bearer ${adminToken})
    .send({
      role: "IT_STAFF",
    });

  expect(roleRes.status).toBe(400);
  expect(roleRes.body.success).toBe(false);
  expect(roleRes.body.error.message).toContain("last active Administrator");
});

it("POST /api/admin/users/:id/reset-password - should reset user initial password", async ()
=> {
  const res = await request(app)
    .post(/api/admin/users/${createdUserId}/reset-password)
    .set("Authorization", Bearer ${adminToken})
    .send({
      newInitialPassword: "NewTempPassword456!",
    });

  expect(res.status).toBe(200);
  expect(res.body.success).toBe(true);
  expect(res.body.data.mustChangePassword).toBe(true);
});
});

```

3.4 Automated Client RTL UI Component Test Code


```

import { render, screen, fireEvent, waitFor } from "@testing-library/react";
import { describe, it, expect, vi } from "vitest";
import { Login } from "../../src/pages/Login";
import { AuthProvider } from "../../src/contexts/AuthContext";
import * as api from "../../src/api";

vi.mock("../../src/api", async (importOriginal) => {
  const actual: any = await importOriginal();
  return {
    ...actual,
    loginApi: vi.fn(),
    getMeApi: vi.fn(),
    logoutApi: vi.fn(),
    changePasswordApi: vi.fn(),
  };
});

describe("Lab 3 Auth UI: Login Screen", () => {
  it("should render the login form with title and input fields", () => {
    render(

    );

    expect(screen.getByText(/Sign in to TokTickIT/i)).toBeInTheDocument();
    expect(screen.getByLabelText(/Email Address/i)).toBeInTheDocument();
    expect(screen.getByLabelText(/Password/i)).toBeInTheDocument();
    expect(screen.getByRole("button", { name: /Sign In/i })).toBeInTheDocument();
  });

  it("should show validation error when fields are empty", async () => {
    render(

    );

    const submitBtn = screen.getByRole("button", { name: /Sign In/i });
    fireEvent.click(submitBtn);

    expect(
      await screen.findByText(/Please fill in both Email and Password fields/i)
    ).toBeInTheDocument();
  });

  it("should display error message on failed login API response", async () => {
    vi.mocked(api.loginApi).mockRejectedValueOnce(
      new Error("Invalid credentials or account deactivated")
    );

    render(

```

```

    );

    fireEvent.change(screen.getByLabelText(/Email Address/i), {
      target: { value: "wrong@toktickit.com" },
    });
    fireEvent.change(screen.getByLabelText(/Password/i), {
      target: { value: "WrongPass123" },
    });

    fireEvent.click(screen.getByRole("button", { name: /Sign In/i }));

    expect(
      await screen.findByText(/Invalid credentials or account deactivated/i)
    ).toBeInTheDocument();
  });

it("should call loginApi and succeed with valid credentials", async () => {
  const mockUser = {
    id: 1,
    email: "jennifer.anderson@toktickit.com",
    name: "Jennifer Anderson",
    role: "REQUESTER" as const,
    mustChangePassword: false,
  };

  vi.mocked(api.loginApi).mockResolvedValueOnce({
    success: true,
    token: "mock-jwt-token",
    user: mockUser,
  });

  const handleSuccess = vi.fn();

  render(

  );

  fireEvent.change(screen.getByLabelText(/Email Address/i), {
    target: { value: "jennifer.anderson@toktickit.com" },
  });
  fireEvent.change(screen.getByLabelText(/Password/i), {
    target: { value: "InitialPassword123!" },
  });

  fireEvent.click(screen.getByRole("button", { name: /Sign In/i }));

  await waitFor(() => {
    expect(api.loginApi).toHaveBeenCalledWith(
      "jennifer.anderson@toktickit.com",
      "InitialPassword123!"
    );
    expect(handleSuccess).toHaveBeenCalled();
  });
});

```

```
});  
});
```

Change Password UI Component Test (`client/tests/lab-03/ChangePassword.test.tsx`)

```

import { render, screen, fireEvent, waitFor } from "@testing-library/react";
import { describe, it, expect, vi } from "vitest";
import { ChangePassword } from "../../src/pages/ChangePassword";
import { AuthProvider } from "../../src/contexts/AuthContext";
import * as api from "../../src/api";

vi.mock("../../src/api", async (importOriginal) => {
  const actual: any = await importOriginal();
  return {
    ...actual,
    loginApi: vi.fn(),
    getMeApi: vi.fn(),
    logoutApi: vi.fn(),
    changePasswordApi: vi.fn(),
  };
});

describe("Lab 3 Auth UI: Change Password Screen", () => {
  it("should render mandatory change password title and criteria checklist", () => {
    render(

    );

    expect(screen.getByText(/Mandatory Password Update/i)).toBeInTheDocument();
    expect(screen.getByLabelText(/Current Password/i)).toBeInTheDocument();
    expect(screen.getByLabelText(/^New Password$/i)).toBeInTheDocument();
    expect(screen.getByLabelText(/^Confirm New Password$/i)).toBeInTheDocument();
    expect(screen.getByText(/At least 8 characters long/i)).toBeInTheDocument();
  });

  it("should keep submit button disabled until all criteria are met", () => {
    render(

    );

    const submitBtn = screen.getByRole("button", { name: /Save New Password/i });
    expect(submitBtn).toBeDisabled();

    // Fill current password
    fireEvent.change(screen.getByLabelText(/Current Password/i), {
      target: { value: "InitialPassword123!" },
    });

    // Fill invalid weak password
    fireEvent.change(screen.getByLabelText(/^New Password$/i), {
      target: { value: "short" },
    });
    fireEvent.change(screen.getByLabelText(/^Confirm New Password$/i), {
      target: { value: "short" },
    });
  });
});

```

```

expect(submitBtn).toBeDisabled();

// Fill valid password meeting all rules
fireEvent.change(screen.getByLabelText(/^New Password$/i), {
  target: { value: "NewSecurePassword456!" },
});
fireEvent.change(screen.getByLabelText(/^Confirm New Password$/i), {
  target: { value: "NewSecurePassword456!" },
});

expect(submitBtn).not.toBeDisabled();
});

it("should call changePasswordApi upon form submission", async () => {
  vi.mocked(api.changePasswordApi).mockResolvedValueOnce({
    success: true,
    message: "Password changed successfully",
  });

  // Mock localStorage token so useAuth has a token
  localStorage.setItem("toktickit_token", "mock-valid-jwt");

  const handleSuccess = vi.fn();

  render(

);

  fireEvent.change(screen.getByLabelText(/Current Password/i), {
    target: { value: "InitialPassword123!" },
  });
  fireEvent.change(screen.getByLabelText(/^New Password$/i), {
    target: { value: "NewSecurePassword456!" },
  });
  fireEvent.change(screen.getByLabelText(/^Confirm New Password$/i), {
    target: { value: "NewSecurePassword456!" },
  });

  const submitBtn = screen.getByRole("button", { name: /Save New Password/i });
  fireEvent.click(submitBtn);

  await waitFor(() => {
    expect(api.changePasswordApi).toHaveBeenCalledWith(
      "mock-valid-jwt",
      "InitialPassword123!",
      "NewSecurePassword456!"
    );
  });

  localStorage.removeItem("toktickit_token");
});
});

```

Application Integration Test (`client/tests/lab-03/AppIntegration.test.tsx`)

```

import { render, screen, fireEvent, waitFor } from "@testing-library/react";
import { describe, it, expect, vi } from "vitest";
import App from "../../src/App";
import * as api from "../../src/api";

vi.mock("../../src/api", async (importOriginal) => {
  const actual: any = await importOriginal();
  return {
    ...actual,
    loginApi: vi.fn(),
    getMeApi: vi.fn(),
    logoutApi: vi.fn(),
    changePasswordApi: vi.fn(),
    fetchTickets: vi.fn().mockResolvedValue({
      success: true,
      data: [],
      meta: { page: 1, limit: 10, totalItems: 0, totalPages: 1 },
    }),
    fetchCategories: vi.fn().mockResolvedValue([]),
    fetchRelatedSystems: vi.fn().mockResolvedValue([]),
    fetchRequesters: vi.fn().mockResolvedValue([
      { id: 1, name: "Jennifer Anderson", email: "jennifer@toktickit.com", department: "CPE"
    },
  ]),
  ];
});

describe("Lab 3 E2E Client Application Integration", () => {
  it("should navigate to Login screen when clicking Sign In button", async () => {
    render();
    const signInBtn = await screen.findByRole("button", { name: /^Sign In$/i });
    fireEvent.click(signInBtn);

    expect(await screen.findByText(/Sign in to TokTickIT/i)).toBeInTheDocument();
    expect(screen.getByLabelText(/Email Address/i)).toBeInTheDocument();
  });

  it("should authenticate user and render role navigation header", async () => {
    const mockUser = {
      id: 1,
      email: "jennifer.anderson@toktickit.com",
      name: "Jennifer Anderson",
      role: "REQUESTER" as const,
      mustChangePassword: false,
    };

    vi.mocked(api.loginApi).mockResolvedValueOnce({
      success: true,
      token: "mock-jwt-token",
      user: mockUser,
    });

    render();
    const signInBtn = await screen.findByRole("button", { name: /^Sign In$/i });
  });

```

```
fireEvent.click(signInBtn);

fireEvent.change(screen.getByLabelText(/Email Address/i), {
  target: { value: "jennifer.anderson@toktickit.com" },
});
fireEvent.change(screen.getByLabelText(/Password/i), {
  target: { value: "InitialPassword123!" },
});

fireEvent.click(screen.getByRole("button", { name: /Sign In/i }));

await waitFor(() => {
  expect(screen.getByText("Jennifer Anderson")).toBeInTheDocument();
  expect(screen.getByText("Requester")).toBeInTheDocument();
});
});
});
```

Answer Part 4: Database Schema Evolution & Idempotent Seed Evidence

4.1 Prisma Database Schema (`server/prisma/schema.prisma`)

```
// TokTickIT Prisma Schema – Lab 3 Database ORM & Data Models
```

```
// Feature Branch: feature/lab03-3-db-schema
```

```
generator client {  
  provider = "prisma-client-js"  
}
```

```
datasource db {  
  provider = "postgresql"  
  url      = env("DATABASE_URL")  
}
```

```
//
```

```
// Enums
```

```
//
```

```
enum Role {  
    REQUESTER  
    IT_STAFF  
    ADMINISTRATOR  
}
```

```
enum RequestedPriority {  
    LOW  
    MEDIUM  
    HIGH  
    URGENT  
}
```

```
enum ITPriority {  
    LOW  
    MEDIUM  
    HIGH  
    URGENT  
}
```

```
enum TicketStatus {  
    NEW  
    OPEN  
    IN_PROGRESS  
    WAITING_FOR_REQUESTER  
    RESOLVED  
    CLOSED  
    REOPENED  
    CANCELLED
```

```
}
```

```
//
```

```
// Lab 3 Data Models
```

```
//
```

```

model User {
  id          Int          @id @default(autoincrement())
  email       String       @unique
  passwordHash String
  name        String
  department  String?
  role        Role         @default(REQUESTER)
  isActive    Boolean      @default(true)
  mustChangePassword Boolean @default(true)
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt

  tickets      Ticket[]      @relation("RequesterTickets")
  ownedTickets Ticket[]      @relation("OwnerTickets")
  publicComments PublicComment[]
  internalNotes InternalNote[]

  @@index([role, isActive])
}

model RequesterUser {
  id          Int          @id @default(autoincrement())
  name        String
  email       String       @unique
  department  String
  isActive    Boolean      @default(true)
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
}

model Category {
  id          Int          @id @default(autoincrement())
  name        String       @unique
  isActive    Boolean      @default(true)
  createdAt   DateTime     @default(now())

  tickets Ticket[]
}

model RelatedSystem {

```

```

    id          Int          @id @default(autoincrement())
    name        String       @unique
    isActive    Boolean      @default(true)
    createdAt   DateTime      @default(now())

    tickets Ticket[]
}

model Ticket {
    id          Int          @id @default(autoincrement())
    ticketNo    String       @unique
    requesterId Int
    ownerId     Int?
    categoryId  Int
    relatedSystemId Int
    requestedPriority RequestedPriority @default(MEDIUM)
    itPriority   ITPriority?
    status       TicketStatus @default(NEW)
    summary      String
    description   String
    requesterResolvedInd Boolean @default(false)
    createdAt     DateTime      @default(now())
    updatedAt     DateTime      @updatedAt

    requester User @relation("RequesterTickets", fields: [requesterId], references: [id])
    owner      User? @relation("OwnerTickets", fields: [ownerId], references: [id])
    category   Category @relation(fields: [categoryId], references: [id])
    relatedSystem RelatedSystem @relation(fields: [relatedSystemId], references: [id])
    attachments Attachment[]
    publicComments PublicComment[]
    internalNotes InternalNote[]

    @@index([requesterId, createdAt])
    @@index([ownerId, status])
    @@index([status])
    @@index([categoryId])
}

model Attachment {
    id          Int          @id @default(autoincrement())
    ticketId    Int
    fileName     String
    fileKey      String
    fileSize     Int
    mimeType     String
    isRemoved    Boolean     @default(false)
    removalReason String?
    removedAt    DateTime?
    createdAt    DateTime     @default(now())

    ticket Ticket @relation(fields: [ticketId], references: [id], onDelete: Cascade)

    @@index([ticketId])
}

```

```

model PublicComment {
  id          Int          @id @default(autoincrement())
  ticketId    Int
  authorId    Int
  content      String
  createdAt   DateTime @default(now())

  ticket Ticket @relation(fields: [ticketId], references: [id], onDelete: Cascade)
  author User   @relation(fields: [authorId], references: [id])

  @@index([ticketId, createdAt])
}

model InternalNote {
  id          Int          @id @default(autoincrement())
  ticketId    Int
  authorId    Int
  content      String
  createdAt   DateTime @default(now())

  ticket Ticket @relation(fields: [ticketId], references: [id], onDelete: Cascade)
  author User   @relation(fields: [authorId], references: [id])

  @@index([ticketId, createdAt])
}

```

4.2 Idempotent Database Seed Script (`server/prisma/seed.ts`)

```
import { getPrisma } from "../src/prisma.js";
import bcrypt from "bcryptjs";
import { Role, RequestedPriority, ITPriority, TicketStatus } from "@prisma/client";

async function main() {
  const prisma = getPrisma();

  console.log("Seeding Lab 3 database...");

  // 1. Seed Categories
  const categories = [
    "Account and Access",
    "Hardware",
    "Software",
    "Network",
  ];

  const categoryMap: Record = {};
  for (const name of categories) {
    const cat = await prisma.category.upsert({
      where: { name },
      update: { isActive: true },
      create: { name, isActive: true },
    });
    categoryMap[name] = cat.id;
  }

  // 2. Seed Related Systems
  const relatedSystems = [
    "Email",
    "Campus Wi-Fi",
    "VPN",
    "LEB2 App",
    "Grade Submission App",
    "Printer",
    "Corporate Laptop",
  ];

  const systemMap: Record = {};
  for (const name of relatedSystems) {
    const sys = await prisma.relatedSystem.upsert({
      where: { name },
      update: { isActive: true },
      create: { name, isActive: true },
    });
    systemMap[name] = sys.id;
  }

  // Standard hashed password: "InitialPassword123!" and "Password123!"
  const initialPasswordHash = await bcrypt.hash("InitialPassword123!", 10);
  const standardPasswordHash = await bcrypt.hash("Password123!", 10);

  // 3. Seed Users (Requesters, IT Staff, Administrator)
  const usersData = [
```

```
// Requesters (4 active, 1 inactive)
{
  email: "jennifer.anderson@toktickit.com",
  name: "Jennifer Anderson",
  department: "Computer Engineering",
  role: Role.REQUESTER,
  isActive: true,
  mustChangePassword: true, // For testing mandatory first-login password change
  passwordHash: initialPasswordHash,
},
{
  email: "michael.brown@toktickit.com",
  name: "Michael Brown",
  department: "Information Technology",
  role: Role.REQUESTER,
  isActive: true,
  mustChangePassword: false,
  passwordHash: standardPasswordHash,
},
{
  email: "sarah.johnson@toktickit.com",
  name: "Sarah Johnson",
  department: "Electronic Engineering",
  role: Role.REQUESTER,
  isActive: true,
  mustChangePassword: false,
  passwordHash: standardPasswordHash,
},
{
  email: "david.lee@toktickit.com",
  name: "David Lee",
  department: "Software Engineering",
  role: Role.REQUESTER,
  isActive: true,
  mustChangePassword: false,
  passwordHash: standardPasswordHash,
},
{
  email: "inactive.requester@toktickit.com",
  name: "Inactive Requester Account",
  department: "Archived",
  role: Role.REQUESTER,
  isActive: false,
  mustChangePassword: false,
  passwordHash: standardPasswordHash,
},

// IT Staff (3 active, 1 inactive)
{
  email: "kevin.patel@toktickit.com",
  name: "Kevin Patel",
  department: "IT Support Desk",
  role: Role.IT_STAFF,
  isActive: true,
  mustChangePassword: false,
  passwordHash: standardPasswordHash,
```

```

    },
    {
      email: "emily.davis@toktickit.com",
      name: "Emily Davis",
      department: "Network Operations",
      role: Role.IT_STAFF,
      isActive: true,
      mustChangePassword: false,
      passwordHash: standardPasswordHash,
    },
    {
      email: "lisa.martinez@toktickit.com",
      name: "Lisa Martinez",
      department: "Systems Administration",
      role: Role.IT_STAFF,
      isActive: true,
      mustChangePassword: false,
      passwordHash: standardPasswordHash,
    },
    {
      email: "inactive.staff@toktickit.com",
      name: "Inactive Staff Account",
      department: "IT Staff Archived",
      role: Role.IT_STAFF,
      isActive: false,
      mustChangePassword: false,
      passwordHash: standardPasswordHash,
    },

    // Administrator (1 active)
    {
      email: "john.smith@toktickit.com",
      name: "John Smith",
      department: "System Administration",
      role: Role.ADMINISTRATOR,
      isActive: true,
      mustChangePassword: false,
      passwordHash: standardPasswordHash,
    },
  ];

const userMap: Record = {};
for (const u of usersData) {
  const user = await prisma.user.upsert({
    where: { email: u.email },
    update: {
      name: u.name,
      department: u.department,
      role: u.role,
      isActive: u.isActive,
      mustChangePassword: u.mustChangePassword,
    },
    create: u,
  });
  userMap[u.email] = user.id;
}

```

```

// Maintain RequesterUser table compatibility
if (u.role === Role.REQUESTER) {
  await prisma.requesterUser.upsert({
    where: { email: u.email },
    update: { name: u.name, department: u.department || "", isActive: u.isActive },
    create: { name: u.name, email: u.email, department: u.department || "", isActive: u.isActive },
  });
}

}

// 4. Seed Realistic Tickets
const sampleTickets = [
  {
    ticketNo: "TKT-2026-001234",
    requesterEmail: "jennifer.anderson@toktickit.com",
    categoryName: "Hardware",
    systemName: "Corporate Laptop",
    requestedPriority: RequestedPriority.MEDIUM,
    itPriority: ITPriority.MEDIUM,
    status: TicketStatus.IN_PROGRESS,
    ownerEmail: "kevin.patel@toktickit.com",
    summary: "Laptop battery drains quickly",
    description: "My laptop battery is draining much faster than usual even when idling.",
    requesterResolvedInd: false,
  },
  {
    ticketNo: "TKT-2026-001233",
    requesterEmail: "michael.brown@toktickit.com",
    categoryName: "Network",
    systemName: "VPN",
    requestedPriority: RequestedPriority.HIGH,
    itPriority: ITPriority.HIGH,
    status: TicketStatus.OPEN,
    ownerEmail: null,
    summary: "Cannot connect to VPN",
    description: "Getting timeout error when authenticating through corporate VPN client.",
    requesterResolvedInd: false,
  },
  {
    ticketNo: "TKT-2026-001232",
    requesterEmail: "david.lee@toktickit.com",
    categoryName: "Software",
    systemName: "Email",
    requestedPriority: RequestedPriority.MEDIUM,
    itPriority: ITPriority.MEDIUM,
    status: TicketStatus.IN_PROGRESS,
    ownerEmail: "emily.davis@toktickit.com",
    summary: "Email not syncing on mobile",
    description: "Outlook application on Android mobile device fails to fetch new emails.",
    requesterResolvedInd: false,
  },
  {
    ticketNo: "TKT-2026-001231",
    requesterEmail: "jennifer.anderson@toktickit.com",
    categoryName: "Account and Access",

```

```

    systemName: "LEB2 App",
    requestedPriority: RequestedPriority.LOW,
    itPriority: ITPriority.LOW,
    status: TicketStatus.RESOLVED,
    ownerEmail: "lisa.martinez@toktickit.com",
    summary: "New employee setup request",
    description: "Please provision standard LEB2 course access for new TA starting next week.",
    requesterResolvedInd: true,
  },
];

for (const t of sampleTickets) {
  const requesterId = userMap[t.requesterEmail];
  const ownerId = t.ownerEmail ? userMap[t.ownerEmail] : null;
  const categoryId = categoryMap[t.categoryName];
  const relatedSystemId = systemMap[t.systemName];

  const createdTicket = await prisma.ticket.upsert({
    where: { ticketNo: t.ticketNo },
    update: {
      status: t.status,
      itPriority: t.itPriority,
      ownerId: ownerId,
      requesterResolvedInd: t.requesterResolvedInd,
    },
    create: {
      ticketNo: t.ticketNo,
      requesterId,
      ownerId,
      categoryId,
      relatedSystemId,
      requestedPriority: t.requestedPriority,
      itPriority: t.itPriority,
      status: t.status,
      summary: t.summary,
      description: t.description,
      requesterResolvedInd: t.requesterResolvedInd,
    },
  });

  // Seed comments/notes for TKT-2026-001234
  if (t.ticketNo === "TKT-2026-001234") {
    await prisma.publicComment.createMany({
      data: [
        {
          ticketId: createdTicket.id,
          authorId: userMap["jennifer.anderson@toktickit.com"],
          content: "Just adding that this issue occurs even when I close all applications.",
          createdAt: new Date("2026-05-12T09:20:00Z"),
        },
        {
          ticketId: createdTicket.id,
          authorId: userMap["kevin.patel@toktickit.com"],
          content: "We are investigating the issue on your device. We will update you shortly.",
        },
      ],
    });
  }
}

```

```

        createdAt: new Date("2026-05-13T10:30:00Z"),
      },
    ],
    skipDuplicates: true,
  });

  await prisma.internalNote.createMany({
    data: [
      {
        ticketId: createdTicket.id,
        authorId: userMap["kevin.patel@tokticket.com"],
        content: "Ran battery diagnostics. Wear level is at 45%. Ordering replacement batt
ery unit.",
        createdAt: new Date("2026-05-13T10:35:00Z"),
      },
    ],
    skipDuplicates: true,
  });
}
}

console.log("Seeding completed successfully.");
}

main()
  .catch((e) => {
    console.error(e);
    process.exit(1);
  })
  .finally(async () => {
    await getPrisma().$disconnect();
  });

```

Answer Part 5: Backend Implementation Source Code Evidence

5.1 Express Server Entrypoint (`server/src/app.ts`)

```

import express, { Request, Response, NextFunction } from "express";
import cors from "cors";
import path from "path";
import fs from "fs";
import multer from "multer";
import { RequestedPriority } from "@prisma/client";
import { getPrisma } from "../prisma.js";
import { authRouter } from "../routes/auth.js";
import { requesterTicketsRouter } from "../routes/requesterTickets.js";
import { staffTicketsRouter } from "../routes/staffTickets.js";
import { adminUsersRouter } from "../routes/adminUsers.js";

export const app = express();

app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use("/api/auth", authRouter);
app.use("/api", requesterTicketsRouter);
app.use("/api", staffTicketsRouter);
app.use("/api", adminUsersRouter);

// Ensure upload directory exists
const uploadDir = path.join(process.cwd(), "uploads");
if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir, { recursive: true });
}

// Multer storage setup
const storage = multer.diskStorage({
  destination: (_req, _file, cb) => {
    cb(null, uploadDir);
  },
  filename: (_req, file, cb) => {
    const uniqueSuffix = Date.now() + "-" + Math.round(Math.random() * 1e9);
    const ext = path.extname(file.originalname);
    cb(null, `${uniqueSuffix}${ext}`);
  },
});

const allowedMimeTypes = ["image/jpeg", "image/png", "image/webp", "application/pdf"];

const upload = multer({
  storage,
  limits: { fileSize: 5 * 1024 * 1024 }, // 5 MB
  fileFilter: (_req, file, cb) => {
    if (allowedMimeTypes.includes(file.mimetype)) {
      cb(null, true);
    } else {
      cb(new Error("UNSUPPORTED_FILE_TYPE"));
    }
  },
});

```

```
// Sequence generator helper for Ticket Numbers: TKT-2026-XXXXXX
async function generateTicketNumber(): Promise {
  const prisma = getPrisma();
  const year = new Date().getFullYear();
  const lastTicket = await prisma.ticket.findFirst({
    orderBy: { id: "desc" },
    select: { id: true },
  });
  const randomOffset = Math.floor(Math.random() * 1000);
  const nextId = ((lastTicket?.id || 0) * 10 + randomOffset + 1) % 900000 + 100000;
  return TKT-${year}-${nextId};
}
```

```
// Health Check
```

```
//
```

```
app.get("/api/health", (_req: Request, res: Response) => {  
  res.status(200).json({ status: "ok", service: "TokTickIT API" });  
});
```

```
//
```

```
// Lab 2 Reference Data REST API Endpoints – Feature 4
//
```

```
// GET /api/requesters - Active Development Requesters
app.get("/api/requesters", async (_req: Request, res: Response) => {
  try {
    const prisma = getPrisma();
```

```
const requesters = await prisma.requesterUser.findMany({
  where: { isActive: true },
  orderBy: { id: "asc" },
  select: { id: true, name: true, email: true, department: true },
});
res.status(200).json({ success: true, data: requesters });
} catch (error) {
  res.status(500).json({ success: false, error: { message: "Internal server error" } });
}
});

// GET /api/categories - Active Categories (Consistent payload format: { success: true, data
})
app.get("/api/categories", async (_req: Request, res: Response) => {
  try {
    const prisma = getPrisma();
    const categories = await prisma.category.findMany({
      where: { isActive: true },
      orderBy: { id: "asc" },
      select: { id: true, name: true },
    });
    res.status(200).json({ success: true, data: categories });
  } catch (error) {
    res.status(500).json({ success: false, error: { message: "Internal server error" } });
  }
});

// GET /api/related-systems - Active Related Systems (Consistent payload format: { success: tr
ue, data })
app.get("/api/related-systems", async (_req: Request, res: Response) => {
  try {
    const prisma = getPrisma();
    const systems = await prisma.relatedSystem.findMany({
      where: { isActive: true },
      orderBy: { id: "asc" },
      select: { id: true, name: true },
    });
    res.status(200).json({ success: true, data: systems });
  } catch (error) {
    res.status(500).json({ success: false, error: { message: "Internal server error" } });
  }
});

//
```

```
// Ticket Endpoints
//
```

```
//
```

```
// Ticket Creation API Endpoint – Feature 6  
//
```

```

// POST /api/tickets - Create Ticket with optional attachments
app.post(
  "/api/tickets",
  (req: Request, res: Response, next: NextFunction) => {
    upload.array("files", 5)(req, res, (err) => {
      if (err instanceof multer.MulterError) {
        if (err.code === "LIMIT_FILE_SIZE") {
          return res.status(400).json({
            success: false,
            error: { code: "FILE_TOO_LARGE", message: "File size exceeds maximum limit of 5MB"
          },
        });
      }
      return res.status(400).json({ success: false, error: { message: err.message } });
    } else if (err) {
      if (err.message === "UNSUPPORTED_FILE_TYPE") {
        return res.status(400).json({
          success: false,
          error: {
            code: "UNSUPPORTED_FILE_TYPE",
            message: "Allowed file types are JPG, PNG, WEBP, and PDF",
          },
        });
      }
      return res.status(400).json({ success: false, error: { message: err.message } });
    }
    next();
  });
},
async (req: Request, res: Response) => {
  try {
    const requesterId = getRequesterId(req);
    if (!requesterId) {
      return res.status(401).json({
        success: false,
        error: { code: "UNAUTHORIZED", message: "Requester identity missing or invalid" },
      });
    }

    const prisma = getPrisma();

```

```

// Verify active requester exists
const requester = await prisma.requesterUser.findFirst({
  where: { id: requesterId, isActive: true },
});
if (!requester) {
  return res.status(403).json({
    success: false,
    error: { code: "FORBIDDEN", message: "Inactive or invalid Development Requester" },
  });
}

const { categoryId, relatedSystemId, requestedPriority, summary, description } = req.body;

// Field validation
const errors: Record = {};
const catId = parseInt(categoryId, 10);
const sysId = parseInt(relatedSystemId, 10);

if (!catId || isNaN(catId)) errors.categoryId = ["Category is required"];
if (!sysId || isNaN(sysId)) errors.relatedSystemId = ["Related System is required"];
if (!summary || summary.trim().length < 5 || summary.trim().length > 150) {
  errors.summary = ["Summary must be between 5 and 150 characters"];
}
if (!description || description.trim().length < 10 || description.trim().length > 2000) {
  errors.description = ["Description must be between 10 and 2000 characters"];
}

const validPriorities = ["LOW", "MEDIUM", "HIGH", "URGENT"];
const priorityVal = (requestedPriority || "MEDIUM").toUpperCase();
if (!validPriorities.includes(priorityVal)) {
  errors.requestedPriority = ["Invalid requested priority"];
}

if (Object.keys(errors).length > 0) {
  return res.status(400).json({
    success: false,
    error: { code: "VALIDATION_ERROR", message: "Validation failed", details: errors },
  });
}

const ticketNo = await generateTicketNumber();

const files = (req.files as Express.Multer.File[]) || [];

const ticket = await prisma.ticket.create({
  data: {
    ticketNo,
    requesterId,
    categoryId: catId,
    relatedSystemId: sysId,
    requestedPriority: priorityVal as RequestedPriority,
    status: "NEW",
    summary: summary.trim(),
    description: description.trim(),
  },
});

```

```

        attachments: {
          create: files.map((f) => ({
            fileName: f.originalname,
            fileKey: f.filename,
            fileSize: f.size,
            mimeType: f.mimetype,
          })),
        },
      },
    include: {
      category: { select: { id: true, name: true } },
      relatedSystem: { select: { id: true, name: true } },
      attachments: true,
    },
  });

  res.status(201).json({ success: true, data: ticket });
} catch (error) {
  console.error("Create ticket error:", error);
  res.status(500).json({ success: false, error: { message: "Failed to create ticket" } });
}
}
);

// GET /api/tickets - List owned tickets (search, filter, sort, paginate)
app.get("/api/tickets", async (req: Request, res: Response) => {
  try {
    const requesterId = getRequesterId(req);
    if (!requesterId) {
      return res.status(401).json({
        success: false,
        error: { code: "UNAUTHORIZED", message: "Requester identity missing or invalid" },
      });
    }

    const prisma = getPrisma();
    const { search, category, priority, status, page = "1", limit = "10", sortBy = "createdAt", sortOrder = "desc" } = req.query;

    const pageNum = Math.max(1, parseInt(page as string, 10) || 1);
    const limitNum = Math.max(1, Math.min(50, parseInt(limit as string, 10) || 10));
    const skip = (pageNum - 1) * limitNum;

    const where: any = { requesterId };

    if (search && typeof search === "string" && search.trim() !== "") {
      const q = search.trim();
      where.OR = [
        { ticketNo: { contains: q, mode: "insensitive" } },
        { summary: { contains: q, mode: "insensitive" } },
      ];
    }

    if (category) {
      const catId = parseInt(category as string, 10);
      if (!isNaN(catId)) where.categoryId = catId;
    }
  }
});

```

```

}

if (priority && typeof priority === "string" && priority !== "ALL") {
  where.requestedPriority = priority.toUpperCase();
}

if (status && typeof status === "string" && status !== "ALL") {
  where.status = status.toUpperCase();
}

const orderField = ["createdAt", "ticketNo", "requestedPriority", "status"].includes(sortBy as string)
  ? (sortBy as string)
  : "createdAt";
const orderDirection = sortOrder === "asc" ? "asc" : "desc";

const [totalItems, tickets] = await Promise.all([
  prisma.ticket.count({ where }),
  prisma.ticket.findMany({
    where,
    skip,
    take: limitNum,
    orderBy: { [orderField]: orderDirection },
    include: {
      category: { select: { id: true, name: true } },
      relatedSystem: { select: { id: true, name: true } },
      attachments: {
        where: { isRemoved: false },
        select: { id: true },
      },
    },
  }),
]);

const data = tickets.map((t) => ({
  ...t,
  attachmentCount: t.attachments.length,
  attachments: undefined,
})));

const totalPages = Math.ceil(totalItems / limitNum) || 1;

res.status(200).json({
  success: true,
  data,
  meta: {
    page: pageNum,
    limit: limitNum,
    totalItems,
    totalPages,
  },
});
} catch (error) {
  console.error("List tickets error:", error);
  res.status(500).json({ success: false, error: { message: "Internal server error" } });
}

```

```

});

// GET /api/tickets/:id - Get owned ticket detail
app.get("/api/tickets/:id", async (req: Request, res: Response) => {
  try {
    const requesterId = getRequesterId(req);
    if (!requesterId) {
      return res.status(401).json({
        success: false,
        error: { code: "UNAUTHORIZED", message: "Requester identity missing or invalid" },
      });
    }

    const ticketId = parseInt(req.params.id, 10);
    if (isNaN(ticketId)) {
      return res.status(400).json({ success: false, error: { message: "Invalid ticket ID" } });
    }

    const prisma = getPrisma();
    const ticket = await prisma.ticket.findUnique({
      where: { id: ticketId },
      include: {
        requester: { select: { id: true, name: true, email: true, department: true } },
        category: { select: { id: true, name: true } },
        relatedSystem: { select: { id: true, name: true } },
        attachments: {
          orderBy: { createdAt: "desc" },
        },
      },
    });

    if (!ticket) {
      return res.status(404).json({ success: false, error: { message: "Ticket not found" } });
    }

    // Requester Ownership Isolation Check
    if (ticket.requesterId !== requesterId) {
      return res.status(403).json({
        success: false,
        error: { code: "FORBIDDEN", message: "Access denied: You do not own this ticket" },
      });
    }

    res.status(200).json({ success: true, data: ticket });
  } catch (error) {
    console.error("Get ticket error:", error);
    res.status(500).json({ success: false, error: { message: "Internal server error" } });
  }
});

//

```

```
// Attachment Endpoints
//
```

```

// POST /api/tickets/:id/attachments - Add attachment to existing ticket
app.post(
  "/api/tickets/:id/attachments",
  (req: Request, res: Response, next: NextFunction) => {
    upload.single("file")(req, res, (err) => {
      if (err instanceof multer.MulterError) {
        if (err.code === "LIMIT_FILE_SIZE") {
          return res.status(400).json({
            success: false,
            error: { code: "FILE_TOO_LARGE", message: "File size exceeds maximum limit of 5MB"
          },
        ));
      }
      return res.status(400).json({ success: false, error: { message: err.message } });
    } else if (err) {
      if (err.message === "UNSUPPORTED_FILE_TYPE") {
        return res.status(400).json({
          success: false,
          error: {
            code: "UNSUPPORTED_FILE_TYPE",
            message: "Allowed file types are JPG, PNG, WEBP, and PDF",
          },
        ));
      }
      return res.status(400).json({ success: false, error: { message: err.message } });
    }
    next();
  });
},
async (req: Request, res: Response) => {
  try {
    const requesterId = getRequesterId(req);
    if (!requesterId) {
      return res.status(401).json({
        success: false,
        error: { code: "UNAUTHORIZED", message: "Requester identity missing or invalid" },
      });
    }

    const ticketId = parseInt(req.params.id, 10);
    if (isNaN(ticketId)) {
      return res.status(400).json({ success: false, error: { message: "Invalid ticket ID" }
    });
  }

  const prisma = getPrisma();
  const ticket = await prisma.ticket.findUnique({
    where: { id: ticketId },
  });

```

```

        include: { attachments: { where: { isRemoved: false } } },
    });

    if (!ticket) {
        return res.status(404).json({ success: false, error: { message: "Ticket not found" }
    });
    }

    if (ticket.requesterId !== requesterId) {
        return res.status(403).json({
            success: false,
            error: { code: "FORBIDDEN", message: "Access denied: You do not own this ticket" },
        });
    }

    if (ticket.attachments.length >= 5) {
        return res.status(400).json({
            success: false,
            error: {
                code: "MAX_ATTACHMENTS_EXCEEDED",
                message: "Maximum limit of 5 active attachments per ticket reached",
            },
        });
    }

    if (!req.file) {
        return res.status(400).json({
            success: false,
            error: { message: "No file uploaded" },
        });
    }

    const attachment = await prisma.attachment.create({
        data: {
            ticketId,
            fileName: req.file.originalname,
            fileKey: req.file.filename,
            fileSize: req.file.size,
            mimeType: req.file.mimetype,
        },
    });

    res.status(201).json({ success: true, data: attachment });
} catch (error) {
    console.error("Upload attachment error:", error);
    res.status(500).json({ success: false, error: { message: "Failed to upload attachment" }
});
}
}
);

// DELETE /api/attachments/:id - Soft-remove attachment with reason
app.delete("/api/attachments/:id", async (req: Request, res: Response) => {
    try {
        const requesterId = getRequesterId(req);
        if (!requesterId) {

```

```

    return res.status(401).json({
      success: false,
      error: { code: "UNAUTHORIZED", message: "Requester identity missing or invalid" },
    });
  }

  const attachmentId = parseInt(req.params.id, 10);
  if (isNaN(attachmentId)) {
    return res.status(400).json({ success: false, error: { message: "Invalid attachment ID" } });
  }

  const { reason } = req.body;
  if (!reason || typeof reason !== "string" || reason.trim() === "") {
    return res.status(400).json({
      success: false,
      error: { code: "REASON_REQUIRED", message: "Removal reason is mandatory" },
    });
  }

  const prisma = getPrisma();
  const attachment = await prisma.attachment.findUnique({
    where: { id: attachmentId },
    include: { ticket: { select: { requesterId: true } } },
  });

  if (!attachment) {
    return res.status(404).json({ success: false, error: { message: "Attachment not found" } });
  }

  if (attachment.ticket.requesterId !== requesterId) {
    return res.status(403).json({
      success: false,
      error: { code: "FORBIDDEN", message: "Access denied: You do not own this attachment" },
    });
  }

  const updated = await prisma.attachment.update({
    where: { id: attachmentId },
    data: {
      isRemoved: true,
      removalReason: reason.trim(),
      removedAt: new Date(),
    },
  });

  res.status(200).json({ success: true, data: updated });
} catch (error) {
  console.error("Soft remove attachment error:", error);
  res.status(500).json({ success: false, error: { message: "Failed to remove attachment" } });
}
});

```

```

// GET /api/attachments/:id/download - Download active attachment
app.get("/api/attachments/:id/download", async (req: Request, res: Response) => {
  try {
    const requesterId = getRequesterId(req);
    if (!requesterId) {
      return res.status(401).json({
        success: false,
        error: { code: "UNAUTHORIZED", message: "Requester identity missing or invalid" },
      });
    }

    const attachmentId = parseInt(req.params.id, 10);
    if (isNaN(attachmentId)) {
      return res.status(400).json({ success: false, error: { message: "Invalid attachment ID" } });
    }

    const prisma = getPrisma();
    const attachment = await prisma.attachment.findUnique({
      where: { id: attachmentId },
      include: { ticket: { select: { requesterId: true } } },
    });

    if (!attachment) {
      return res.status(404).json({ success: false, error: { message: "Attachment not found" } });
    }

    if (attachment.ticket.requesterId !== requesterId) {
      return res.status(403).json({
        success: false,
        error: { code: "FORBIDDEN", message: "Access denied: You do not own this attachment" },
      });
    }

    if (attachment.isRemoved) {
      return res.status(410).json({
        success: false,
        error: {
          code: "ATTACHMENT_REMOVED",
          message: "This attachment has been soft-removed and cannot be downloaded",
        },
      });
    }

    const filePath = path.join(uploadDir, attachment.fileKey);
    if (!fs.existsSync(filePath)) {
      return res.status(404).json({ success: false, error: { message: "File missing on server storage" } });
    }

    res.setHeader("Content-Type", attachment.mimeType);
    res.setHeader("Content-Disposition", attachment; filename=`${attachment.fileName}`);
    fs.createReadStream(filePath).pipe(res);
  } catch (error) {

```

```
    console.error("Download attachment error:", error);
    res.status(500).json({ success: false, error: { message: "Failed to download attachment" }
  });
}
});

export default app;
```

5.2 Authentication Middleware (`server/src/middleware/authMiddleware.ts`)

```

import { Request, Response, NextFunction } from "express";
import jwt from "jsonwebtoken";
import { Role } from "@prisma/client";
import { getPrisma } from "../prisma.js";

export const JWT_SECRET = process.env.JWT_SECRET || "toktickit-super-secret-jwt-key-lab3";

export interface AuthUser {
  id: number;
  email: string;
  name: string;
  role: Role;
  isActive: boolean;
  mustChangePassword: boolean;
}

export interface AuthenticatedRequest extends Request {
  user?: AuthUser;
}

export async function extractUserFromRequest(req: Request): Promise {
  let token: string | null = null;

  // 1. Check Authorization Bearer header
  const authHeader = req.headers.authorization;
  if (authHeader && authHeader.startsWith("Bearer ")) {
    token = authHeader.substring(7);
  } else if (req.headers["x-auth-token"]) {
    // 2. Check x-auth-token fallback header
    token = req.headers["x-auth-token"] as string;
  }

  if (!token) return null;

  try {
    const decoded = jwt.verify(token, JWT_SECRET) as { id: number };
    const prisma = getPrisma();
    const user = await prisma.user.findUnique({
      where: { id: decoded.id },
      select: {
        id: true,
        email: true,
        name: true,
        role: true,
        isActive: true,
        mustChangePassword: true,
      },
    });

    if (!user || !user.isActive) return null;
    return user;
  } catch (err) {
    return null;
  }
}

```

```

}

export async function authenticateToken(
  req: AuthenticatedRequest,
  res: Response,
  next: NextFunction
): Promise {
  const user = await extractUserFromRequest(req);
  if (!user) {
    res.status(401).json({
      success: false,
      error: { message: "Unauthorized access or inactive account" },
    });
    return;
  }

  req.user = user;
  next();
}

export function requireRoles(...allowedRoles: Role[]) {
  return (req: AuthenticatedRequest, res: Response, next: NextFunction): void => {
    if (!req.user) {
      res.status(401).json({
        success: false,
        error: { message: "Authentication required" },
      });
      return;
    }

    if (!allowedRoles.includes(req.user.role)) {
      res.status(403).json({
        success: false,
        error: { message: "Forbidden: You do not have permission to access this resource" },
      });
      return;
    }

    next();
  };
}

export function enforcePasswordChange(
  req: AuthenticatedRequest,
  res: Response,
  next: NextFunction
): void {
  if (req.user && req.user.mustChangePassword && req.path !== "/change-password" && req.path !== "/api/auth/change-password") {
    res.status(403).json({
      success: false,
      error: {
        code: "MUST_CHANGE_PASSWORD",
        message: "You must change your initial password before accessing application features",
      },
    });
  }
}

```

```
});  
return;  
}  
next();  
}
```

5.3 Authentication Routes (`server/src/routes/auth.ts`)

```

import { Router, Response } from "express";
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import { getPrisma } from "../prisma.js";
import {
  authenticateToken,
  AuthenticatedRequest,
  JWT_SECRET,
} from "../middleware/authMiddleware.js";

export const authRouter = Router();

// POST /api/auth/login
authRouter.post("/login", async (req: AuthenticatedRequest, res: Response): Promise => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      res.status(400).json({
        success: false,
        error: { message: "Email and password are required" },
      });
      return;
    }

    const prisma = getPrisma();
    const user = await prisma.user.findUnique({
      where: { email: email.trim().toLowerCase() },
    });

    if (!user) {
      res.status(401).json({
        success: false,
        error: { message: "Invalid email or password" },
      });
      return;
    }

    if (!user.isActive) {
      res.status(401).json({
        success: false,
        error: { message: "Account is disabled. Please contact Administrator." },
      });
      return;
    }

    const isMatch = await bcrypt.compare(password, user.passwordHash);
    if (!isMatch) {
      res.status(401).json({
        success: false,
        error: { message: "Invalid email or password" },
      });
      return;
    }
  }

```

```

const token = jwt.sign({ id: user.id, role: user.role }, JWT_SECRET, {
  expiresIn: "24h",
});

res.status(200).json({
  success: true,
  token,
  user: {
    id: user.id,
    email: user.email,
    name: user.name,
    role: user.role,
    mustChangePassword: user.mustChangePassword,
  },
});
} catch (error) {
  res.status(500).json({
    success: false,
    error: { message: "Internal server error" },
  });
}
});

// POST /api/auth/logout
authRouter.post("/logout", authenticateToken, (_req: AuthenticatedRequest, res: Response) => {
  res.status(200).json({
    success: true,
    message: "Logged out successfully",
  });
});

// GET /api/auth/me
authRouter.get("/me", authenticateToken, (req: AuthenticatedRequest, res: Response) => {
  res.status(200).json({
    success: true,
    user: req.user,
  });
});

// POST /api/auth/change-password
authRouter.post("/change-password", authenticateToken, async (req: AuthenticatedRequest, res:
Response): Promise => {
  try {
    const { currentPassword, newPassword } = req.body;

    if (!currentPassword || !newPassword) {
      res.status(400).json({
        success: false,
        error: { message: "Current password and new password are required" },
      });
      return;
    }

    if (newPassword.length < 8) {
      res.status(400).json({

```

```

        success: false,
        error: { message: "New password must be at least 8 characters long" },
    });
    return;
}

const prisma = getPrisma();
const user = await prisma.user.findUnique({
    where: { id: req.user!.id },
});

if (!user) {
    res.status(404).json({
        success: false,
        error: { message: "User not found" },
    });
    return;
}

const isMatch = await bcrypt.compare(currentPassword, user.passwordHash);
if (!isMatch) {
    res.status(400).json({
        success: false,
        error: { message: "Current password is incorrect" },
    });
    return;
}

const newPasswordHash = await bcrypt.hash(newPassword, 10);
const updatedUser = await prisma.user.update({
    where: { id: user.id },
    data: {
        passwordHash: newPasswordHash,
        mustChangePassword: false,
    },
    select: {
        id: true,
        email: true,
        name: true,
        role: true,
        isActive: true,
        mustChangePassword: true,
    },
});

res.status(200).json({
    success: true,
    message: "Password changed successfully",
    user: updatedUser,
});
} catch (error) {
    res.status(500).json({
        success: false,
        error: { message: "Internal server error" },
    });
}

```

```
});
```

5.4 Requester Ticket Routes (`server/src/routes/requesterTickets.ts`)

```
import { Router, Response } from "express";
import { getPrisma } from "../prisma.js";
import {
  authenticateToken,
  requireRoles,
  enforcePasswordChange,
  AuthenticatedRequest,
} from "../middleware/authMiddleware.js";

export const requesterTicketsRouter = Router();

// Sequence generator helper for Ticket Numbers: TKT-2026-XXXXXX
async function generateTicketNumber(): Promise {
  const prisma = getPrisma();
  const year = new Date().getFullYear();
  const lastTicket = await prisma.ticket.findFirst({
    orderBy: { id: "desc" },
    select: { id: true },
  });
  const randomOffset = Math.floor(Math.random() * 1000);
  const nextId = ((lastTicket?.id || 0) * 10 + randomOffset + 1) % 900000 + 100000;
  return TKT-${year}-${nextId};
}

//
```

```
// 1. GET /api/requesters/tickets
//
```

```
requesterTicketsRouter.get(
  "/requesters/tickets",
  authenticateToken,
  requireRoles("REQUESTER"),
  enforcePasswordChange,
  async (req: AuthenticatedRequest, res: Response): Promise => {
    try {
      const prisma = getPrisma();
      const requesterId = req.user!.id;

      const page = parseInt(req.query.page as string, 10) || 1;
      const limit = parseInt(req.query.limit as string, 10) || 10;
      const skip = (page - 1) * limit;

      const search = (req.query.search as string) || "";
      const category = (req.query.category as string) || "";
      const priority = (req.query.priority as string) || "";
```

```

const status = (req.query.status as string) || "";
const sortBy = (req.query.sortBy as string) || "createdAt";
const sortOrder = (req.query.sortOrder as string)?.toLowerCase() === "asc" ? "asc" : "desc";

const whereClause: any = {
  requesterId,
};

if (search.trim()) {
  whereClause.OR = [
    { ticketNo: { contains: search.trim(), mode: "insensitive" } },
    { summary: { contains: search.trim(), mode: "insensitive" } },
    { description: { contains: search.trim(), mode: "insensitive" } },
  ];
}

if (category && category !== "ALL") {
  const catId = parseInt(category, 10);
  if (!isNaN(catId)) {
    whereClause.categoryId = catId;
  } else {
    whereClause.category = { name: { equals: category, mode: "insensitive" } };
  }
}

if (priority && priority !== "ALL") {
  whereClause.requestedPriority = priority;
}

if (status && status !== "ALL") {
  whereClause.status = status;
}

const totalItems = await prisma.ticket.count({ where: whereClause });
const tickets = await prisma.ticket.findMany({
  where: whereClause,
  skip,
  take: limit,
  orderBy: { [sortBy]: sortOrder },
  include: {
    category: true,
    relatedSystem: true,
    attachments: true,
  },
});

res.status(200).json({
  success: true,
  data: tickets,
  pagination: {
    totalItems,
    totalPages: Math.ceil(totalItems / limit) || 1,
    currentPage: page,
    pageSize: limit,
  },
});

```

```
});  
} catch (error) {  
  console.error("Fetch Requester Tickets Error:", error);  
  res.status(500).json({  
    success: false,  
    error: { message: "Failed to fetch tickets" },  
  });  
}  
}  
);  
  
//
```

```
// 2. POST /api/requesters/tickets
//
```

```
requesterTicketsRouter.post(
  "/requesters/tickets",
  authenticateToken,
  requireRoles("REQUESTER"),
  enforcePasswordChange,
  async (req: AuthenticatedRequest, res: Response): Promise => {
    try {
      const prisma = getPrisma();
      const requesterId = req.user!.id;
      const { categoryId, relatedSystemId, requestedPriority, summary, description } = req.body;

      if (!summary || typeof summary !== "string" || summary.trim().length < 5) {
        res.status(400).json({
          success: false,
          error: { message: "Summary must be at least 5 characters long" },
        });
        return;
      }

      if (!description || typeof description !== "string" || description.trim().length < 10) {
        res.status(400).json({
          success: false,
          error: { message: "Description must be at least 10 characters long" },
        });
        return;
      }

      const catId = parseInt(categoryId, 10);
      const sysId = parseInt(relatedSystemId, 10);

      if (isNaN(catId) || isNaN(sysId)) {
```



```
// 3. GET /api/tickets/:id/comments (Public Comments)
//
```

```
requesterTicketsRouter.get(
  "/tickets/:id/comments",
  authenticateToken,
```

```

enforcePasswordChange,
async (req: AuthenticatedRequest, res: Response): Promise => {
  try {
    const prisma = getPrisma();
    const ticketId = parseInt(req.params.id, 10);

    if (isNaN(ticketId)) {
      res.status(400).json({
        success: false,
        error: { message: "Invalid ticket ID" },
      });
      return;
    }

    const ticket = await prisma.ticket.findUnique({
      where: { id: ticketId },
    });

    if (!ticket) {
      res.status(404).json({
        success: false,
        error: { message: "Ticket not found" },
      });
      return;
    }

    // Authorization check for Requesters
    if (req.user!.role === "REQUESTER" && ticket.requesterId !== req.user!.id) {
      res.status(403).json({
        success: false,
        error: { message: "Forbidden: You can only view comments on your own tickets" },
      });
      return;
    }

    const comments = await prisma.publicComment.findMany({
      where: { ticketId },
      orderBy: { createdAt: "asc" },
      include: {
        author: {
          select: { id: true, name: true, email: true, role: true },
        },
      },
    });

    res.status(200).json({
      success: true,
      data: comments,
    });
  } catch (error) {
    console.error("Fetch Public Comments Error:", error);
    res.status(500).json({
      success: false,
      error: { message: "Failed to fetch public comments" },
    });
  }
}

```

```
}  
);  
  
//
```

```
// 4. POST /api/tickets/:id/comments (Add Public Comment)  
//
```

```

requesterTicketsRouter.post(
  "/tickets/:id/comments",
  authenticateToken,
  enforcePasswordChange,
  async (req: AuthenticatedRequest, res: Response): Promise => {
    try {
      const prisma = getPrisma();
      const ticketId = parseInt(req.params.id, 10);
      const { content } = req.body;

      if (isNaN(ticketId)) {
        res.status(400).json({
          success: false,
          error: { message: "Invalid ticket ID" },
        });
        return;
      }

      if (!content || typeof content !== "string" || content.trim().length === 0) {
        res.status(400).json({
          success: false,
          error: { message: "Comment content cannot be empty" },
        });
        return;
      }

      const ticket = await prisma.ticket.findUnique({
        where: { id: ticketId },
      });

      if (!ticket) {
        res.status(404).json({
          success: false,
          error: { message: "Ticket not found" },
        });
        return;
      }

      // Authorization check for Requesters
      if (req.user!.role === "REQUESTER" && ticket.requesterId !== req.user!.id) {

```

```

    res.status(403).json({
      success: false,
      error: { message: "Forbidden: You can only comment on your own tickets" },
    });
    return;
  }

  const newComment = await prisma.publicComment.create({
    data: {
      ticketId,
      authorId: req.user!.id,
      content: content.trim(),
    },
    include: {
      author: {
        select: { id: true, name: true, email: true, role: true },
      },
    },
  });

  res.status(201).json({
    success: true,
    data: newComment,
  });
} catch (error) {
  console.error("Create Public Comment Error:", error);
  res.status(500).json({
    success: false,
    error: { message: "Failed to create public comment" },
  });
}
}
);

```

5.5 IT Staff Ticket Queue & Workflow Routes (`server/src/routes/staffTickets.ts`)

```
import { Router, Response } from "express";
import { getPrisma } from "../prisma.js";
import {
  authenticateToken,
  requireRoles,
  enforcePasswordChange,
  AuthenticatedRequest,
} from "../middleware/authMiddleware.js";
```

```
export const staffTicketsRouter = Router();
```

```
//
```

```
// 1. GET /api/staff/tickets (IT Staff Ticket Queue)
```

```
//
```

```

staffTicketsRouter.get(
  "/staff/tickets",
  authenticateToken,
  requireRoles("IT_STAFF", "ADMINISTRATOR"),
  enforcePasswordChange,
  async (req: AuthenticatedRequest, res: Response): Promise => {
    try {
      const prisma = getPrisma();
      const page = parseInt(req.query.page as string, 10) || 1;
      const limit = parseInt(req.query.limit as string, 10) || 10;
      const skip = (page - 1) * limit;

      const search = (req.query.search as string) || "";
      const category = (req.query.category as string) || "";
      const priority = (req.query.priority as string) || "";
      const status = (req.query.status as string) || "";
      const sortBy = (req.query.sortBy as string) || "createdAt";
      const sortOrder = (req.query.sortOrder as string)?.toLowerCase() === "asc" ? "asc" : "desc";

      const myQueue = req.query.myQueue === "true";

      const whereClause: any = {};

      if (myQueue) {
        whereClause.ownerId = req.user!.id;
      }

      if (search.trim()) {
        whereClause.OR = [
          { ticketNo: { contains: search.trim(), mode: "insensitive" } },

```

```

        { summary: { contains: search.trim(), mode: "insensitive" } },
        { description: { contains: search.trim(), mode: "insensitive" } },
    ];
}

if (category && category !== "ALL") {
    const catId = parseInt(category, 10);
    if (!isNaN(catId)) {
        whereClause.categoryId = catId;
    } else {
        whereClause.category = { name: { equals: category, mode: "insensitive" } };
    }
}

if (priority && priority !== "ALL") {
    whereClause.OR = [
        { requestedPriority: priority },
        { itPriority: priority },
    ];
}

if (status && status !== "ALL") {
    whereClause.status = status;
}

const totalItems = await prisma.ticket.count({ where: whereClause });
const tickets = await prisma.ticket.findMany({
    where: whereClause,
    skip,
    take: limit,
    orderBy: { [sortBy]: sortOrder },
    include: {
        requester: { select: { id: true, name: true, email: true, department: true } },
        owner: { select: { id: true, name: true, email: true, role: true } },
        category: true,
        relatedSystem: true,
        attachments: true,
    },
});

res.status(200).json({
    success: true,
    data: tickets,
    pagination: {
        totalItems,
        totalPages: Math.ceil(totalItems / limit) || 1,
        currentPage: page,
        pageSize: limit,
    },
});
} catch (error) {
    console.error("Fetch Staff Tickets Queue Error:", error);
    res.status(500).json({
        success: false,
        error: { message: "Failed to fetch IT staff ticket queue" },
    });
}

```

```
}  
}  
);  
  
//
```

```
// 2. GET /api/staff/tickets/:id (Single Ticket Detail for IT Staff)  
//
```

```

staffTicketsRouter.get(
  "/staff/tickets/:id",
  authenticateToken,
  requireRoles("IT_STAFF", "ADMINISTRATOR"),
  enforcePasswordChange,
  async (req: AuthenticatedRequest, res: Response): Promise => {
    try {
      const prisma = getPrisma();
      const ticketId = parseInt(req.params.id, 10);

      if (isNaN(ticketId)) {
        res.status(400).json({
          success: false,
          error: { message: "Invalid ticket ID" },
        });
        return;
      }

      const ticket = await prisma.ticket.findUnique({
        where: { id: ticketId },
        include: {
          requester: { select: { id: true, name: true, email: true, department: true } },
          owner: { select: { id: true, name: true, email: true, role: true } },
          category: true,
          relatedSystem: true,
          attachments: true,
          publicComments: {
            include: { author: { select: { id: true, name: true, email: true, role: true } }
          },
          orderBy: { createdAt: "asc" },
        },
        internalNotes: {
          include: { author: { select: { id: true, name: true, email: true, role: true } }
        },
        orderBy: { createdAt: "asc" },
      },
    },
  });
}

```

```
if (!ticket) {
  res.status(404).json({
    success: false,
    error: { message: "Ticket not found" },
  });
  return;
}

res.status(200).json({
  success: true,
  data: ticket,
});
} catch (error) {
  console.error("Fetch Staff Ticket Detail Error:", error);
  res.status(500).json({
    success: false,
    error: { message: "Failed to fetch ticket detail" },
  });
}
}
);

//
```

```
// 3. PATCH /api/staff/tickets/:id/assign (Claim or Reassign Ticket Owner)
//
```

```
staffTicketsRouter.patch(
  "/staff/tickets/:id/assign",
  authenticateToken,
  requireRoles("IT_STAFF", "ADMINISTRATOR"),
  enforcePasswordChange,
  async (req: AuthenticatedRequest, res: Response): Promise => {
    try {
      const prisma = getPrisma();
      const ticketId = parseInt(req.params.id, 10);

      if (isNaN(ticketId)) {
        res.status(400).json({
          success: false,
          error: { message: "Invalid ticket ID" },
        });
        return;
      }

      const targetOwnerId = req.body.ownerId !== undefined ? req.body.ownerId : req.user!.id;

      if (targetOwnerId !== null) {
```

```

const ownerUser = await prisma.user.findUnique({
  where: { id: targetOwnerId },
});

if (!ownerUser || !ownerUser.isActive || ownerUser.role === "REQUESTER") {
  res.status(400).json({
    success: false,
    error: { message: "Owner ID must belong to an active IT Staff or Administrator account" },
  });
  return;
}

const updatedTicket = await prisma.ticket.update({
  where: { id: ticketId },
  data: {
    ownerId: targetOwnerId,
    status: "IN_PROGRESS",
  },
  include: {
    owner: { select: { id: true, name: true, email: true, role: true } },
  },
});

res.status(200).json({
  success: true,
  data: updatedTicket,
});
} catch (error: any) {
  if (error.code === "P2025") {
    res.status(404).json({
      success: false,
      error: { message: "Ticket not found" },
    });
    return;
  }
  console.error("Assign Ticket Error:", error);
  res.status(500).json({
    success: false,
    error: { message: "Failed to assign ticket" },
  });
}
}
);

//

```

```
// 4. PATCH /api/staff/tickets/:id/workflow (Update IT Priority & Ticket Status)
//
```

```

staffTicketsRouter.patch(
  "/staff/tickets/:id/workflow",
  authenticateToken,
  requireRoles("IT_STAFF", "ADMINISTRATOR"),
  enforcePasswordChange,
  async (req: AuthenticatedRequest, res: Response): Promise => {
    try {
      const prisma = getPrisma();
      const ticketId = parseInt(req.params.id, 10);
      const { itPriority, status } = req.body;

      if (isNaN(ticketId)) {
        res.status(400).json({
          success: false,
          error: { message: "Invalid ticket ID" },
        });
        return;
      }

      const updateData: any = {};
      if (itPriority) updateData.itPriority = itPriority;
      if (status) updateData.status = status;

      if (Object.keys(updateData).length === 0) {
        res.status(400).json({
          success: false,
          error: { message: "At least one field (itPriority or status) must be provided" },
        });
        return;
      }

      const updatedTicket = await prisma.ticket.update({
        where: { id: ticketId },
        data: updateData,
        include: {
          category: true,
          relatedSystem: true,
          owner: { select: { id: true, name: true, email: true, role: true } },
        },
      });

      res.status(200).json({
        success: true,
        data: updatedTicket,
      });
    } catch (error: any) {
      if (error.code === "P2025") {
        res.status(404).json({
          success: false,
          error: { message: "Ticket not found" },
        });
        return;
      }
    }
    console.error("Update Ticket Workflow Error:", error);
  }
);

```

```
res.status(500).json({
  success: false,
  error: { message: "Failed to update ticket workflow" },
});
}
}
);
//
```

```
// 5. GET /api/tickets/:id/notes (List Internal Notes - IT Staff/Admin ONLY)
//
```

```
staffTicketsRouter.get(
  "/tickets/:id/notes",
  authenticateToken,
  requireRoles("IT_STAFF", "ADMINISTRATOR"),
  enforcePasswordChange,
  async (req: AuthenticatedRequest, res: Response): Promise => {
    try {
      const prisma = getPrisma();
      const ticketId = parseInt(req.params.id, 10);

      if (isNaN(ticketId)) {
        res.status(400).json({
          success: false,
          error: { message: "Invalid ticket ID" },
        });
        return;
      }

      const ticket = await prisma.ticket.findUnique({
        where: { id: ticketId },
      });

      if (!ticket) {
        res.status(404).json({
          success: false,
          error: { message: "Ticket not found" },
        });
        return;
      }

      const notes = await prisma.internalNote.findMany({
        where: { ticketId },
        orderBy: { createdAt: "asc" },
        include: {
          author: { select: { id: true, name: true, email: true, role: true } },
        },
      });
```

```
    },
  });

  res.status(200).json({
    success: true,
    data: notes,
  });
} catch (error) {
  console.error("Fetch Internal Notes Error:", error);
  res.status(500).json({
    success: false,
    error: { message: "Failed to fetch internal notes" },
  });
}
}
);

//
```

```
// 6. POST /api/tickets/:id/notes (Create Internal Note - IT Staff/Admin ONLY)
//
```

```
staffTicketsRouter.post(
  "/tickets/:id/notes",
  authenticateToken,
  requireRoles("IT_STAFF", "ADMINISTRATOR"),
  enforcePasswordChange,
  async (req: AuthenticatedRequest, res: Response): Promise => {
    try {
      const prisma = getPrisma();
      const ticketId = parseInt(req.params.id, 10);
      const { content } = req.body;

      if (isNaN(ticketId)) {
        res.status(400).json({
          success: false,
          error: { message: "Invalid ticket ID" },
        });
        return;
      }

      if (!content || typeof content !== "string" || content.trim().length === 0) {
        res.status(400).json({
          success: false,
          error: { message: "Internal note content cannot be empty" },
        });
        return;
      }
    }
  }
}
```

```

const ticket = await prisma.ticket.findUnique({
  where: { id: ticketId },
});

if (!ticket) {
  res.status(404).json({
    success: false,
    error: { message: "Ticket not found" },
  });
  return;
}

const newNote = await prisma.internalNote.create({
  data: {
    ticketId,
    authorId: req.user!.id,
    content: content.trim(),
  },
  include: {
    author: { select: { id: true, name: true, email: true, role: true } },
  },
});

res.status(201).json({
  success: true,
  data: newNote,
});
} catch (error) {
  console.error("Create Internal Note Error:", error);
  res.status(500).json({
    success: false,
    error: { message: "Failed to create internal note" },
  });
}
}
);

```

5.6 Administrator User Management Routes (`server/src/routes/adminUsers.ts`)

```
import { Router, Response } from "express";
import bcrypt from "bcryptjs";
import { getPrisma } from "../prisma.js";
import {
  authenticateToken,
  requireRoles,
  enforcePasswordChange,
  AuthenticatedRequest,
} from "../middleware/authMiddleware.js";

export const adminUsersRouter = Router();
```

```
//
```

```
// 1. GET /api/admin/users (List users with search & filters - Admin ONLY)
//
```

```

adminUsersRouter.get(
  "/admin/users",
  authenticateToken,
  requireRoles("ADMINISTRATOR"),
  enforcePasswordChange,
  async (req: AuthenticatedRequest, res: Response): Promise => {
    try {
      const prisma = getPrisma();
      const page = parseInt(req.query.page as string, 10) || 1;
      const limit = parseInt(req.query.limit as string, 10) || 10;
      const skip = (page - 1) * limit;

      const search = (req.query.search as string) || "";
      const roleFilter = (req.query.role as string) || "";
      const isActiveFilter = req.query.isActive as string;
      const sortBy = (req.query.sortBy as string) || "id";
      const sortOrder = (req.query.sortOrder as string)?.toLowerCase() === "desc" ? "desc" :
"asc";

      const whereClause: any = {};

      if (search.trim()) {
        whereClause.OR = [
          { name: { contains: search.trim(), mode: "insensitive" } },
          { email: { contains: search.trim(), mode: "insensitive" } },
          { department: { contains: search.trim(), mode: "insensitive" } },
        ];
      }

      if (roleFilter && roleFilter !== "ALL") {

```

```
    whereClause.role = roleFilter;
  }

  if (isActiveFilter !== undefined && isActiveFilter !== "ALL") {
    whereClause.isActive = isActiveFilter === "true";
  }

  const totalItems = await prisma.user.count({ where: whereClause });
  const users = await prisma.user.findMany({
    where: whereClause,
    skip,
    take: limit,
    orderBy: { [sortBy]: sortOrder },
    select: {
      id: true,
      email: true,
      name: true,
      department: true,
      role: true,
      isActive: true,
      mustChangePassword: true,
      createdAt: true,
      updatedAt: true,
    },
  });

  res.status(200).json({
    success: true,
    data: users,
    pagination: {
      totalItems,
      totalPages: Math.ceil(totalItems / limit) || 1,
      currentPage: page,
      pageSize: limit,
    },
  });
} catch (error) {
  console.error("Fetch Admin Users Error:", error);
  res.status(500).json({
    success: false,
    error: { message: "Failed to fetch users" },
  });
}
}
);

//
```

```
// 2. POST /api/admin/users (Create user with initial password - Admin ONLY)
//
```

```

adminUsersRouter.post(
  "/admin/users",
  authenticateToken,
  requireRoles("ADMINISTRATOR"),
  enforcePasswordChange,
  async (req: AuthenticatedRequest, res: Response): Promise => {
    try {
      const prisma = getPrisma();
      const { email, name, department, role, isActive, initialPassword } = req.body;

      if (!email || typeof email !== "string" || !email.includes("@")) {
        res.status(400).json({
          success: false,
          error: { message: "Valid email address is required" },
        });
        return;
      }

      if (!name || typeof name !== "string" || name.trim().length === 0) {
        res.status(400).json({
          success: false,
          error: { message: "Full Name is required" },
        });
        return;
      }

      const normalizedEmail = email.trim().toLowerCase();
      const existingUser = await prisma.user.findUnique({
        where: { email: normalizedEmail },
      });

      if (existingUser) {
        res.status(409).json({
          success: false,
          error: { message: "A user with this email address already exists" },
        });
        return;
      }

      const tempPassword = initialPassword || "InitialPassword123!";
      const passwordHash = await bcrypt.hash(tempPassword, 10);

      const newUser = await prisma.user.create({
        data: {
          email: normalizedEmail,
          name: name.trim(),
          department: department ? department.trim() : null,
          role: role || "REQUESTER",
          isActive: isActive !== undefined ? Boolean(isActive) : true,
          passwordHash,
          mustChangePassword: true,
        },
        select: {
          id: true,

```

```
        email: true,
        name: true,
        department: true,
        role: true,
        isActive: true,
        mustChangePassword: true,
        createdAt: true,
        updatedAt: true,
    },
    });

    res.status(201).json({
        success: true,
        data: newUser,
    });
} catch (error) {
    console.error("Create User Error:", error);
    res.status(500).json({
        success: false,
        error: { message: "Failed to create user" },
    });
}
}
);

//
```

```
// 3. PATCH /api/admin/users/:id (Edit user profile or activation status)
//
```

```
adminUsersRouter.patch(
  "/admin/users/:id",
  authenticateToken,
  requireRoles("ADMINISTRATOR"),
  enforcePasswordChange,
  async (req: AuthenticatedRequest, res: Response): Promise => {
    try {
      const prisma = getPrisma();
      const targetUserId = parseInt(req.params.id, 10);

      if (isNaN(targetUserId)) {
        res.status(400).json({
          success: false,
          error: { message: "Invalid user ID" },
        });
        return;
      }
    }
  }
}
```

```

const targetUser = await prisma.user.findUnique({
  where: { id: targetUserId },
});

if (!targetUser) {
  res.status(404).json({
    success: false,
    error: { message: "User not found" },
  });
  return;
}

const { name, email, department, role, isActive } = req.body;

// Prevent self-deactivation
if (req.user!.id === targetUserId && isActive === false) {
  res.status(400).json({
    success: false,
    error: { message: "Self-deactivation of Administrator account is prohibited" },
  });
  return;
}

// Prevent deactivating or downgrading the last active Administrator account
if (targetUser.role === "ADMINISTRATOR" && (isActive === false || (role && role !== "ADMINISTRATOR"))) {
  const activeAdminCount = await prisma.user.count({
    where: { role: "ADMINISTRATOR", isActive: true },
  });

  if (activeAdminCount <= 1) {
    res.status(400).json({
      success: false,
      error: { message: "Cannot deactivate or downgrade the last active Administrator account" },
    });
    return;
  }
}

const updateData: any = {};
if (name !== undefined) updateData.name = name.trim();
if (department !== undefined) updateData.department = department ? department.trim() : null;
if (role !== undefined) updateData.role = role;
if (isActive !== undefined) updateData.isActive = Boolean(isActive);

if (email !== undefined && email.trim().toLowerCase() !== targetUser.email) {
  const normalizedEmail = email.trim().toLowerCase();
  const conflictUser = await prisma.user.findUnique({ where: { email: normalizedEmail } });

  if (conflictUser) {
    res.status(409).json({
      success: false,
      error: { message: "Email address is already in use by another account" },
    });
  }
}

```

```
        return;
    }
    updateData.email = normalizedEmail;
}

const updatedUser = await prisma.user.update({
  where: { id: targetUserId },
  data: updateData,
  select: {
    id: true,
    email: true,
    name: true,
    department: true,
    role: true,
    isActive: true,
    mustChangePassword: true,
    createdAt: true,
    updatedAt: true,
  },
});

res.status(200).json({
  success: true,
  data: updatedUser,
});
} catch (error) {
  console.error("Update User Error:", error);
  res.status(500).json({
    success: false,
    error: { message: "Failed to update user" },
  });
}
}
);

//
```

```
// 4. POST /api/admin/users/:id/reset-password (Reset initial password)
//
```

```
adminUsersRouter.post(
  "/admin/users/:id/reset-password",
  authenticateToken,
  requireRoles("ADMINISTRATOR"),
  enforcePasswordChange,
  async (req: AuthenticatedRequest, res: Response): Promise => {
    try {
      const prisma = getPrisma();
```

```
const targetUserId = parseInt(req.params.id, 10);
const { newInitialPassword } = req.body;

if (isNaN(targetUserId)) {
  res.status(400).json({
    success: false,
    error: { message: "Invalid user ID" },
  });
  return;
}

const targetUser = await prisma.user.findUnique({
  where: { id: targetUserId },
});

if (!targetUser) {
  res.status(404).json({
    success: false,
    error: { message: "User not found" },
  });
  return;
}

const tempPassword = newInitialPassword || "InitialPassword123!";
const passwordHash = await bcrypt.hash(tempPassword, 10);

const updatedUser = await prisma.user.update({
  where: { id: targetUserId },
  data: {
    passwordHash,
    mustChangePassword: true,
  },
  select: {
    id: true,
    email: true,
    name: true,
    role: true,
    mustChangePassword: true,
  },
});

res.status(200).json({
  success: true,
  message: "Password reset successfully",
  data: updatedUser,
});
} catch (error) {
  console.error("Reset Password Error:", error);
  res.status(500).json({
    success: false,
    error: { message: "Failed to reset password" },
  });
}
}
```

);

Answer Part 6: Client Frontend Source Code Evidence

6.1 Authentication Context (`client/src/contexts/AuthContext.tsx`)

```

import React, { createContext, useContext, useState, useEffect } from "react";
import { User } from "../types";
import { loginApi, changePasswordApi, getMeApi, logoutApi } from "../api";

interface AuthContextType {
  user: User | null;
  token: string | null;
  loading: boolean;
  login: (email: string, password: string) => Promise;
  logout: () => void;
  changePassword: (currentPassword: string, newPassword: string) => Promise;
  setUser: React.Dispatch<>;
}

const AuthContext = createContext(undefined);

export const AuthProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(localStorage.getItem("toktickit_token"));
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const initAuth = async () => {
      const savedToken = localStorage.getItem("toktickit_token");
      if (savedToken) {
        try {
          const userData = await getMeApi(savedToken);
          setUser(userData);
          setToken(savedToken);
        } catch (e) {
          localStorage.removeItem("toktickit_token");
          localStorage.removeItem("toktickit_user");
          setToken(null);
          setUser(null);
        }
      }
      setLoading(false);
    };

    initAuth();
  }, []);

  const login = async (email: string, password: string): Promise => {
    const data = await loginApi(email, password);
    setToken(data.token);
    setUser(data.user);
    localStorage.setItem("toktickit_token", data.token);
    localStorage.setItem("toktickit_user", JSON.stringify(data.user));
    return data.user;
  };

  const logout = () => {
    if (token) {
      logoutApi(token);
    }
  };

```

```

    }
    setToken(null);
    setUser(null);
    localStorage.removeItem("toktickit_token");
    localStorage.removeItem("toktickit_user");
  };

  const changePassword = async (currentPassword: string, newPassword: string) => {
    if (!token) throw new Error("Not authenticated");
    await changePasswordApi(token, currentPassword, newPassword);
    if (user) {
      const updatedUser = { ...user, mustChangePassword: false };
      setUser(updatedUser);
      localStorage.setItem("toktickit_user", JSON.stringify(updatedUser));
    }
  };

  return (
    {children}
  );
};

export const useAuth = (): AuthContextType => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error("useAuth must be used within an AuthProvider");
  }
  return context;
};

```

6.2 Client API Client Helper (`client/src/api.ts`)

```

import {
  RequesterUser,
  Category,
  RelatedSystem,
  Ticket,
  Attachment,
  TicketListResponse,
  User,
  AuthResponse,
} from "../types";

const API_URL = import.meta.env.VITE_API_URL ?? "http://localhost:3000";

// Re-export Category interface for backwards compatibility with Lab 1
export type { Category };

export interface SystemStatus {
  online: boolean;
  categories: Category[];
}

export async function checkSystem(): Promise {
  const healthRes = await fetch(`${API_URL}/api/health`).catch(() => {
    throw new Error("Unable to connect to TokTickIT API");
  });

  if (!healthRes.ok) {
    throw new Error("Unable to connect to TokTickIT API");
  }

  const catRes = await fetch(`${API_URL}/api/categories`).catch(() => {
    throw new Error("Unable to connect to TokTickIT API");
  });

  if (!catRes.ok) {
    throw new Error("Unable to connect to TokTickIT API");
  }

  const rawData = await catRes.json();
  const categories: Category[] = Array.isArray(rawData) ? rawData : rawData.data;
  return { online: true, categories };
}

//

```

```
// Auth API Client Functions
//
```

```

export async function loginApi(email: string, password: string): Promise {
  const res = await fetch(`${API_URL}/api/auth/login`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password }),
  });

  const json = await res.json();
  if (!res.ok) {
    throw new Error(json.error?.message || "Invalid credentials or deactivated account");
  }
  return json;
}

export async function changePasswordApi(
  token: string,
  currentPassword: string,
  newPassword: string
): Promise<{ success: boolean; message: string }> {
  const res = await fetch(`${API_URL}/api/auth/change-password`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: Bearer ${token},
    },
    body: JSON.stringify({ currentPassword, newPassword }),
  });

  const json = await res.json();
  if (!res.ok) {
    throw new Error(json.error?.message || "Failed to change password");
  }
  return json;
}

export async function getMeApi(token: string): Promise {
  const res = await fetch(`${API_URL}/api/auth/me`, {
    headers: {
      Authorization: Bearer ${token},
    },
  });

  const json = await res.json();
  if (!res.ok) {
    throw new Error(json.error?.message || "Failed to fetch user session");
  }
  return json.user;
}

export async function logoutApi(token: string): Promise {
  await fetch(`${API_URL}/api/auth/logout`, {
    method: "POST",
    headers: {
      Authorization: Bearer ${token},
    },
  });
}

```

```
    },  
    }).catch(() => {});  
}
```

```
//
```

```
// Lab 2 API Client Functions  
//
```

```
export async function fetchRequesters(): Promise {
  const res = await fetch(`${API_URL}/api/requesters`);
  if (!res.ok) throw new Error("Failed to load Development Requesters");
  const json = await res.json();
  return json.data;
}

export async function fetchCategories(): Promise {
  const res = await fetch(`${API_URL}/api/categories`);
  if (!res.ok) throw new Error("Failed to load Ticket Categories");
  const json = await res.json();
  return Array.isArray(json) ? json : json.data;
}

export async function fetchRelatedSystems(): Promise {
  const res = await fetch(`${API_URL}/api/related-systems`);
  if (!res.ok) throw new Error("Failed to load Related Systems");
  const json = await res.json();
  return json.data;
}

export async function createTicket(
  requesterId: number,
  formData: FormData
): Promise {
  const res = await fetch(`${API_URL}/api/tickets`, {
    method: "POST",
    headers: {
      "x-requester-id": requesterId.toString(),
    },
    body: formData,
  });

  const json = await res.json();
  if (!res.ok) {
    const errorMsg = json.error?.message || "Failed to create ticket";
    const details = json.error?.details;
    const error = new Error(errorMsg) as Error & { details?: Record };
  }
}
```

```

        error.details = details;
        throw error;
    }
    return json.data;
}

export async function fetchTickets(
    requesterId: number,
    params: {
        search?: string;
        category?: string;
        priority?: string;
        status?: string;
        page?: number;
        limit?: number;
        sortBy?: string;
        sortOrder?: string;
    } = {}
): Promise {
    const query = new URLSearchParams();
    if (params.search) query.append("search", params.search);
    if (params.category && params.category !== "ALL") query.append("category", params.category);
    if (params.priority && params.priority !== "ALL") query.append("priority", params.priority);
    if (params.status && params.status !== "ALL") query.append("status", params.status);
    if (params.page) query.append("page", params.page.toString());
    if (params.limit) query.append("limit", params.limit.toString());
    if (params.sortBy) query.append("sortBy", params.sortBy);
    if (params.sortOrder) query.append("sortOrder", params.sortOrder);

    const res = await fetch(`${API_URL}/api/tickets?${query.toString()}`, {
        headers: {
            "x-requester-id": requesterId.toString(),
        },
    });

    const json = await res.json();
    if (!res.ok) throw new Error(json.error?.message || "Failed to fetch tickets");
    return json;
}

export async function fetchTicketDetail(
    requesterId: number,
    ticketId: number
): Promise {
    const res = await fetch(`${API_URL}/api/tickets/${ticketId}`, {
        headers: {
            "x-requester-id": requesterId.toString(),
        },
    });

    const json = await res.json();
    if (!res.ok) {
        const error = new Error(json.error?.message || "Failed to fetch ticket detail");
        (error as any).status = res.status;
        (error as any).code = json.error?.code;
        throw error;
    }
}

```

```

    }
    return json.data;
  }

export async function uploadAttachment(
  requesterId: number,
  ticketId: number,
  file: File
): Promise {
  const formData = new FormData();
  formData.append("file", file);

  const res = await fetch(`${API_URL}/api/tickets/${ticketId}/attachments`, {
    method: "POST",
    headers: {
      "x-requester-id": requesterId.toString(),
    },
    body: formData,
  });

  const json = await res.json();
  if (!res.ok) throw new Error(json.error?.message || "Failed to upload attachment");
  return json.data;
}

export async function softRemoveAttachment(
  requesterId: number,
  attachmentId: number,
  reason: string
): Promise {
  const res = await fetch(`${API_URL}/api/attachments/${attachmentId}`, {
    method: "DELETE",
    headers: {
      "x-requester-id": requesterId.toString(),
      "Content-Type": "application/json",
    },
    body: JSON.stringify({ reason }),
  });

  const json = await res.json();
  if (!res.ok) throw new Error(json.error?.message || "Failed to remove attachment");
  return json.data;
}

export function getAttachmentDownloadUrl(attachmentId: number): string {
  return `${API_URL}/api/attachments/${attachmentId}/download;
}

```

6.3 Client TypeScript Types (`client/src/types.ts`)

```
export type Role = "REQUESTER" | "IT_STAFF" | "ADMINISTRATOR";

export interface User {
  id: number;
  email: string;
  name: string;
  role: Role;
  mustChangePassword: boolean;
  isActive?: boolean;
}

export interface AuthResponse {
  success: boolean;
  token: string;
  user: User;
}

export interface RequesterUser {
  id: number;
  name: string;
  email: string;
  department: string;
}

export interface Category {
  id: number;
  name: string;
}

export interface RelatedSystem {
  id: number;
  name: string;
}

export type Priority = "LOW" | "MEDIUM" | "HIGH" | "URGENT";
export type Status = "NEW" | "IN_PROGRESS" | "RESOLVED" | "CLOSED";

export interface Attachment {
  id: number;
  ticketId: number;
  fileName: string;
  fileKey: string;
  fileSize: number;
  mimeType: string;
  isRemoved: boolean;
  removalReason?: string | null;
  removedAt?: string | null;
  createdAt: string;
}

export interface Ticket {
  id: number;
  ticketNo: string;
  requesterId: number;
```

```

    categoryId: number;
    relatedSystemId: number;
    requestedPriority: Priority;
    status: Status;
    summary: string;
    description: string;
    createdAt: string;
    updatedAt: string;
    category: Category;
    relatedSystem: RelatedSystem;
    requester?: RequesterUser;
    attachments?: Attachment[];
    attachmentCount?: number;
  }

export interface PaginationMeta {
  page: number;
  limit: number;
  totalItems: number;
  totalPages: number;
}

export interface TicketListResponse {
  success: boolean;
  data: Ticket[];
  meta: PaginationMeta;
}

```

6.4 Login Screen Page (`client/src/pages/Login.tsx`)

```

import React, { useState } from "react";
import { useAuth } from "../contexts/AuthContext";

interface LoginProps {
  onSuccess?: () => void;
  onSwitchToLegacy?: () => void;
}

export const Login: React.FC = ({ onSuccess, onSwitchToLegacy }) => {
  const { login } = useAuth();
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState(null);
  const [submitting, setSubmitting] = useState(false);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);

    if (!email.trim() || !password) {
      setError("Please fill in both Email and Password fields.");
      return;
    }

    setSubmitting(true);
    try {
      await login(email.trim(), password);
      if (onSuccess) {
        onSuccess();
      }
    } catch (err: any) {
      setError(err.message || "Invalid credentials or account deactivated.");
    } finally {
      setSubmitting(false);
    }
  };

  return (

```



```

    { /* Hot Pink Heading as requested by user */ }

```

Sign in to TokTickIT

Select Development Requester or enter your credentials to access IT Services

```
{error && (
```

```
{error}
```

```
)}
```

Email Address

```
{email} setEmail(e.target.value)}  
disabled={submitting}  
autoFocus  
</>
```

Password

```
..... setPassword(e.target.value)}  
disabled={submitting}  
</>
```

{submitting ? "Signing in..." : "Sign In"}

```
{onSwitchToLegacy && (
```

Select Development Requester (Lab 1/2 Mode)

}}

TokTickIT Access Management System • Zen Green Theme

);
};

6.5 Mandatory Change Password Screen Page (`client/src/pages/ChangePassword.tsx`)

```

import React, { useState } from "react";
import { useAuth } from "../contexts/AuthContext";

interface ChangePasswordProps {
  isMandatory?: boolean;
  onSuccess?: () => void;
  onCancel?: () => void;
}

export const ChangePassword: React.FC = ({
  isMandatory = false,
  onSuccess,
  onCancel,
}) => {
  const { changePassword } = useAuth();
  const [currentPassword, setCurrentPassword] = useState("");
  const [newPassword, setNewPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [error, setError] = useState(null);
  const [successMsg, setSuccessMsg] = useState(null);
  const [submitting, setSubmitting] = useState(false);

  // Real-time validation checks
  const hasMinLength = newPassword.length >= 8;
  const hasUpper = /[A-Z]/.test(newPassword);
  const hasLower = /[a-z]/.test(newPassword);
  const hasNumberOrSpecial = /[0-9!@#$$%^&*(),.?:{}|<>]/.test(newPassword);
  const matchesConfirm = newPassword === confirmPassword && confirmPassword.length > 0;

  const isValidForm =
    currentPassword.length > 0 &&
    hasMinLength &&
    hasUpper &&
    hasLower &&
    hasNumberOrSpecial &&
    matchesConfirm;

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);
    setSuccessMsg(null);

    if (!isValidForm) {
      if (newPassword !== confirmPassword) {
        setError("New password and confirmation do not match.");
      } else {
        setError("Please ensure your new password meets all security criteria.");
      }
      return;
    }

    setSubmitting(true);
    try {
      await changePassword(currentPassword, newPassword);
    }
  }

```

```

        setSuccessMsg("Password changed successfully!");
        if (onSuccess) {
            setTimeout(onSuccess, 1000);
        }
    } catch (err: any) {
        setError(err.message || "Failed to update password.");
    } finally {
        setSubmitting(false);
    }
};

return (

```

```

        { /* Hot Pink Heading as requested by user */ }

```

```

        {isMandatory ? "Mandatory Password Update" : "Change Password"}

```

```

        {isMandatory
          ? "You must update your initial password before accessing TokTickIT."
          : "Update your account password to maintain security."}

```

```

        {error && (

```

```

            {error}

```

```

        )}

```

```

        {successMsg && (

```

```

            {successMsg}

```

```

        )}

```

Current Password

```
 setCurrentPassword(e.target.value)}  
  disabled={submitting}  
  autoFocus  
</>
```

New Password

```
 setNewPassword(e.target.value)}  
  disabled={submitting}  
</>
```

Confirm New Password

```
 setConfirmPassword(e.target.value)}  
  disabled={submitting}  
</>
```

Password Criteria

- `{hasMinLength ? "[✓]" : "[ ]"}` At least 8 characters long
- `{hasUpper && hasLower ? "[✓]" : "[ ]"}` Upper & lower case letters

- `{hasNumberOrSpecial ? "[✓]" : "[ ]"}` Number & special character
- `{matchesConfirm ? "[✓]" : "[ ]"}` Confirmation matches new password

```
{!isMandatory && onCancel && (
```

Cancel

```
)}
```

`{submitting ? "Updating..." : "Save New Password"}`

```
);  
};
```

6.6 Header Navigation Bar Component (`client/src/components/Header.tsx`)

```

import React from "react";
import { RequesterUser, User } from "../types";

export type NavTab =
  | "my-tickets"
  | "create-ticket"
  | "staff-queue"
  | "my-queue"
  | "user-management";

interface HeaderProps {
  currentRequester?: RequesterUser | null;
  authUser?: User | null;
  activeTab: string;
  onTabChange: (tab: NavTab) => void;
  onChangeRequester?: () => void;
  onChangePassword?: () => void;
  onLogout?: () => void;
  onLoginClick?: () => void;
}

export const Header: React.FC = ({
  currentRequester,
  authUser,
  activeTab,
  onTabChange,
  onChangeRequester,
  onChangePassword,
  onLogout,
  onLoginClick,
}) => {
  const role = authUser?.role;

  const renderRoleBadge = () => {
    if (!role) return null;
    switch (role) {
      case "REQUESTER":
        return Requester;
      case "IT_STAFF":
        return IT Staff;
      case "ADMINISTRATOR":
        return Administrator;
      default:
        return null;
    }
  };

  return (

    { /* Brand & Identity */ }

```

```

{
  if (role === "ADMINISTRATOR") onTabChange("user-management");
  else if (role === "IT_STAFF") onTabChange("staff-queue");
  else onTabChange("my-tickets");
}}

```

>



TokTickIT

```

{/* Role-based Navigation Tabs */}

```

```

{(!authUser || role === "REQUESTER") && (

```

```

  <>

```

```

    nav-link border-0 bg-transparent d-flex align-items-center gap-1 ${
      activeTab === "my-tickets" ? "active" : ""
    }
    onClick={() => onTabChange("my-tickets")}
    >
      
      My Tickets

```

```

    nav-link border-0 bg-transparent d-flex align-items-center gap-1 ${
      activeTab === "create-ticket" ? "active" : ""
    }
    onClick={() => onTabChange("create-ticket")}
    >
      +
      Create Ticket

```

```

  })

```

```

{role === "IT_STAFF" && (

```

```

  <>

```

```

    nav-link border-0 bg-transparent d-flex align-items-center gap-1 ${
      activeTab === "staff-queue" ? "active" : ""
    }
    onClick={() => onTabChange("staff-queue")}
    >
      Ticket Queue

```

```

    nav-link border-0 bg-transparent d-flex align-items-center gap-1 ${
      activeTab === "my-queue" ? "active" : ""
    }
    onClick={() => onTabChange("my-queue")}
    >
      My Queue

```

```

    nav-link border-0 bg-transparent d-flex align-items-center gap-1 ${
      activeTab === "create-ticket" ? "active" : ""
    }
    onClick={() => onTabChange("create-ticket")}
    >
      Create Ticket

```

```

  })

```

```

{role === "ADMINISTRATOR" && (
  <>
    nav-link border-0 bg-transparent d-flex align-items-center gap-1 ${
      activeTab === "user-management" ? "active" : ""
    }
    onClick={() => onTabChange("user-management")}
    >
      User Management
    </nav-link>
    nav-link border-0 bg-transparent d-flex align-items-center gap-1 ${
      activeTab === "staff-queue" ? "active" : ""
    }
    onClick={() => onTabChange("staff-queue")}
    >
      Ticket Queue
    </nav-link>
  </>
)}

```

```

{/* User Identity & Actions */}

```

```

{authUser ? (

```

```

  <div>
    <img alt="User icon" data-bbox="212 468 230 482"/>
    {authUser.name}
    </div>
    <div>
      {renderRoleBadge()}
      {onChangePassword && (
        <button>Change Password</button>
      )}
      {onLogout && (
        <button>Logout</button>
      )}
    </div>

```

```

) : currentRequester ? (

```

```

  <div>
    <img alt="User icon" data-bbox="212 768 230 782"/>
    {currentRequester.name}
    </div>
    <div>
      {currentRequester.department}
      {onChangeRequester && (
        <button>Change Requester</button>
      )}
    </div>

```

```

) : (

```

```
Guest Context
{onLoginClick && (
```

Sign In

```
)}
```

```
)}
```

```
);
};
```

6.7 Create Ticket View Component (`client/src/components/CreateTicketView.tsx`)

```

import React, { useState, useEffect } from "react";
import { RequesterUser, Category, RelatedSystem, Priority, Ticket } from "../types";
import { fetchCategories, fetchRelatedSystems, createTicket } from "../api";

interface CreateTicketViewProps {
  currentRequester: RequesterUser;
  onTicketCreated: (ticket: Ticket) => void;
  onCancel: () => void;
}

export const CreateTicketView: React.FC = ({
  currentRequester,
  onTicketCreated,
  onCancel,
}) => {
  const [categories, setCategories] = useState([]);
  const [systems, setSystems] = useState([]);
  const [loadingRefData, setLoadingRefData] = useState(true);

  // Form State
  const [categoryId, setCategoryId] = useState("");
  const [relatedSystemId, setRelatedSystemId] = useState("");
  const [requestedPriority, setRequestedPriority] = useState("MEDIUM");
  const [summary, setSummary] = useState("");
  const [description, setDescription] = useState("");
  const [selectedFiles, setSelectedFiles] = useState([]);

  // UI & Validation State
  const [submitting, setSubmitting] = useState(false);
  const [fieldErrors, setFieldErrors] = useState({});
  const [globalError, setGlobalError] = useState(null);
  const [fileError, setFileError] = useState(null);
  const [createdTicket, setCreatedTicket] = useState(null);

  useEffect(() => {
    Promise.all([fetchCategories(), fetchRelatedSystems()])
      .then(([cats, sys]) => {
        setCategories(cats);
        setSystems(sys);
        if (cats.length > 0) setCategoryId(cats[0].id.toString());
        if (sys.length > 0) setRelatedSystemId(sys[0].id.toString());
      })
      .catch((err) => setGlobalError(err.message))
      .finally(() => setLoadingRefData(false));
  }, []);

  const handleFileChange = (e: React.ChangeEvent) => {
    setFileError(null);
    if (!e.target.files) return;
    const newFiles = Array.from(e.target.files);

    const allowedMime = ["image/jpeg", "image/png", "image/webp", "application/pdf"];
    let err: string | null = null;

```

```

if (selectedFiles.length + newFiles.length > 5) {
  err = "Maximum limit of 5 attachments per ticket reached";
}

const validFiles: File[] = [];
for (const f of newFiles) {
  if (!allowedMime.includes(f.type)) {
    err = Invalid file type for '${f.name}'. Allowed: JPG, PNG, WEBP, PDF;
    break;
  }
  if (f.size > 5 * 1024 * 1024) {
    err = File '${f.name}' exceeds maximum size of 5MB;
    break;
  }
  validFiles.push(f);
}

if (err) {
  setFileError(err);
} else {
  setSelectedFiles((prev) => [...prev, ...validFiles]);
}
};

const removeFile = (index: number) => {
  setSelectedFiles((prev) => prev.filter((_, i) => i !== index));
  setFileError(null);
};

const validate = (): boolean => {
  const errors: Record = {};
  if (!categoryId) errors.categoryId = "Category is required";
  if (!relatedSystemId) errors.relatedSystemId = "Related system is required";
  if (!summary || summary.trim().length < 5 || summary.trim().length > 150) {
    errors.summary = "Summary must be between 5 and 150 characters";
  }
  if (!description || description.trim().length < 10 || description.trim().length > 2000) {
    errors.description = "Description must be between 10 and 2000 characters";
  }

  setFieldErrors(errors);
  return Object.keys(errors).length === 0;
};

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  setGlobalError(null);
  if (!validate()) return;

  setSubmitting(true);
  try {
    const formData = new FormData();
    formData.append("categoryId", categoryId);
    formData.append("relatedSystemId", relatedSystemId);
    formData.append("requestedPriority", requestedPriority);
    formData.append("summary", summary);
  }

```

```
formData.append("description", description);
selectedFiles.forEach((file) => formData.append("files", file));

const ticket = await createTicket(currentRequester.id, formData);
setCreatedTicket(ticket);
} catch (err: any) {
  if (err.details) {
    const mapped: Record = {};
    Object.keys(err.details).forEach((key) => {
      mapped[key] = Array.isArray(err.details[key]) ? err.details[key][0] : err.details[key];
    });
    setFieldErrors(mapped);
  }
  setGlobalError(err.message || "Failed to create ticket");
} finally {
  setSubmitting(false);
}
};

if (createdTicket) {
  return (
```



Ticket Created Successfully!

Your IT support request has been recorded.

Ticket Number:

{createdTicket.ticketNo}

Status:

NEW

Summary:

{createdTicket.summary}

```
{
  setCreatedTicket(null);
  setSummary("");
  setDescription("");
  setSelectedFiles([]);
}
```

+ Create Another Ticket

```
onTicketCreated(createdTicket)}
>
📋 View My Tickets
```

```
);
}
```

```
return (
```

Create IT Support Ticket

← Back to My Tickets

```
{globalError && (
```



```
{globalError}
```

```
)}
```

```
{loadingRefData ? (
```

Loading reference data...

```
) : (
```

```
{/* System Generated Fields */}
```

1. Ticket Metadata (Read-Only)

Ticket No.

Auto-generated after submis

Requester

```
{<code ${currentRequester.name} (${currentRequester.departme
```

```
nt}}}
```

disabled

Initial Status

New

```
{/* Classification */}
```

2. Problem Classification

Category *

{c.name} ▼

{fieldErrors.categoryId && (

{fieldErrors.categoryId}

)}

Related System *

{s.name} ▼

{fieldErrors.relatedSystemId && (

{fieldErrors.relatedSystemId}

)}

Requested Priority

Low ▼

```
{/* Problem Details */}
```

3. Problem Details

Ticket Summary *

```
{summary.length} / 150
```

```
    <input type="text" form-control ${fieldErrors.summary ? "is-invalid" : ""}
      placeholder="Briefly describe the issue (e.g. Laptop battery drains quickly)"
      value={summary}
      maxLength={150}
      onChange={(e) => setSummary(e.target.value)}
    />
    {fieldErrors.summary && (
      {fieldErrors.summary}
    )}
```

Description *

```
{description.length} / 2000
```

```
    <input type="text" form-control ${fieldErrors.description ? "is-
```